30082

Oscilloskop og Signalgenerator



Xingjia
s255327
Oscilloskop



Pelle
s255327
Oscilloskop



Christian
s255327
FPGA



Eva
s255327
FPGA

G8 GROUP

XILINX. ISE DESIGN SUITE

MICROCHIP STUDIO FOR AVR® AND SAM DEVICES

Indholdsfortegnelse

0.1 Indledning 2
0.2 Opsætningen af Kredsløbet for Oscilloskopet og Signalgeneratoren 3
0.3 Kravsspecifikationer 5
0.3.1 Oscilloskop Krav 5
0.3.2 Signalgenerator Krav 5
0.3.3 Filter Krav 6
0.3.4 Opfyldte testkrav for Oscilloskop og Signalgenerator 6
1. Oscilloskop 7
1.1 Brugervejledning og introduktion til Labview 7
1.1.1 How to Labview 7
1.1.2 Forståelse for Labviews datapakker 10
1.2 Moduldiagram 11
1.3 Design af Oscilloskopet 11
1.3.1 ADC Konvertering og Timer1 (Design af Samplerate) 11
1.3.2 Begrænsninger og Minimum Record Length 13
1.3.3 UART0 og Acknowledge Byte 15
1.3.4 Max Record length 16
1.3.5 Kontinuert Uafbrudte Målinger 16
1.3.6 Brug af interrupt og polling + interrupt diagram 16
1.3.7 SPI-kommunikation 19
1.4 Flow Chart for Main.c 20
1.5 Bodeplot 22
2. Signalgenerator 25
2.1 SigGenTop 25
2.2 SigGenKontrolUpdate 27
2.3 SigGenDataPath 32
2.4 SigGenTop.ucf 40
2.5 DispMux & SevenSeg5 41
3. Filter 42
4. Test 47

4.1 Oscilloscope test 47
4.1.1 Labview Generator Panel 47
4.1.2 Labview Generator Kommandofunktionen 49
4.1.3 Sample Rate and Record Length Control 50
4.2 FPGA og filter test 51
4.2.1 Signaler ved hhv. 24Hz og 24 kHz 51
4.2.2 Sinussignaler ved forskellige frekvenser 63
4.3 Integration mellem MCU, FPGA og Filtre 70
4.3.1 SPI Test 70
4.3.2 Dataintegritet 70
4.3.3 Acknowledge Byte 72
4.3.4 Bode Plot 74
5. Konklusion 75
6. Koden Bilag 77
6.1 Koden Bilag for Oscilloskop 77
6.2 Koden Bilag for Signalgenerator 90

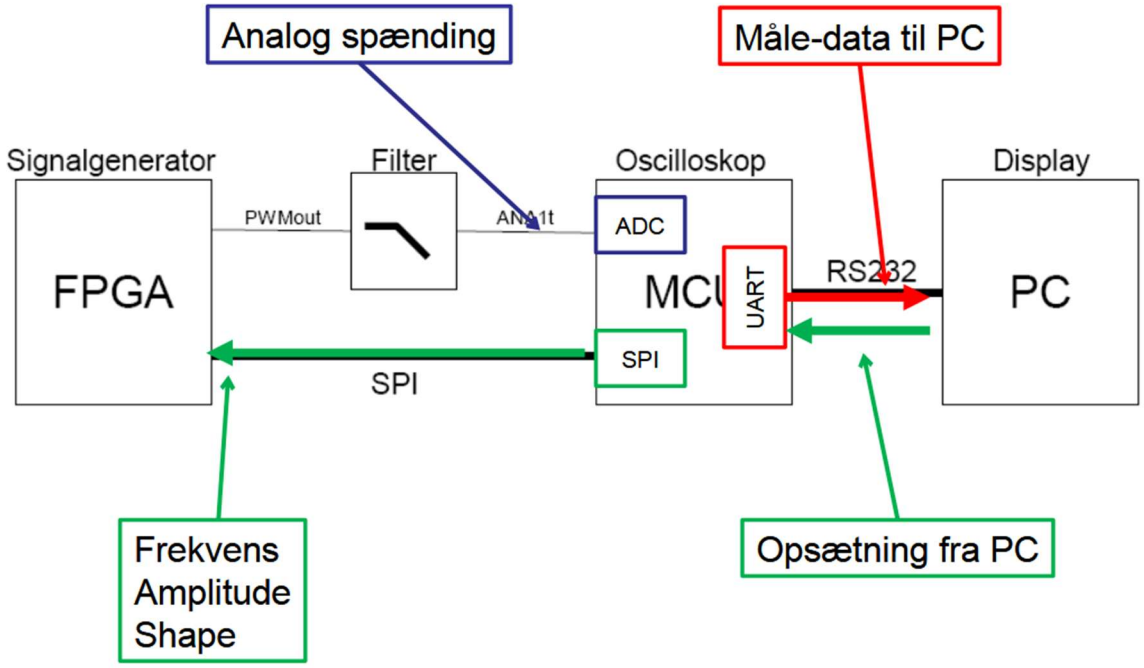
0.1 Indledning

Rapporten omhandler udviklingen af et oscilloskop, med et Atmega2560 board, samt en signalgenerator med et FPGA Basys 2 board. I rapporten refereres oscilloskopet til som MCUen, og signalgeneratoren som FPGAen.

Rapporten er opdelt for oscilloskop, signalgenerator, filter og test. I disse dele vil rapportens pålagte kravspecifikationer redegøres for. Kravspecifikationerne rapporten kan aflæses i 0.2 kravsspecifikationer. Rapporten afsluttes med en konklusion og bilag. Bilagene indeholder koden for vores c-moduler og VHDL-moduler.

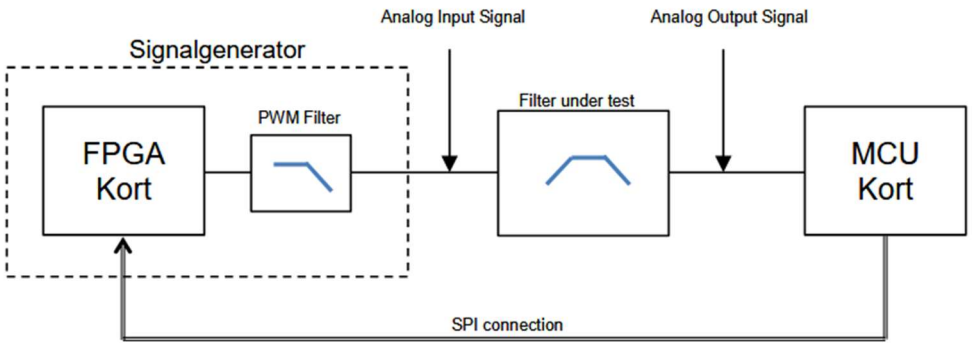
0.2 Opsætningen af Kredsløbet for Oscilloskopet og Signalgeneratoren

Vores opsætning for kredsløbet ændres efter hvad vi tester for. Billedet herunder, viser kredsløb er opsætning 'standard' opsætning, se figur 1.



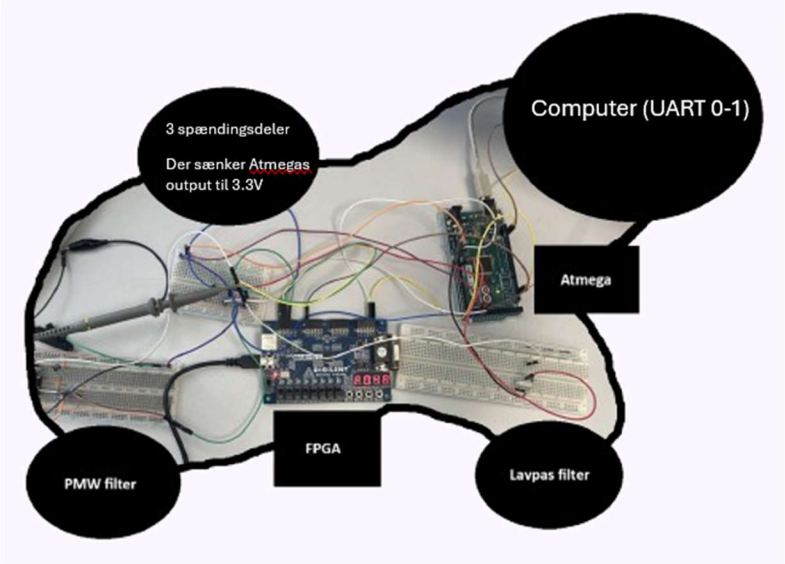
Figur 1. Opsætning af kredsløbet.

Vores opsætning når et lavpasfilter testes med vores bodeplot-funktionen, se figur 2. Betydning af bodeplot-funktionen forklares i afsnit 1.5.



Figur 2. Kredsløb med filter.

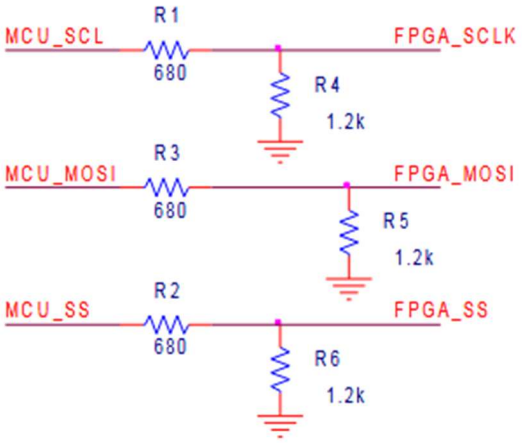
Billedet herunder viser den virkelige opsætning, af kredsløbet, når et lavpas-filter testes, se figur 3. Bemærk forbindelserne mellem MCUen og FPGAen (SPI) går igennem 3 spændingsdeler.



Figur 3. Forsøgsopsætning

Spændingsdeler

Atmegaen outputter 5V fra dens porte (5V digital høj Atmega). Og FPGAens porte er sat til 3.3V (3.3V digital høj FPGA). Det er lavet 3 spændingsdeler til at sænke Atmegaens digitale høje outputs fra 5V -> 3.3V. Se figur, 4.



Figur 4. Spændingsdeler.

0.3 Kravsspecifikationer

0.3.1 Oscilloskop Krav

- ADCkonvertering, med 8-bit opløsning, for målte spændinger fra 0V til 3.3V
- Oscilloskopet skal kunne lave kontinuere, ubrudte målinger indenfor de pålagte parameterkrav.
- Oscilloskopet parameter skal kunne indstilles fra Labview
- Parameterkrav: for min- og max samplerate = fra 10 sps til 10000 sps.
- Parameterkrav: for min- og max record length = fra 10 ADC mål per. pakke til 1000 ADC mål per. pakke.
- Parameterkrav: minimum record length - beregnet og implementeret.
- Moduldiagram for oscilloskopets c-kode.
- Interruptdiagram for oscilloskopets c-kode.
- Flowchart for oscilloskopets Main.c.
- Protokolleret acknowledge byte.
- Læsning af protokolleret datapakker i terminal. Og modtagelse af data fra FPGA i terminal.

Oscilloskop ekstra krav

- Bodeplot funktion for osciloskop
- MCU viser handshake byte fra FPGA på UART0 (PC-terminal). Samtidig viser MCU antal felj på UART0 (PC-terminal).

0.3.2 Signalgenerator Krav

- Skal kunne modtage protokol med data af shape, amplitude og frekvens
- SPI-forbindelse skal køre med baudrate på 500 kbaud
- Test af sikker SPI-forbindelse
- Relevante funktions-, flowcharts- og tilstandsdiagrammer

Ekstra krav:

- Shape, amplitude og frekvens gøres synligt på 7-segment display
- Sinus look-up tabel
- 2-vejskommunikation med acknowledge handshake
- Ny signalform i signalgeneratoren

0.3.3 Filter Krav

- Designed passende filter til udglatning af PWM-pulser

0.3.4 Opfyldte testkrav for Oscilloskop og Signalgenerator

Oscilloskop

- Parametre
- Samplerate; 10sps – 10000sps
- Record length fra 10 til 1000 ADC / Pakke

Signalgenerator

- Lavpas filter samt aktivt filter
- Parametre indstilles fra labview
- SHAPE, AMPL, FREQ vises på 7seg
- Sinussignal fra signalgenerator
- SPI-test demonstrering sikker forbindelse
- Generering af nye signaler testet og bekræftet, inverteret savtak og trekant

1. Oscilloskop

1.1 Brugervejledning og introduktion til Labview

I afsnittet forklares hvordan Labview styres af en bruger. Og hvordan brugerens brug af Labview oversættes til indstillingen af oscilloskopet og signalgeneratoren. Og forklaring af Labview-datapakker.

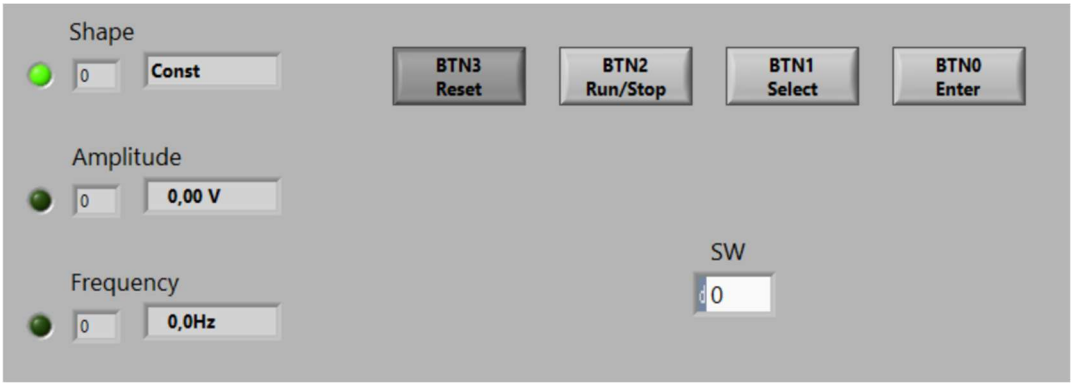
1.1.1 How to Labview

Indstillingen af signalgeneratoren og oscilloskopet gøres gennem Labview.

Labview har tre menuer; generator-menuen, oscilloskops-menuen og bodeplot-menuen.

I generator-menuen findes fire knapper; enter, select, run/stop og reset. Generator-menuen bruges til indstilling for det ønsket signal - som signalgeneratoren skal udføre.

Select bruges til at 'bladre' i generator-menuen. Og det markeres med den grønne 'lampe', se figur 5.



Figur 5. LabView menu.

I shape kan man vælge at udføre seks forskellige signaltyper. SW=0 for constant, SW=1 for square, SW=2 for sawtooth, SW=3 for sinus, SW=4 for trekant og SW=5 for inverteret sawtooth. For at gemme en SW-værdi skal der trykkes enter - efter man har indtastet en SW-

værdi. Signalgeneratoren kan udføre et signal med en amplitude for 0 til 3.3V. Og det signal kan have en frekvens mellem 0-23.7kHz.

Amplituden og frekvensen repræsenteres med én enkelt databyte i den datapakke Labview sender til signalgeneratoren. Amplituden og frekvensen bestemmes derfor med et tal fra 0 til 255 i SW-feltet.

Brugeren kan bruge disse formlerne for valg af amplitude og frekvens: $Amp = \frac{SW}{255} \cdot 3.3V$, $Freq = \frac{SW}{255} \cdot 23.7kHz$. Amplituden- og frekvensens valgte SW-værdier displayes også på FPGAens sevenseg-display.

Bemærk; maksimumfrekvensen, $23.7kHz$, kommer fra hvordan SW-maksimum (255) fordeles i FPGAens 10-bits-register. Og derfra ganges med FPGAens frekvens på 24Hz. Regnestykket bliver $987 \cdot 24Hz \approx 24.7kHz$. Se figur 6.

$$F_{Digital} = [f_7\ f_6\ f_5\ f_4\ f_3\ f_2\ f_1\ f_0] \quad \text{binary}$$

$$F_{Frequency} = [0\ 0\ f_7\ f_6\ f_5\ f_4\ 0\ f_3\ f_2\ 0\ f_1\ f_0] * 24 \quad [Hz]$$

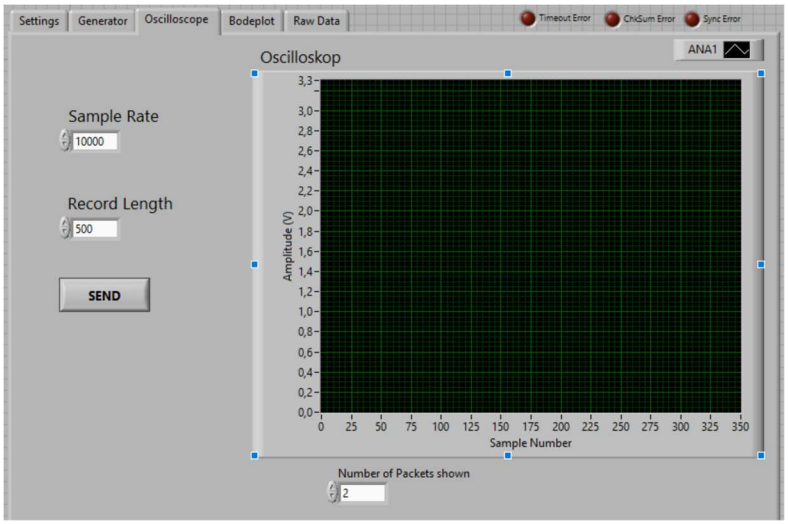
Figur 6. Frekvens omregning.

Reset nulstiller SW-værdierne i Labview og de tilsvarende registre i FPGAen.

Run/Stop tænder og slukker for signalgeneratoren.

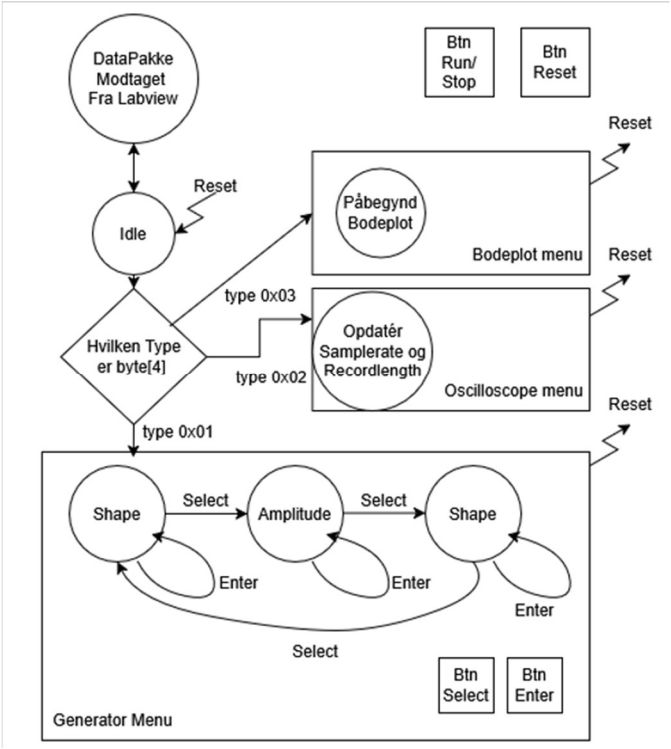
Oscillator-menuen, i Labview, skriver kun til MCUen. Her vælges sampleraten og record length for oscilloskopet.

Samplerate kan vælges for en værdi mellem 0-65536 samples per sekund. Det bemærkes at oscilloskopet er designet specifikt til at være funktionsdygtig med samplerate mellem 10-10kHz. Record length kan vælges til længde mellem 10-1000 ADC-målinger. Antallet af viste modtaget datapakker kan også vælges i menuen, se figur 7.



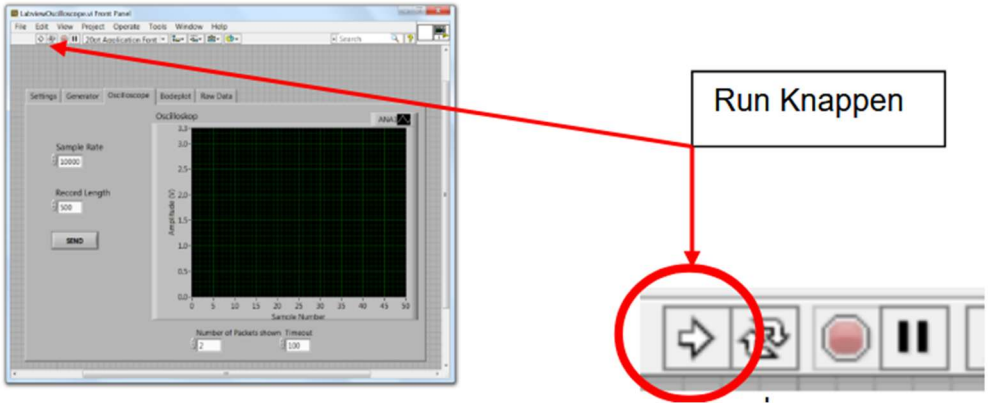
Figur 7. Oscilloskop i LabView.

Labviews bodeplot menuen har en knap; start. Det påbegynder Bodeplot funktionen. Frekvenser mellem 0-23.7k testes i forhold til et lavpas-filter. De tilsvarende SW-værdier, for frekvenserne, kan ses tælle op til 255 fra 0 - på FPGAens sevenseg-display. Labviews menuer kan visualiseres med dette tilstandsdiagram, se figur 8.



Figur 8. Tilstandsdiagram over LabViews menu.

Bemærk at PIL-knappen skal trykkes for at 'tænde' Labview, se figur 9.



Figur 9. LabView med instruks.

1.1.2 Forståelse for Labviews datapakker

Datapakker sendt eller modtage for Labview forekommer altid i en ens grundform, se figur 10. Hvis en ugyldig datapakke modtages af Labview vil pågældende error-lampe lyse rødt.

Packet Header			Packet Data			Packet Footer
Sync	Length	Type	Data 1	…..	Data N	Checksum
2 bytes	2 bytes	1 byte	N bytes			2 bytes

Figur 10. LabView protokol.

Databytene fra datapakkerne forstås på hexadecimal-form af Labview. En datapakke for Labview består altid af faste synkroniserings bytes og checksum bytes.

De to synkroniseringsbytes i hexadecimal-form: 55AA

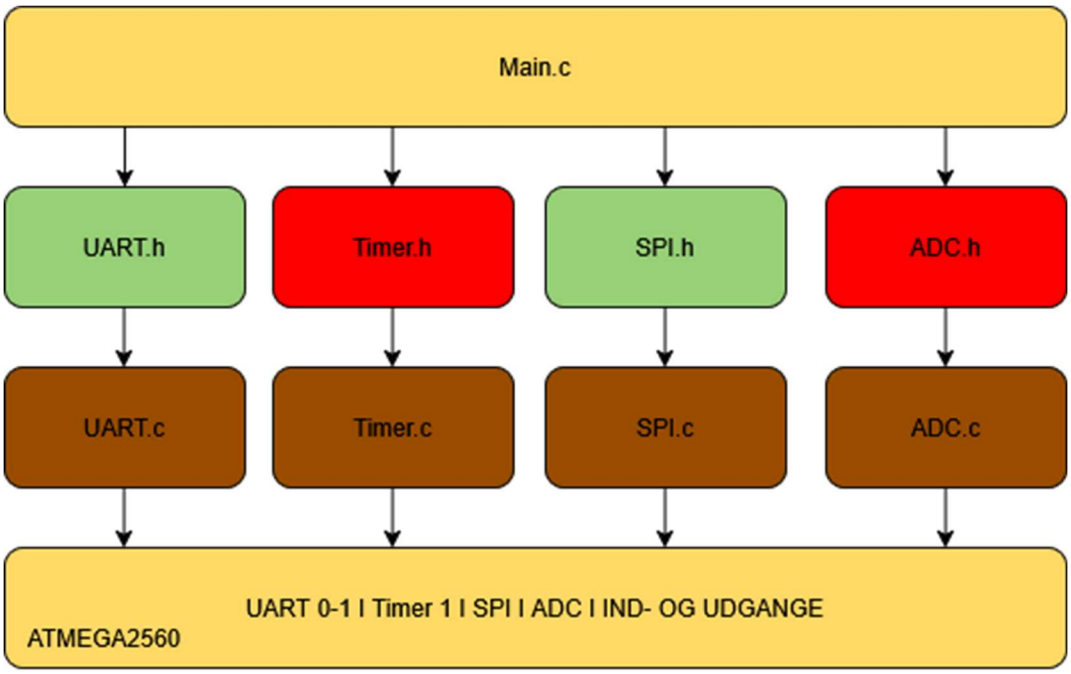
De to checksum bytes i hexadecimal-form: 0000

Totallængden bytesne og typebyten har fastepladser i datapakken.

Bemærk at den datapakke signalgeneratoren modtager fra Labview - følger ikke grundformen. Labview datapakke går først igennem en protokol i MCUen inden den bliver sendt til FPGAen. For mere information om protokollen; læs afsnit 1.3.7 SPI-kommunikation.

1.2 Moduldiagram

Moduldiagrammet herunder viser opdelingen af vores c-kode-moduler, se figur 11. Modulerne er opdelt efter de korresponderende periferienheder for Atmega 2560. Figur 11, viser så de periferienheder der bruges for MCUen.



Figur 11. Moduldiagram over c-koden.

1.3 Design af Oscilloskopet

I dette afsnit redegøres der for kodens design, valg af parameter og hvordan det stemmer overens med MCUens arkitektur/ virke. I afsnittet svares også på en kravspecifikationer oscilloskopet.

1.3.1 ADC Konvertering og Timer1 (Design af Samplerate)

ADCen har som standard en opløsning på 10-bits, for hver sampling, hvorfra vi kun bruger de 8 øverste bits (en byte). Opløsningen ADC-målingerne sænkes derfor en smule. Fordelen med den formindskede opløsning er mere effektiv UART-kommunikation. Fordi, UART

kommunikerer, ligeledes, med en byte store data pakker + 2 synkronisering bits. Med 10-bit opløsning ville ADC-målingen blive delt i to UART-datapakker. Den reduceret opløsning tillader også en meget højere clk-frekvens, på 1MHz, for ADC-konverteringen, se figur 12.

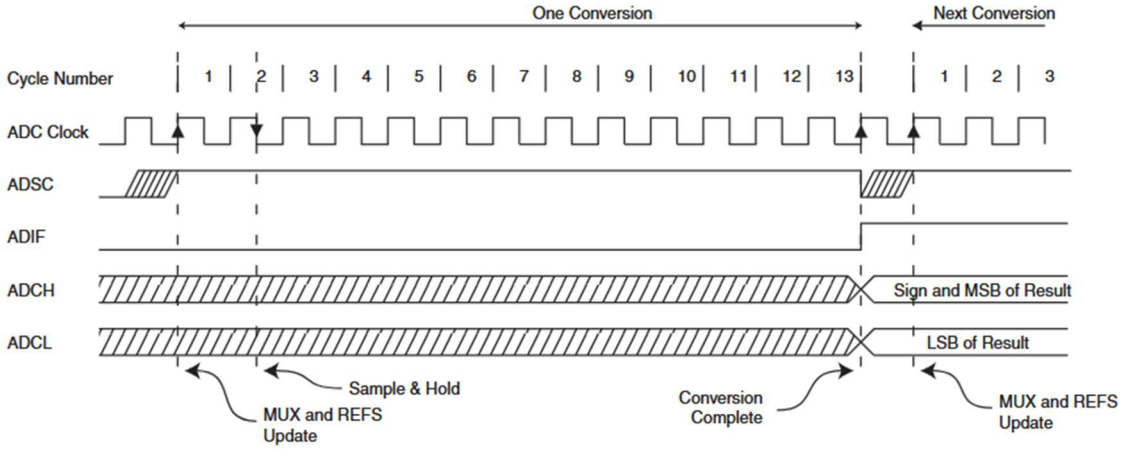
> By default, the successive approximation circuitry requires an input clock frequency between 50kHz and 200kHz. If a lower resolution than 10 bits is needed, the input clock frequency to the ADC can be as high as 1000kHz to get a higher sample rate.

Figur 12. Side 278 i MEGA2560 databog.

Vi benytter derfor en prescaler på 16 for MCUens frekvens: $\frac{16000000\,Hz}{16} = 1MHz$.

En ADC-konvertering tager bruger 13 ADC-clk-slag, på $1MHz$, da vi bruger single conversion mode for ADCen, se figur 13.

Figure 26-5. ADC Timing Diagram, Single Conversion



Figur 13 Single Conversion Mode, s.280 databog atmega2560

En ADC-konvertering tager derfor: $13 \cdot \frac{1}{1000000} s = 13\mu s$.

ADCens samplerate bestemmes af brugeren fra Labview interfacet. Den valgte samplerate indlæses i ORCA1. ORCA1's værdi bliver bestemt ud fra formlen $\frac{F_timer}{bruger_input} - 1$. Og hvor:

$F_timer = \frac{16000000Hz}{64} = 250000Hz$.

ORCA1 er topvalue for Timer1. Hvor Timer1s countvalue, TCN1, er for hurtigt Timer1 tæller op til topvalue. Efter topvalue er nået bliver countvaluen for TCN1 nulstillet. TCN1 tællerfrekvens sættes konstant med denne formel: $\frac{1}{F_timer}$.

Regnestykket med OCRA1 og TCN1 sammensat: $\frac{F_timer}{OCR1A+1} = samplerate$

Timer1 initialiseres med CTC-mode (clear timer og compare). CTC fungerer sådan; nå Timer1s countvalue når den givet topvalue vil ISR, for Timer1, trigges. Under Timer1 ISR sættes ADCen i gang med en ny konvertering vead at sætte ADSC bit til 1.

ISR sikre at compare matchet bliver op-prioteret i CPUen - så timing-fejl ikke forekommer. Timer1 er et 16-bitregister. Og kombination med F_timer på 250000Hz tillader det oscilloskopet at kunne fremstille en frekvens ned 10Hz: $\frac{250000}{10} - 1 = 27777.8 <$ 65536 ($maksimum\ tal\ en$ 16 bit register kan holde)

Det vides også at en ADC-konvertering, ved 10khz, kan nå at finde sted mellem Timer1. $100\mu s - 13\mu s = 87\mu s$.

1.3.2 Begrænsninger og Minimum Record Length

Vores Oscilloskop datamæssige begrænsende faktor er, hovedsageligt, UART-kommunikationen mellem Atmega boardet og Labview. Deres UART-kommunikations baudrate er sat til 115200bps.

Ifølge databogen er vores timing-error på 2.1%, se figur 14. Det bemærkes at vores UART-kommunikation er lavet dobbelt speed. UARTens UBRR-værdi er derfor ≈ 16.

Table 22-12. Examples of UBRRn Settings for Commonly Used Oscillator Frequencies

Baud Rate (bps)	f_{osc} = 16.0000MHz				f_{osc} = 18.4320MHz				f_{osc} = 20.0000MHz			
	U2Xn = 0		U2Xn = 1		U2Xn = 0		U2Xn = 1		U2Xn = 0		U2Xn = 1	
	UBRR	Error	UBRR	Error	UBRR	Error	UBRR	Error	UBRR	Error	UBRR	Error
2400	416	-0.1%	832	0.0%	479	0.0%	959	0.0%	520	0.0%	1041	0.0%
4800	207	0.2%	416	-0.1%	239	0.0%	479	0.0%	259	0.2%	520	0.0%
9600	103	0.2%	207	0.2%	119	0.0%	239	0.0%	129	0.2%	259	0.2%
14.4K	68	0.6%	138	-0.1%	79	0.0%	159	0.0%	86	-0.2%	173	-0.2%
19.2K	51	0.2%	103	0.2%	59	0.0%	119	0.0%	64	0.2%	129	0.2%
28.8K	34	-0.8%	68	0.6%	39	0.0%	79	0.0%	42	0.9%	86	-0.2%
38.4K	25	0.2%	51	0.2%	29	0.0%	59	0.0%	32	-1.4%	64	0.2%
57.6K	16	2.1%	34	-0.8%	19	0.0%	39	0.0%	21	-1.4%	42	0.9%
76.8K	12	0.2%	25	0.2%	14	0.0%	29	0.0%	15	1.7%	32	-1.4%
115.2K	8	-3.5%	16	2.1%	9	0.0%	19	0.0%	10	-1.4%	21	-1.4%
230.4K	3	8.5%	8	-3.5%	4	0.0%	9	0.0%	4	8.5%	10	-1.4%
250K	3	0.0%	7	0.0%	4	-7.8%	8	2.4%	4	0.0%	9	0.0%
0.5M	1	0.0%	3	0.0%	–	–	4	-7.8%	–	–	4	0.0%
1M	0	0.0%	1	0.0%	–	–	–	–	–	–	–	–
Max.(1)	1Mbps		2Mbps		1.152Mbps		2.304Mbps		1.25Mbps		2.5Mbps	

Note: 1. UBRR = 0, Error = 0.0%

Figur 14 UBBR-indstillinger for frekvenser

Typisk anbefales en error-rate på >3%. Det overholder vi med en baud på 115200bps. Den relative lave error-rate kommer en ulempe: sænket hastig for UART-kommunikationen.

Mellem hver sampling fyldes en, af to, buffere med en databyte. Problemet kommer med tømning af bufferen - mere information om dette i afsnit 1.3.5. Tømningen af bufferne gøres med UART.

UART kan ikke nå at tømme en buffer, når; record length er for lav i forhold til sampleraten. Pågældende buffer kan risikere at få sine værdier overskrevet, af nye ADC-værdier, imens den er i gang med at sende den aktuelle måling.

I ligningen herunder, bestemmes den minimale record length for den maksimale samplerate på 10kHz, se figur x. Bemærk UART kræver 10 bits per byte sendt. Og 7 bytes er reserveret i bufferen for synk-, checksum-, type- og længe bytes. Se figur 15.

$$solve\left(\left(\frac{(r+7)\cdot 10}{115200} < \frac{r}{10000}\right), r\right) = (46.05, \infty)$$

Figur 15 minimum record length 10kHz

Record length skal så være på minimum 47 for 10kHz - hvis UART skal kunne følge med.

I Koden bruger vi denne formel til at bestemme minimum record length:

$$MinimumRecordLegnth = \frac{70 * SampleRate + (115.2K - 10 * SampleRate) - 1}{115.2K - 10 * SampleRate}$$

Denne formel runde altid minimum record length op - f.eks. 46.05 -> 47. For c-koden vil nemlig ellers automatisk runde 46.05 ned til 46.

Vores kode er så sat op til at hæve record length automatisk, med MinimumRecordLength-formelen, hvis brugerens input for record length er lav i forhold til den valgte samplerate. Dette demonstreres i test-afsnit 4.1.3.

1.3.3 UART0 og Acknowledge Byte

UART0 bruges som en sikkerhedsforanstaltning for oscilloskopet. Alle datapakker der transmitterer, fra MCU til FPGA, indlæses også i computerens terminal - med UART0. Brugeren kan så se hvordan datapakken ser ud efter den filtreres gennem MCUens SPI-transmissions protokol - læs afsnit 1.3.7 for MCU protokol. Og dermed tjekke om den pågældende datapakke stemmer overens med hvad der forventes ad datapakken.

UART0 bruges også til at vise hvad FPGAen sender tilbage med SPI i terminalen. Hvis MCUens protokolleret-datapakke stemmer overens med FPGAens egen protokol - sendes en byte (0x00) til MCUen.

Acknowledge byten (0xAB), fra den protokolleret-datapakke sendt fra MCUen, er placeret sidst i datapakken (5. byte). FPGAens afsender 0x00-byten i det moment den modtager acknowledge byten (fuld duplex kommunikation).

Acknowledge byten bruges også til at identificere fejl i den modtaget datapakke. F.eks. 0x40 indikerer fejl i den adresse-bestemmende-byte for den modtaget datapakke - for FPGAen. Alle modtaget byte for MCUen fra FPGAen, som ikke er 0x00, vil markeres som fejl. De fejl optælles - og vises i terminalen via UART0. Alt dette vises i test-afsnit 4.3.3.

1.3.4 Max Record length

Record length er også begrænset til 1000 ADC-målinger. Da bufferne er arrays makser ud på 1007. Bemærk at de 7 ekstra, i arrayet, er tiltænk datapakkens synkbytes, checksumbytes, typebyte og længdebytes. Og vi kan også lave record length på 10 ADC-målinger. Alt dette vise i test-afsnit 4.4.2.

1.3.5 Kontinuert Uafbrudte Målinger

Der benyttes to buffere for de indkommende ADC-målinger. De veksler mellem at fyldes og tømmes for data. En buffer kan nemlig ikke tømmes og fyldes samtidigt.

Hvis kode var lavet med en buffer; ville den data der ikke nås at sendes, bagerst i bufferen, blive overskrevet af nye ADC-målinger. Med to buffere undgås dette scenarie.

Brugen af to buffere for de indkommende ADC-målinger, og bestemmelsen af minimum record length (afsnit 1.3.2) betyder i opnår flotte kontinuerer, uafbrudte målinger for vores oscilloskop. Se test-afsnit 4.4.2 for dette.

1.3.6 Brug af interrupt og polling + interrupt diagram

ISR (interrupt service rutine) skal bruges for modtagelsen af ADC-værdier. ISR sørger for at det indkommende ADC-data bliver opprioriteret. Det sikre korrekte og kontinuere ADC-målinger.

ISR bruges, af samme grund, til ADCens timer 1 - læs afsnit 1.3.1 for mere information.

Modtagelsen af data fra Labview skal, ifølge kravsspecifikationerne, foregå via. intterupt.

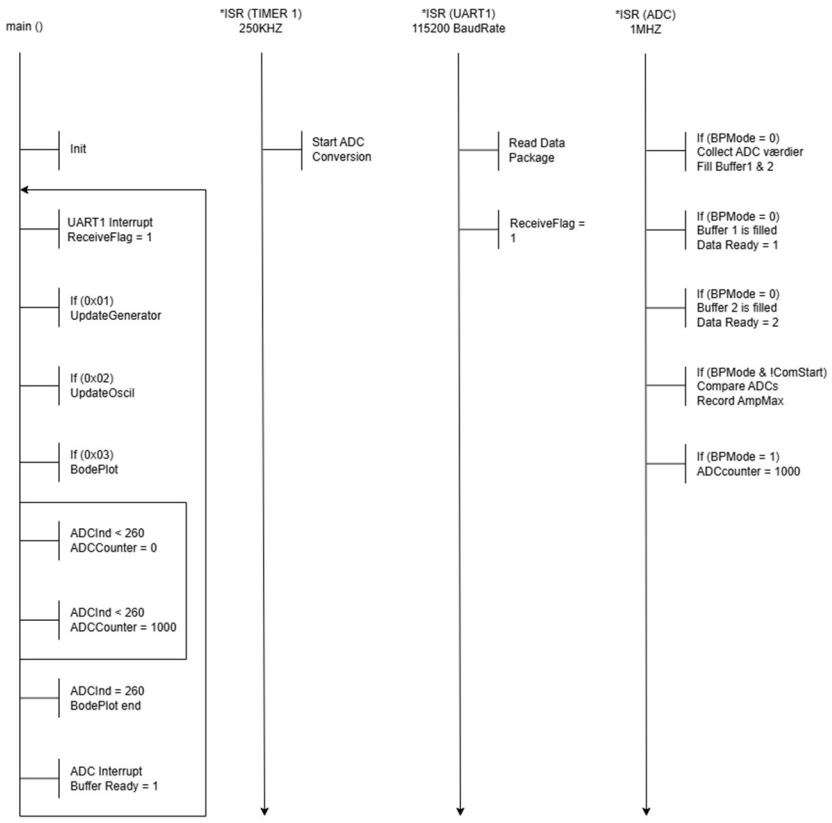
Vi bruger ISR for UART1 til modtagelsen af data fra Labview. ISR sikre at dataen fra Labview bliver indlæst korrekt. Men selve transmission, af den modtaget data, foregår med polling. Transmission af data fra de to buffere, for ADCen, foregår også med polling.

Nå ISR trigges; sættes den pågældende periferelle enhed i gang med dens protokol. Altså, CPUen i gang værende protokol bliver ikke 'pauset' for hele tiden det tager, f.eks; at ADCens ISR laver en ADC-konvertering $(13\mu s)$. Nej, CPUen bliver kun pauset i så lang tid det tager for ISR at sættes et 'gå-i-gang flag' i f.eks. ADCen. Og derfor er polling faktisk tilstrækkelig for transmission af data fra bufferne for ADCen.

En transmission af en byte versus modtagelse af en byte ved 10kHz:

$$transmission\ af\ byte\ \frac{10}{11520} = 86.8\mu s < 100\mu s\ modtagelse\ af\ en\ byte\ 10kHz.$$

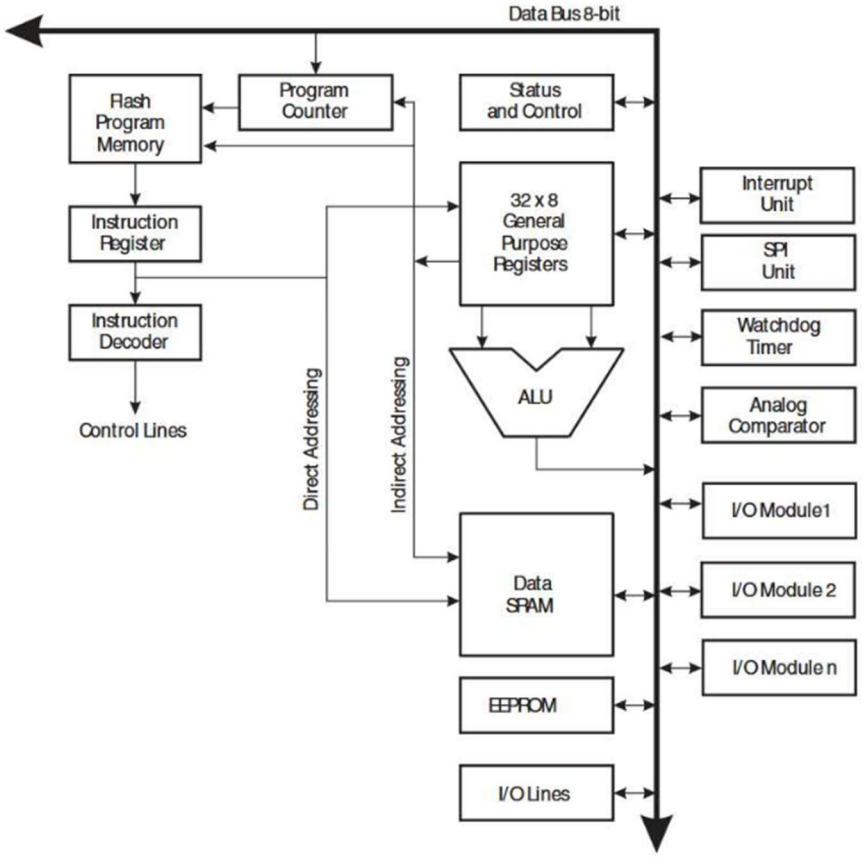
Selve ISR flagsætning, for compare match a og ADCen, må derfor kunne nå at finde sted i overheadtiden på: $100\mu s - 86.8\mu s = 13.2\mu s$. Interrupt diagrammet herunder viser de ISRer der bruges. Og hvor flagene fra ISRerne bruges i main, figur 16.



Figur 16 Interruptdiagram.

Alle variabler, der deles mellem vores main-program og en pågældende ISR, deklareres som volatile. Det gøres så compileren altid læser de værdier fra SRAM. Og ikke fra et af CPUens general purpose registrer. Det er væsentligt, fordi; værdierne ændres asynkront af en ISR for main-program. CPUens main-program kan derfor være blind for ændringen af variablen. Uden volatile kan main-programmet derfor risikere at arbejde med en forældet værdi og dermed ikke registrere de ændringer, som ISRen foretager. Se figur 17.

Figure 7-1. Block Diagram of the AVR Architecture



Figur 17 Blokdiagram af AVR-arkitektur fra bogen

1.3.7 SPI-kommunikation

MCU'en modtager data-pakker fra LabVIEW via UART1. Derefter sender MCU'en de ompakkede data-pakker til FPGA'en via SPI, hvilket gør det muligt for FPGA'en at udføre LabVIEW's kommandoer. MCU'en og FPGA'en er konfigureret til at køre med en baud rate på 500K (bits per sekund).

MCU'en fungerer som Master, mens FPGA'en fungerer som Slave. I denne proces kommunikerer MCU'en og FPGA'en ved hjælp af en på forhånd defineret protokol.

Denne protokol er en selvudviklet kommunikationsprotokol tilpasset for FPGAen, se Figur 18.

Frame-struktur (5 bytes i alt)

Byte	Navn	Fast værdi/interval
Byte 1	Head	0xFF (fast)
Byte 2	Type	0x01–0x06
Byte 3	Data	Afhænger af Type
Byte 4	Checksum	0x00 (fast)
Byte 5	Acknowledge Byte	0xAB (fast)

Amplitude Data-mapping (når Type = 0x02)

Digital (Dec)	Digital (Hex)	Tilsvarende spænding
0	0x00	0 V
255	0xFF	3,3 V

)ata-mapping (når Type = 0x03)

Digital (Dec)	Digital (Hex)	Tilsvarende frekvens
0	0x00	0 kHz
255	0xFF	23,7 kHz

Type-definition (Byte 2)

Type (Hex)	Funktion
0x01	Shape (Bølgeform)
0x02	Amplitude
0x03	Frequency (Frekvens)
0x04	Reset
0x05	Run (Start)
0x06	Stop

Shape Data-definition (når Type = 0x01)

Data (Hex)	Bølgeform
0x01	Constant (Konstant)
0x02	Firkant
0x03	Savtak
0x04	Sinus
0x05	Trekant
0x06	Inverteret Savtak

Figur 18 MCU Tabel 1: Protocol af SPI-kommunikation fra MCU til FPGA

Protokollen består af en fast frame-struktur på 5 bytes, hvor den første byte (Head) altid er 0xFF, og den fjerde byte (Checksum) altid er 0x00. Den anden byte angiver kommandotypen (Type), mens den tredje byte indeholder den tilhørende data (Data).

Den femte byte er fast sat til 0xAB og fungerer som en acknowledge-byte. For mere information for acknowledge-byte læs afsnit 1.3.3 - og test-afsnit 4.3.3.

1.4 Flow Chart for Main.c

Ved opstart af main-programmet peges først på .h-modulerne -hvorfra funktioner og variabler indledes for de perielle enheder. Dette kan også ses i moduldiagrammet (afsnit 1.2).

Når der sendes en datapakke fra Labview til MCUen trigges ISR for UART1. I ISRen indlæses bytene en ad adgange (bit for bit). Den 5. byte, som der bliver læst ind, har en særlig betydning.

Den 5. byte sammenlignes med tre mulige typer/cases. Typen af den 5. byte bruges til at angive hvad total længde er for den indkommende datapakke. Og hvilken Labview menu dataen kommer fra.

Type 0x01 = data fra generator-menuen, type 0x02 er data fra oscilloskops-menuen og type 0x03 = data fra bodeplot menuen. Bemærk at de indkommende databyte skal forstås som hexadecimal tal.

Et flag sættes når længden af datapakken matcher den forventet længde - pålagt af det 5. byte. På den måde vides det at hele datapakken er indlæst.

Hvis datapakkens 5. byte er af type 0x01 - bliver datapakkens 6. byte set nærmere på.

Den 6. byte opdeles desuden i fire typer/cases. De fire typer repræsenterer de fire knapper fra generator-menuen. Type 0x00 = enter, type 0x01 = select, type 0x02 = run/stop og type 0x03 reset.

Labview har selv ingen logik for dens knapper. Logikken for knapperne er derfor udarbejdet i main.c.

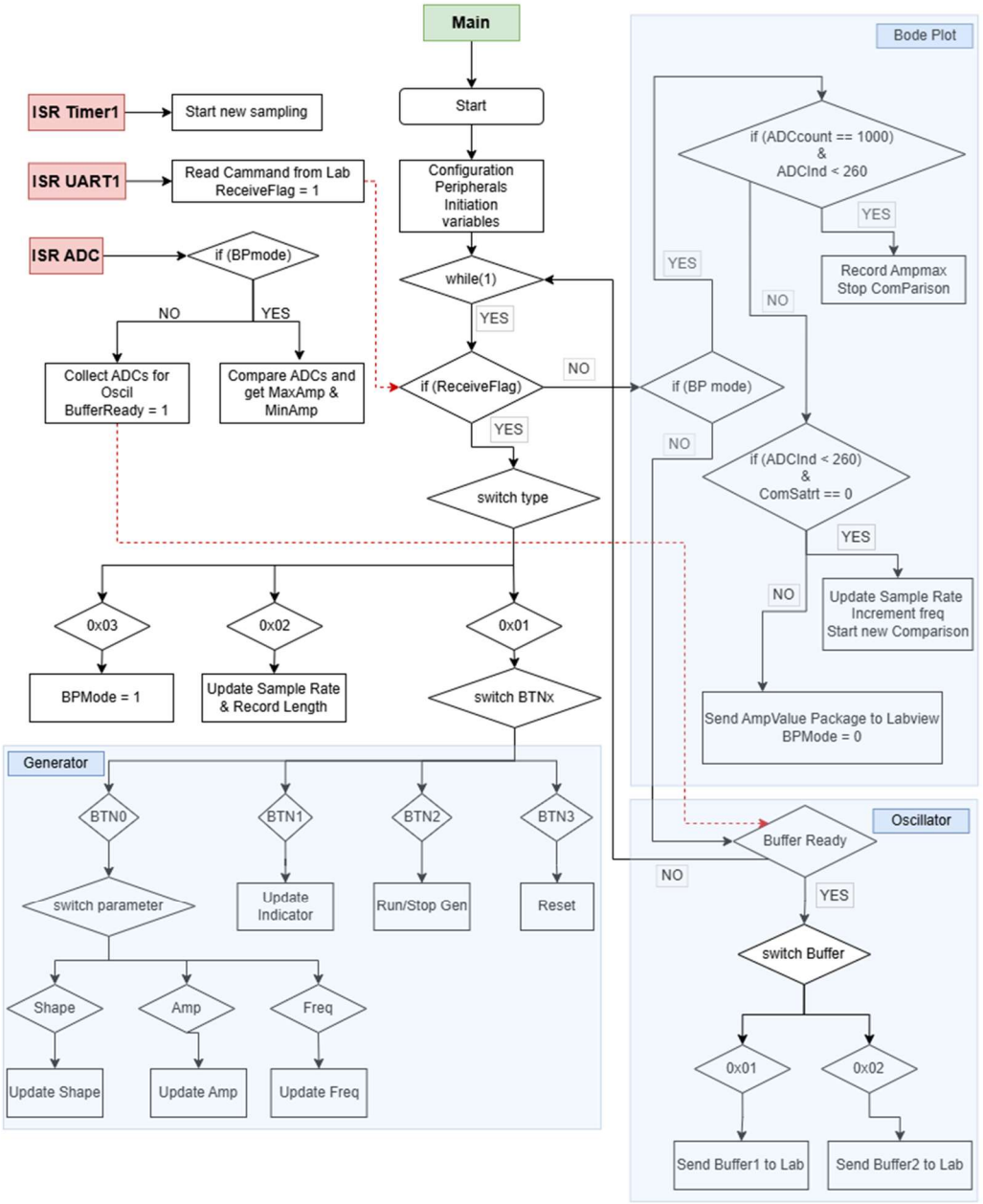
Nå en knap trykkes i Labview sendes en datapakke til MCUen - og derfra tilbage til Labview. Denne kommunikation er med UART1.

Run/Stop knappen har en toggle-funktion i dens case. Run/Stop bruges til at slukke eller tænde signalgeneratoren.

Reset nulstiller alle SW-værdier valgt i generator-menuen. Og de tilsvarende registre i signalgeneratoren nulstilles også.

Select bruges til 'browser' generator-menuen. Der er en bagvedlæggende tilstandsmaskine for select - læs afsnit 1.1. Og enter gemmer så den indtastet værdi for den pågældende SW-værdi i generator-menuen og FPGAen.

Flowchartet for main.c, se figur 19, viser tydeligt mains behandling af type 0x02 og 0x03 for den 5. byte. Bodeplot påbegyndes ved type 0x02, og værdierne i oscilloskops-menuen opdateres ved type 0x03.



Figur 19 Flowchart for main()

Læs afsnit 1.5 Bodeplot og 1.3 Oscilloskop design for information om main.c betydning for oscilloskopet.

1.5 Bodeplot

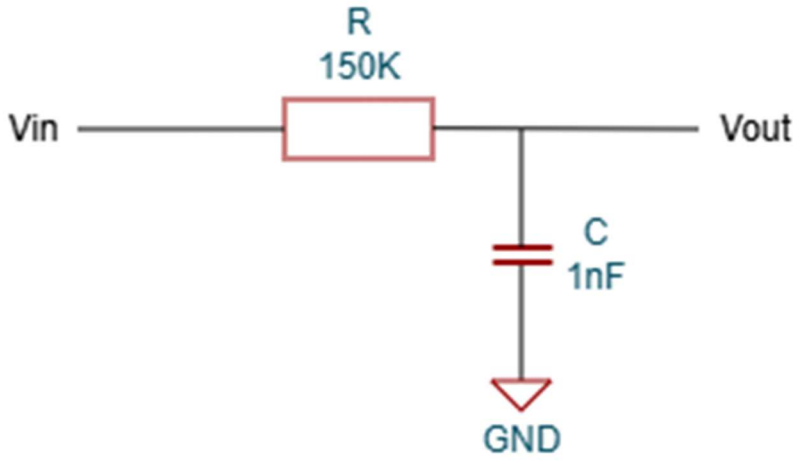
Under ”Bodeplot” fanen er det meningen at man skal kunne få lavet et plot af amplituden af et sinussignal som funktion af frekvensen.

Ideen er at det analoge signal fra signalgeneratoren (efter PWM-filteret) sendes igennem et filter som man ønsker at undersøge. Vi valgte et 1. ordens lavpasfilter med en grænsefrekvens på 1000 Hz som vores testfilter. Til dette formål valgte vi en modstand på 150 kΩ og en kondensator på 1 nF, hvilket resulterede i en faktisk grænsefrekvens tæt på 1061 Hz.

RC lavpasfilterets grænsefrekvens-formel:

$$f_c = \frac{1}{2 * \pi * R * C} = \frac{1}{2 * \pi * 150K\Omega * 1nF} \approx 1061Hz$$

Figuren herunder illustrerer kredsløbsforbindelsen for det filter, der skal testes, se figur 20.



Figur 20 Filter opsætning

Vin er forbundet til output signalet fra PWM-filteret. Vout er forbundet til MCU ADC0 kanal.

Microcontrolleren beder signalgeneratoren om at lave et sinussignal ved den laveste frekvens (FDigital = 1 ~ FFrekvens ≈ 24Hz) og med maksimal amplitude.

Efter sinussignalet har passeret lav-pas filteret, måles amplituden af dette signal med microcontrolleren (dette bruges som reference).

Frekvensen tælles nu én op i signalgeneratoren og amplituden måles igen af microcontrolleren. Dette repeteres for alle 255 frekvenser. De målte amplituden omregnes til en relativ værdi i forhold til referenceamplituden. Disse værdier sendes til LabviewOscilloscope gennem en datapakke. Bodeplottet vises i bodeplot-menuen.

Når der trykkes på start, i bodeplot-menuen, sendes en datapakke med type 0x03 til microcontrolleren.

Opstart af BPMode

Når MCU'en modtager en datapakke via UART1 med type-byte 0x03, sættes flaget ReceiveFlag af ISR. I hovedprogrammets while(1)-loop registreres dette flag, og programmet går ind i case 0x03 i switch-strukturen, se afsnit 1.4. Her initialiseres Bode Plot-målingen:

- Den globale variabel BPMode sættes til 1, hvilket aktiverer Bode Plot-tilstanden både i hovedloopet og i ADC-interruptet.
- ADCInd nulstilles til 5, så indekseringen i resultatbufferen AmpValues[] starter ved det første gyldige datapunkt (de første 5 bytes er reserveret til protokol-header).
- Signalgeneratoren på FPGA'en konfigureres via PutStrFPGA() til at udsende et sinussignal (0x04) med maksimal amplitude (0xFF).
- Adressebyten for frekvensparameteren (0x03) forberedes til brug i den efterfølgende frekvenssweep.

Frekvenssweep og prøvetagning

Selve frekvenssweepet kører kontinuerligt i hovedloopet, så længe BPMode er sat. For hvert frekvenspunkt (styret af ADCInd, fra 5 til 259, svarende til 255 testfrekvenser) udføres følgende:

1. Funktionen DigitalToFrequency() omsætter det aktuelle digitale frekvensord til den tilsvarende analoge frekvens, som bruges til at vælge en passende samplingsrate. OCR1A (Timer1s topvalue) justeres derved, så ADC'en altid sampler med ca. 10 prøver pr. periode af testsignalet – uafhængigt af, om frekvensen er lav eller høj. Sampleraten bruges ikke til korrekt gengivelses af et signal i bodeplot-plot. Nej, dens funktion er kun at måle amp-max og amp-min. En samplerate på 10Hz giver også mulighed for at lave 1000 ADC-målinger per frekvens målt.
2. Den nye frekvensværdi sendes til FPGA'en via SPI (PutStrFPGA()), så signalgeneratoren skifter til den næste testfrekvens.
3. AmpMax og AmpMin nulstilles, og flaget ComStart sættes til 1, hvilket starter selve amplitudemålingen for det aktuelle frekvenspunkt.

Amplitudemåling i ADC-interruptet

Mens ComStart er aktiv, indsamler ADC_vect-interruptet 1000 ADC-prøver af det indkomne signal. For hver prøve opdateres AmpMax og AmpMin, så de til sammen afspejler signalets spidsværdier (peak-to-peak) ved den givne frekvens. Når 1000 prøver er nået, registreres dette i hovedloopet:

- Amplituden beregnes som AmpMax - AmpMin og gemmes i AmpValues[ADCInd].
- ComStart nulstilles, og ADCInd øges, hvorefter sweepet fortsætter til næste frekvenspunkt.

Afslutning og dataoverførsel

Når ADCInd når 260 (alle 255 frekvenspunkter er målt), afsluttes Bode Plot-tilstanden:

- BPMode, ComStart og ADCcounter nulstilles, og ADCInd sættes tilbage til 5.
- Den fulde resultatbuffer AmpValues[] (262 bytes, inklusive protokol-header og -footer) sendes til LabVIEW via UART1 (PutStrlab()) og samtidig til PC-terminalen via UART0 (PutStrPC()) til verifikation.
- Samplingsraten (OCR1A) nulstilles til standardværdien (200), så systemet vender tilbage til normal oscilloskop-tilstand.

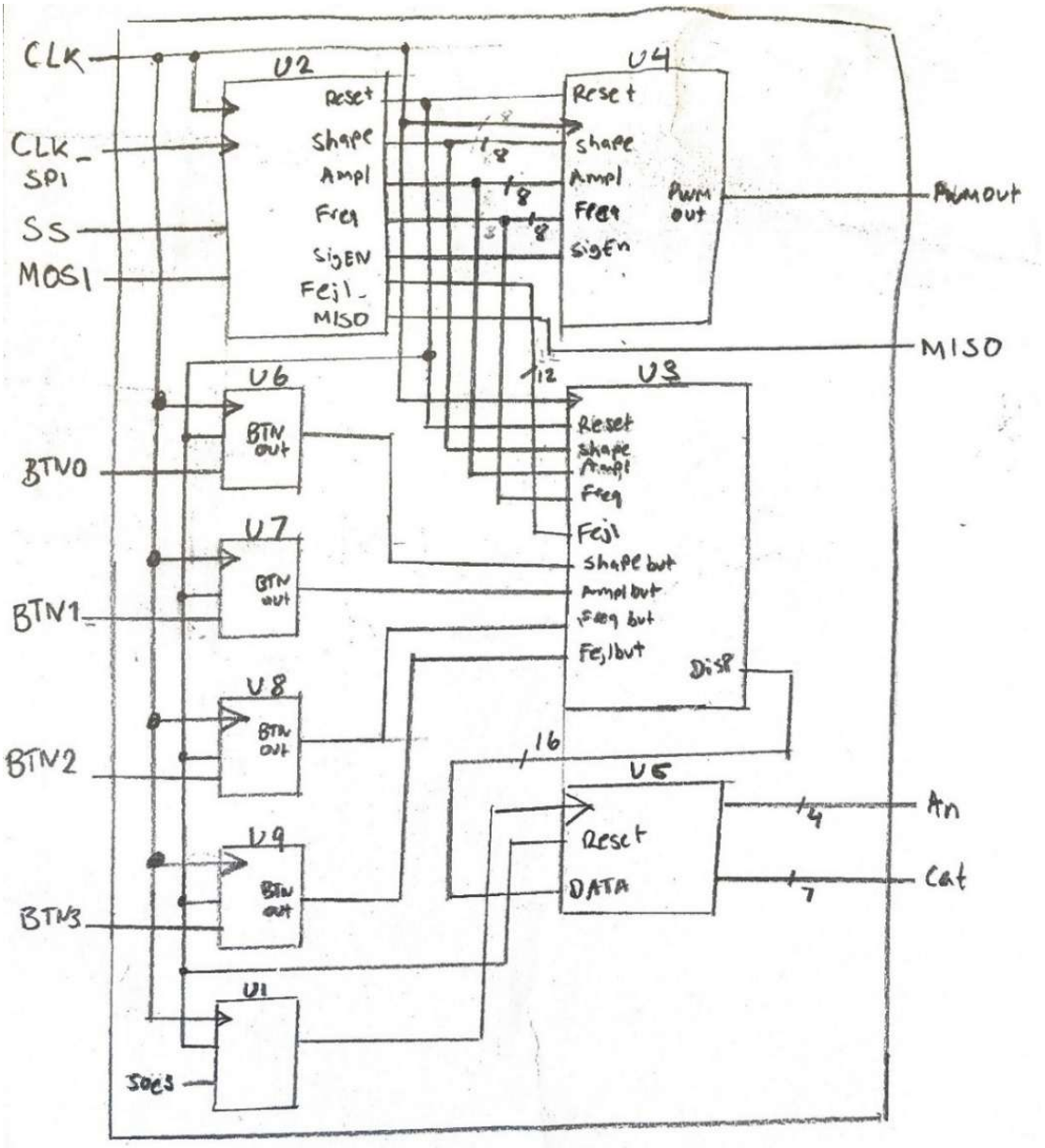
Det komplette resultat af signalernes amplitude-frekvens-karakteristik, bodeplot, ligger i AmpValues[]. AmpValues[] repræsenterer så signalerne amplituderespons, ved givne testfrekvenser, for lavpas filteret.

2. Signalgenerator

Signalgeneratorens funktion i projektet er at indsamle data gennem en protokol, som MCU'en sender, til at skabe et digitalt til analogt signal via PWM.

2.1 SigGenTop

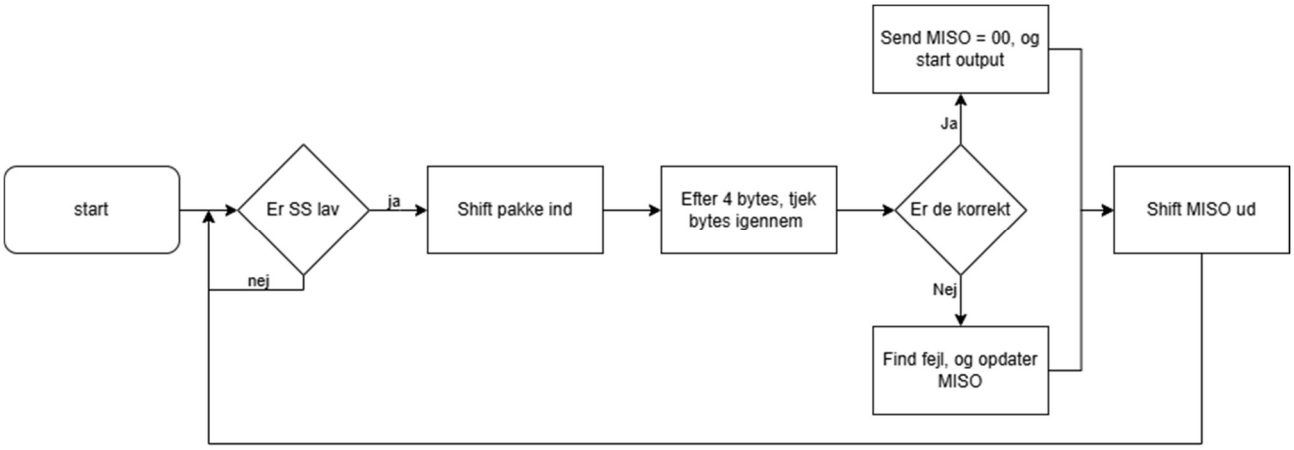
Til dannelse af signalgeneratoren på FPGA boardet er der dannet forskellige moduler, se figur 21. Heraf er det vigtigste modul U2, hvoraf en tilstandsmaskine og skifteregister sørger for, de rigtige informationer fra protokollen sendt fra mikrocontrolleren bliver læst korrekt. U4-modulet danner selve outputsignalet baseret fra U2. De andre moduler er med til at få syvsegmentsdisplayet til at fungere. I topmodulet har vi samlet alle småmoduler sammen og lavede portmap til hver, i forhold til hvilke signaler der skal forbindes til hvilke.



Figur 21. Funktionsdiagram over hele vhdl-koden.

2.2 SigGenKontrolUpdate

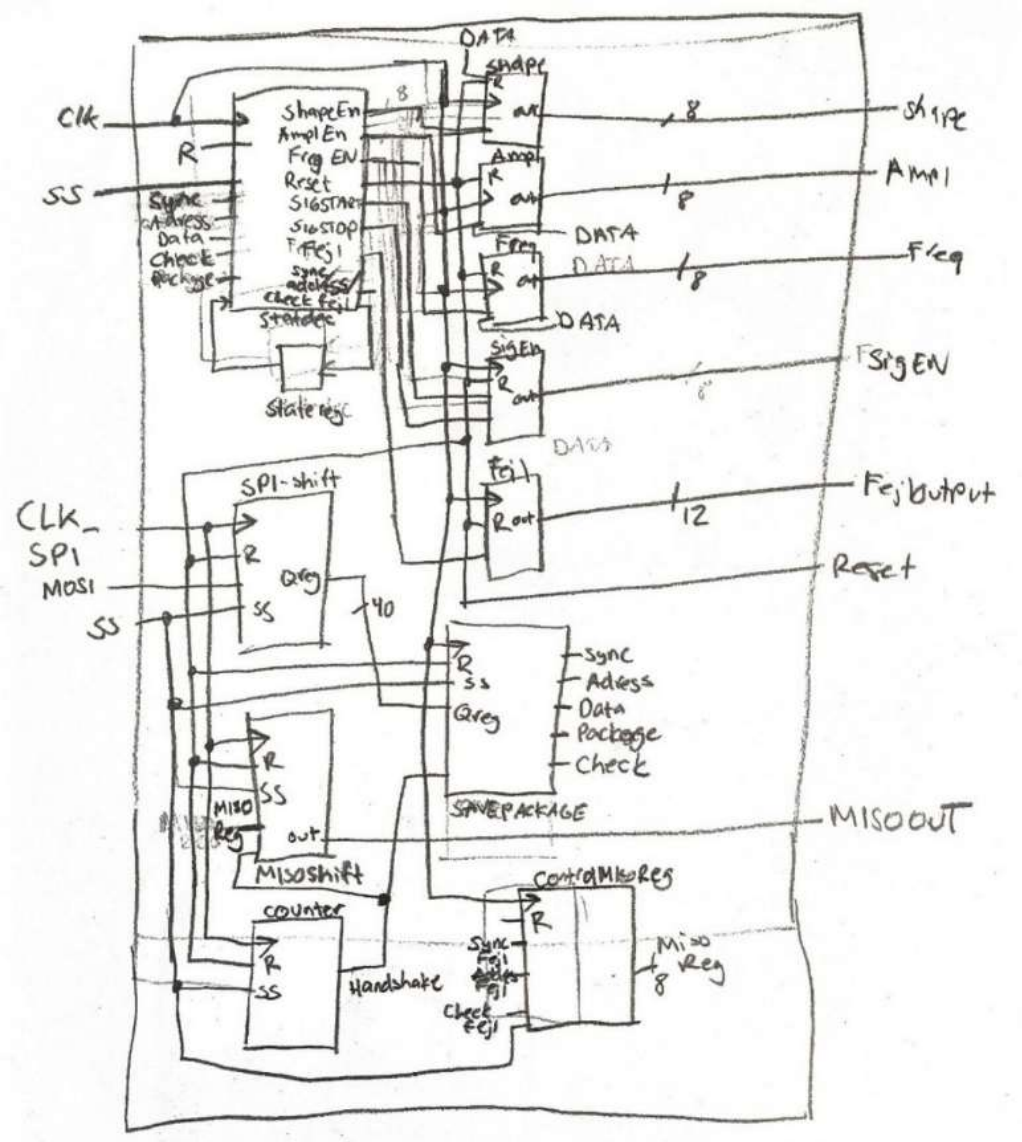
Flowet i modulet er som følgende; først tjekkes om SS er lav, altså om slaven er blevet vækket. Derefter bliver en pakke shifted ind via MOSI, og efter 4 bytes tjekkes bytesene igennem. Hvis de er korrekte, er MISO x00, og der kommer output ud af signalgeneratoren, ellers findes den bestemte fejlkode afhængig af fejlen, og MISO bliver shifted ud, se figur 22.



Figur 22. Flowchart over SigGenKontrolUpdate.

Funktionsdiagram

På figur 23 ses hele funktionsdiagrammet over SigGenKontrolUpdate. Dette modul består af en tilstandsmaskine med StateDec og StateReg. 5 registre gemmer værdierne til shape, amplitude, frekvens, SigEN (enable signal til PWM out) og fejl. 2 skifteregistre der shifter ind via MOSI og shifter ud via MISO, dertil et tilhørende controlregister der holder styr på fejlkoderne. En counter tæller til 32 clk, samt en process, der gemmer de tilhørende bytes fra MOSI ind i deres eget register.

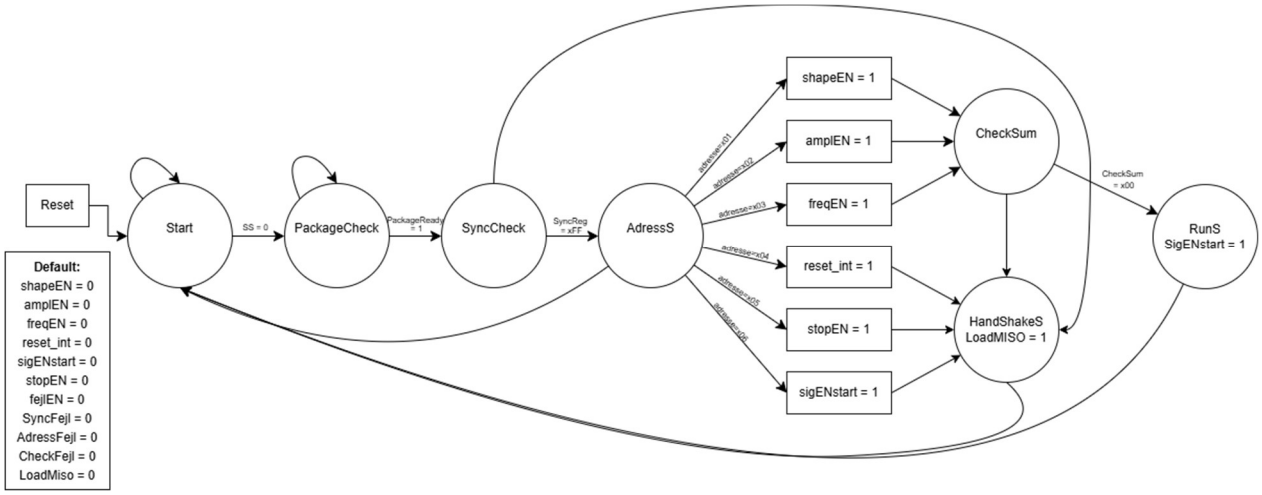


Figur 23. Funktionsdiagram over SigGenKontrolUpdate.

Tilstandsmaskine

Tilstandsmaskinen er opbygget på den måde, se figur 24, at SS skal blive lav, før man går fra start til næste state, altså masteren skal vække slaven. Derefter skal protokollen, uden den sidste dummybyte, kigges igennem, og hver byte bliver gemt på individuelt register. Når pakken på 4 bytes er shiftet ind i FPGAen, bliver packageCheck 1. Derefter skal syncbyte, adressebyte og checksum tjekkes igennem i hver sin state. Hvis sync-, adresse- og checkSumbyte er tilladte og korrekte, skiftes state og sigEn bliver sat høj, og dermed kan der

komme PWMout signaler. Hvis derimod bytesne ikke er korrekte skiftes til HandShakeS, hvoraf LoadMiso bliver 1, derved sendes en bestemt fejlkode tilbage til MCUen. Og der startes herefter forfra fra start.



Figur 24. Tilstandsdiagram over tilstandsmaskinen.

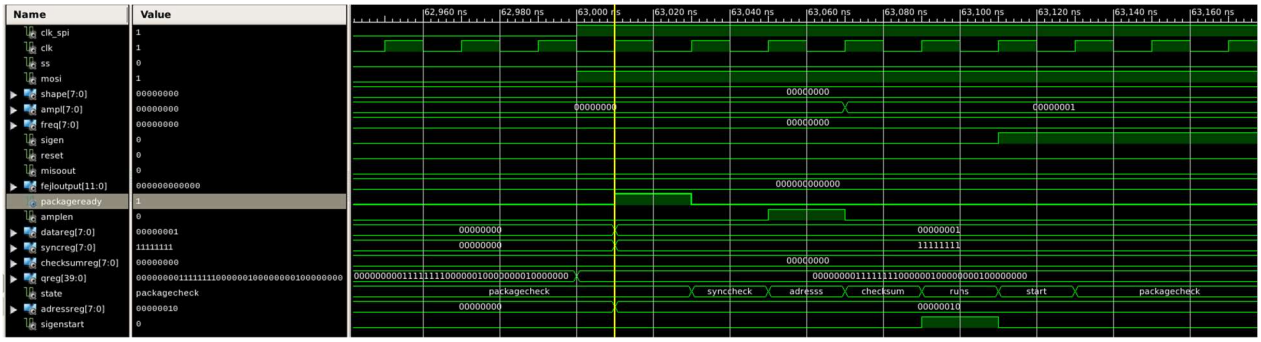
Desuden er det adressebyten fra protokollen, der fortæller hvilken handling, der skal ske. Hvis det enten shape, amplitude eller frekvens er valgt, er det den efterfølgende databyte, der bliver gemt i deres tilhørende register. På tabel 1 ses oversigten af funktionerne.

Tabel 1. Oversigt over adressefunktion.

Funktion	Adresse i binær	Adresse i hex
Shape	00000001	01
Amplitude	00000010	02
Frekvens	00000011	03
Reset	00000100	04
Stop	00000101	05
Run	00000110	06

Tilstandsmaskine - test

På testebænken er der simuleret en korrekt pakke, se figur 25, hvordan der skiftes state. Der er i testbænken zoomet meget ind, det hele er stadig inden for samme SCLK. Her ses fra pakken bliver klar, hvoraf sync-, adresse-, data- og checksumregister bliver opdateret lige efter fra Qreg. Til der skiftes fra packageCheck til syncCheck, til adressS, til CheckSum state, til RunS, hvor sigENstart bliver høj og til sidst tibage til Start, hvoraf sigEN er konstant høj indtil de nye protokoller fortæller noget andet.



Figur 25. Testbænk over tilstandsflowet med korrekt pakke.

Handshake

HandShake er etableret ved at skabe en 2-vejsforbindelse gennem SPI, heraf MISO. Dertil sender MCU'en en såkaldt dummybyte, som er den 5'te byte, da MISO kun fungere ved 2-vejs kommunikation. Endnu en counter på 32 skulle laves, da MISO skulle sende tilbage stadig indeni samme protokol, og FPGA'en skulle vide hvornår, skifteregisteret skulle begynde at sende data tilbage til MCU'en. Når 32 bits var gået, sendes et handshake signal til saveRecievedPackage processen og MISOshiftReg, som shifter byten ud på nedadgående flanke, da MCU'en læser på nedadgående flanke. Inden MISOShiftReg overhovedet begynder at shifte, bliver pakkens indhold gemt i registre, samt kørt igennem tilstandsmaskinen. Hvis der opstår fejl i bytesene, bliver enable signaler høj og der skiftes state til HandShakeS, der sætter LoadMISO høj. Baseret på hvor fejlen er opdateres MisoReg (byten der skal sendes ud) inde i ControlMISOReg, se tabel 2.

Tabel 2. Oversigt over MISO kode.

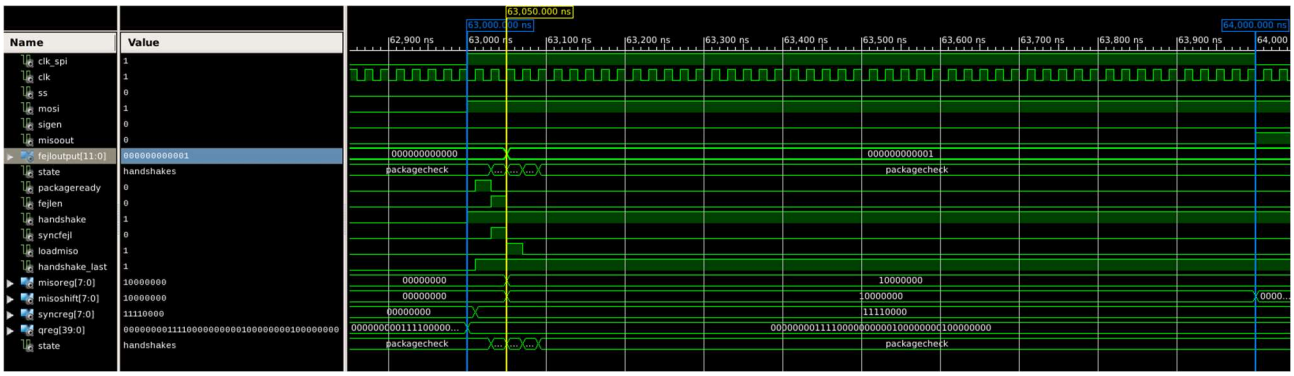
Handling	MISO i binær	MISO i HEX
Forkert syncbyte	10000000	80
Forkert adressebyte	01000000	40
Forket CheckSumbyte	00100000	20
Ingen fejl	00000000	00

Dette kan lade sig gøre, da skifteregistrene og counteren ikke kører på samme clk som alt andet. De kører på SCLK, som er den clk, der kommer fra MCU'en på 500 kbaud raten, sammenlignet med den interne clk fra FPGA boardet på 50 mHz, så selvom der kun er et halv SCLK fra counteren er færdig med at tælle, til MISO skal begynde at shifte ud.

$\frac{50000000}{500000 \cdot 2} = 50$, der går stadig 50 clks i mellem, som er mere end rigeligt tid.

Handshake - test

På figur 26 ses en testbænk, de 2 blå streger er en halv SCLK. Fra første blå streg sker en rising edge fra SCLK og de første 4 bytes er sendt, til anden blå streg, hvor der sker falling edge, hvor MISO begynder at sende ud. Der kan derfor ses, ved den gule linje, hvor MISOreg er opdateret ordentlig, masser af clk tilbage. Heraf i denne testbænk var der simuleret en forkert syncbyte.

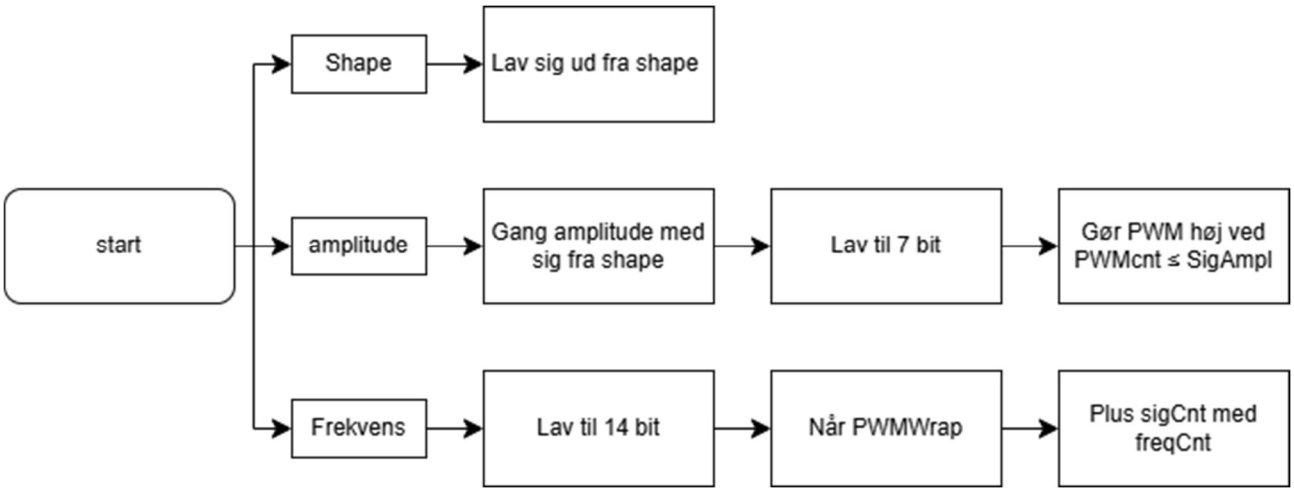


Figur 26. Zoomet ind mellem MOSI og MISO med forkert pakke.

Testet med MCU og LabView i 4.3.3.

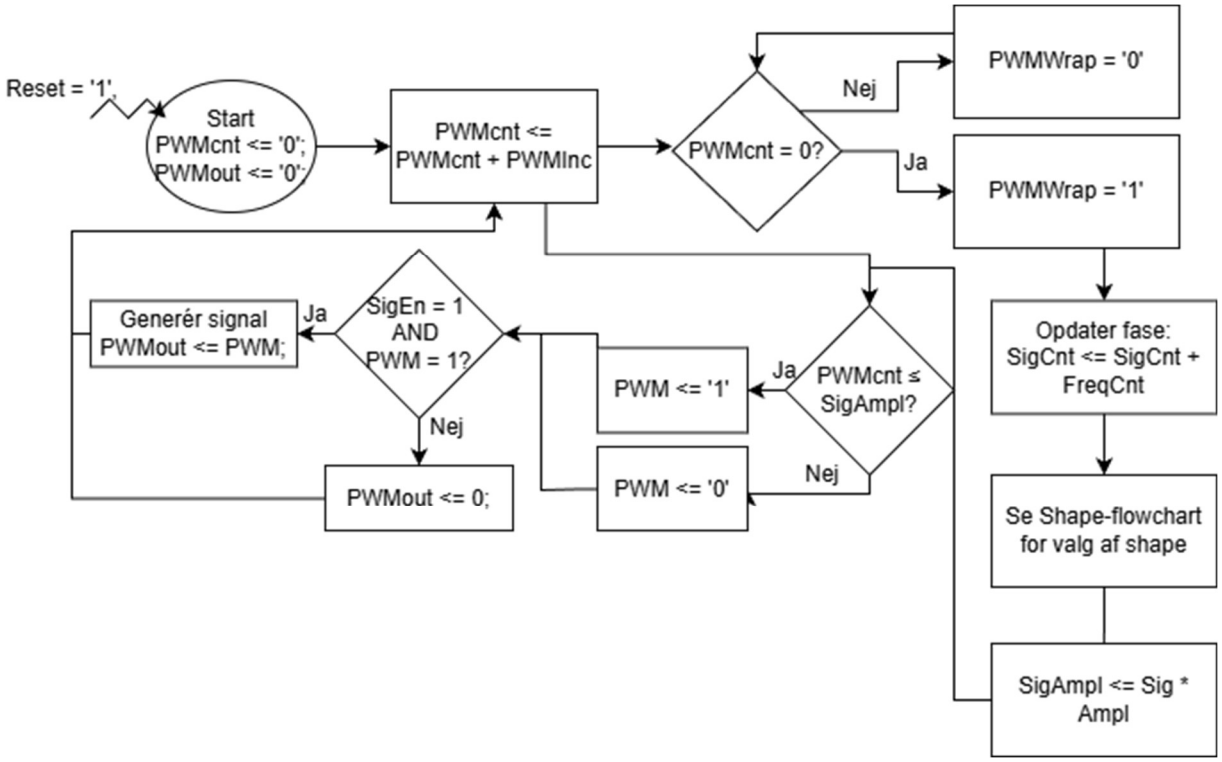
2.3 SigGenDataPath

Modul U4 fra figur 21 viser SigGenDataPath. Dette modul laver selve outputsignalet ud af FPGAen, baseret på hvad shape, amplitude og frekvens er. Disse data får vi fra modul U2. Og på figur 27 ses et flowchart over det basale flow i SigGenDatapath. Heraf sker alle processerne parallelt, ud fra hvad shape og sigCnt er, bliver der lavet et bestemt signal, Sig. Amplituden bliver ganget med Sig fra shape, og tager de 7 MSB, SigAmpl, for at kunne sammenligne med PWMcnt, som også er 7-bit. Hvis PWMcnt er ≤ SigAmpl bliver PWM-output signal højt, mens den er lav, når tælleren når højere end SigAmpl. Ved PWM-output signal højt forstås en udgangsspænding på 3.3V. Frekvensen bliver lavet til 12 bit semi-logaritmisk, plus 2 0'er foran til at lave 14 bit, freqCnt, og ved hver PWMwrap plusses sigCnt, som er de MSB af phaseAcc med freqCnt.



Figur 27. Flowchart over SigGenDatapath.

I nedenstående flowchart, se figur 28, vises hvordan PWM-outputtet genereres. PWMcnt tæller løbende op, når vi når 7 bit, 128, wrapper PWMcnt rundt, og PWMwrap sættes, som bruges til at opdatere signalets fase, SigCnt. Den aktuelle signalværdi afhænger af den valgte shape og ganges med Ampl til SigAmpl. Derefter sammenlignes PWMcnt med SigAmpl, hvilket bestemmer om PWM er høj eller lav. Til sidst bestemmer SigEn, om PWM-signalet sendes ud på PWMOut eller om outputtet holdes lavt.



Figur 28 Flowchart PWMOut

Shape

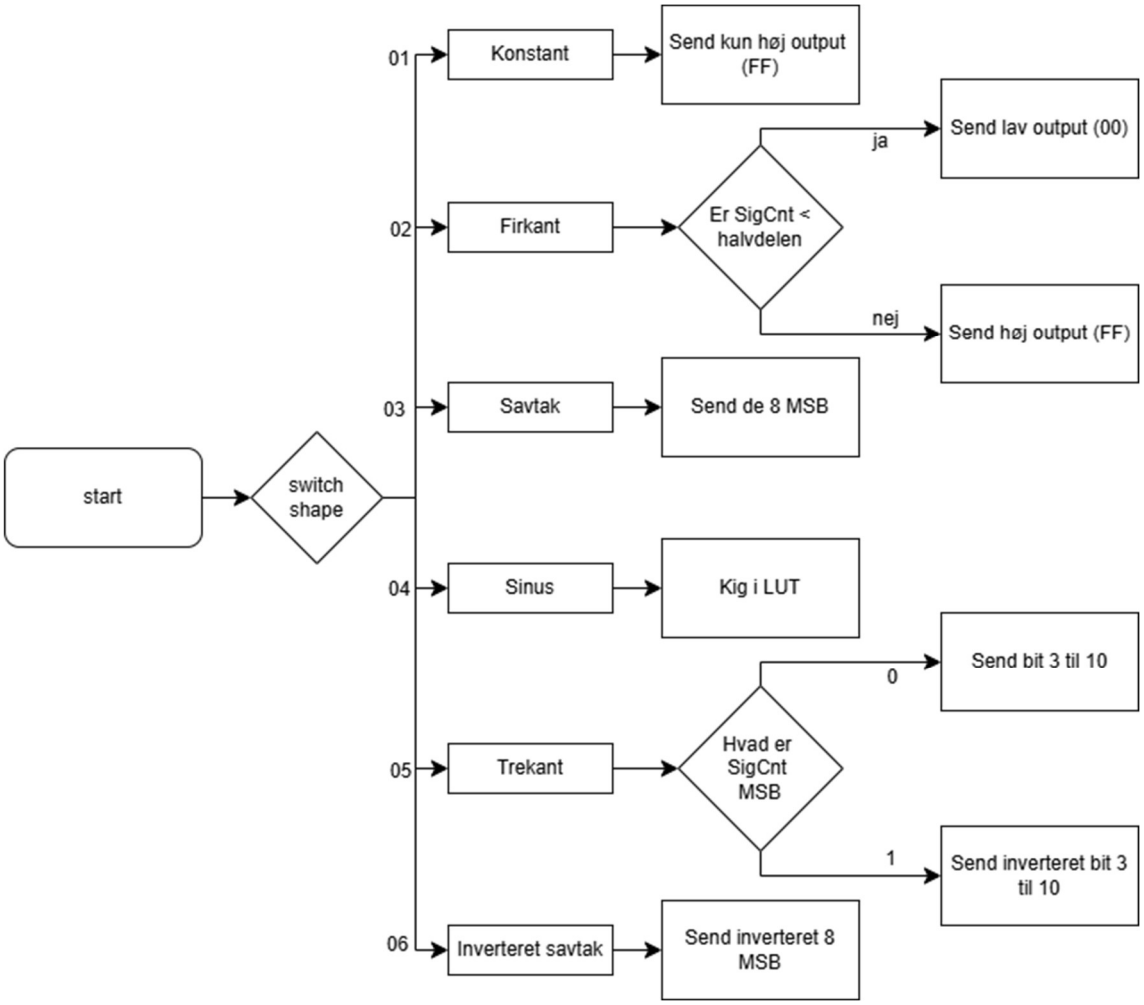
I forhold til shape, kan vi lave 6 forskellige kurveformer, se tabel 3. Det er en multiplexer, der vælger hvilket signal, der skal sendes videre i Sig, ud fra hvad Shape er.

Tabel 3. Oversigt over shape.

Shape	Adresse i binær	Adresse i hex
Konstant	00000001	01
Firkant	00000010	02
Savtak	00000011	03
Sinus	00000100	04
Trekant	00000101	05
Inverteret savtak	00000110	06
Sinus (default)	Andet	

På figur 29 ses et flowchart over, hvordan signalerne er dannet, ud fra hvilken shape der er valgt. Hvis shape er konstant, så er signalet høj hele tiden (kun 1).

- Hvis shape er firkant, tjekkes SigCnt om den er mindre end 800 (halvdelen), så sendes lav output, modsat hvis SigCnt er mere end 800, så sendes høje output.
- Hvis shape er savtak, sendes de 8 MSB, så de 4 LSB bliver ignoreret, så der kun tælles 1 op for hvert 16. bit. Derved går vi fra 12 bit fra SigCnt til 8 bit.
- Hvis shape er sinus, bruges SigCnt i look-up table for sinusværdi.
- Hvis shape er trekant, tjekkes hvis MSB er højt/lavt. Hvis den er lav, er vi under halvdelen og de næste 8 MSB bliver sendt. Mens hvis MSB er høj, sendes de næste 8 MSB inverteret.
- Hvis shape er inverteret sav, sendes de 8 MSB inverteret.



Figur 29. Flowchart shape.

SinusLUT

Der bruges et look-up table til sinuskurven. Sinus look-up table er en tabel, der viser alle sinusværdier, som er lavet på forhånd og gemt indeni ROM. Derved kan FPGA'en slå værdien op, uden det fylder i normale flip-flop registre eller skulle enkeltvis til at regne sinusværdi ud. Denne LUT er baseret på 12-bit SigCnt og 8-bit SigSinus. Der er 4096 forskellige adresser fra SigCnt, som repræsenterer x-aksens værdier. Der returnerer en 256 opløsning af sinuspunkter, som skal repræsentere y-aksens værdier. Heraf er SigSinus unsigned, derfor er midtpunktet omkring 127, mens top er på 255 og bund på 0.

Formlen er for SigSinus er således: $\left(1 + \sin\left(2 \cdot \pi \cdot \frac{x}{4096}\right)\right) \cdot 127$, hvoraf x er SigCnt.

Fx ved sinus 0, altså start, $\left(1 + \sin\left(2 \cdot \pi \cdot \frac{0}{4096}\right)\right) \cdot 127 = 127$, kommer 127 ud, som også er midtpunktet.

Amplitude

Amplituden fra FPGA'en er lavet mellem 0-3,3 V, hvoraf amplituden fra 8-bit er lineær, se tabel 4.

Tabel 4. Oversigt over amplitude.

Amplitude (Peak to peak)	Amplitude i hex	Amplitude i volt
0	00	0
1	01	0,0129
127	7F	1,7
255	FF	3,3

Da vi fjerner 9-bit ved SigAmpl <= MulC(15:9), kunne man også sige at vi skalerer ned med 2^9 altså 512. Derved ses SigAmpl = $\frac{MulC}{512}$. Vi ser også at vi ved SigAmpl = 0 ikke har en PWM duty cycle på 0, men 0.78%. Det kommer fra koden:
PWMcomp: PWM <= '1' when PWMcnt(6 downto 0) <= SigAmpl else '0';

Vi ser at PWM <= '1' én gang ved SigAmpl = 0 pga. "≤"-tegnet.

Firkant - udregning

I vores kode anvendes amplituden til at forøge PWM-frekvensen. Dette sker i vores AmplDec process, som ender med SigAmpl (15 downto 9), altså et 7-bitsignal, med de 7 øverste MSB fra produktet mellem Sig og Ampl. PWMout er højt når PWMcnt ≤ SigAmpl, derved ses at højere valgt amplitude resulterer i længere dutycycle. Vi regner et hurtigt eksempel med firkantsignalet, se tabel 5.

Tabel 5. Udregning af firkantsignal.

SigCnt	SigSquare	Ampl	Sig * Ampl = MulC	MulC(15:0)
<0x800	"0000 0000"	"1111 1111"	"0000 0000" * "1111 1111"	0
≥0x800	"1111 1111"	"1111 1111"	"1111 1111" * "1111 1111"	"1111 1110 0000 0001"
SigAmpl = MulC(15:9)		SigAmpl Dec	PWM-duty cycle regning	PWM-duty
0		0	0+1/128 * 100	0.78%
"111 1111"		2^7-1= 127	127+1/128 * 100	100%

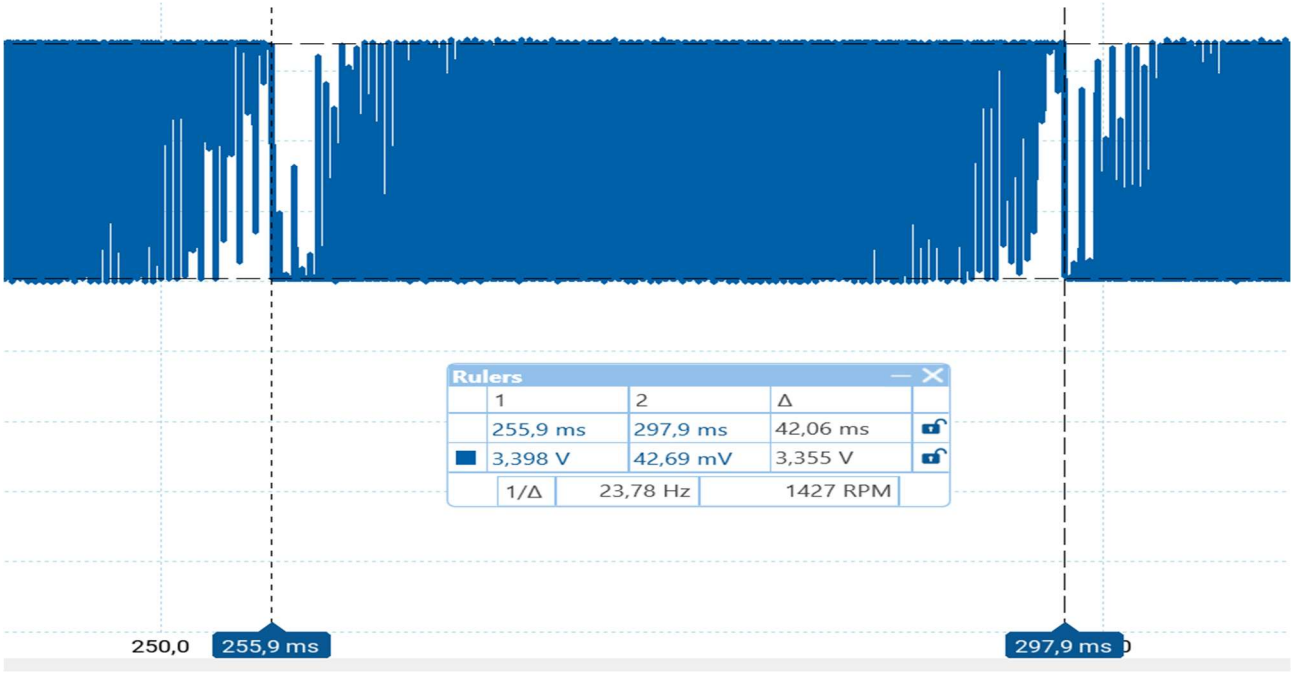
Savtak - udregning + test

Endnu et eksempel med savtak, se tabel 6.

Tabel 6 SigAmpl eksempel savtak

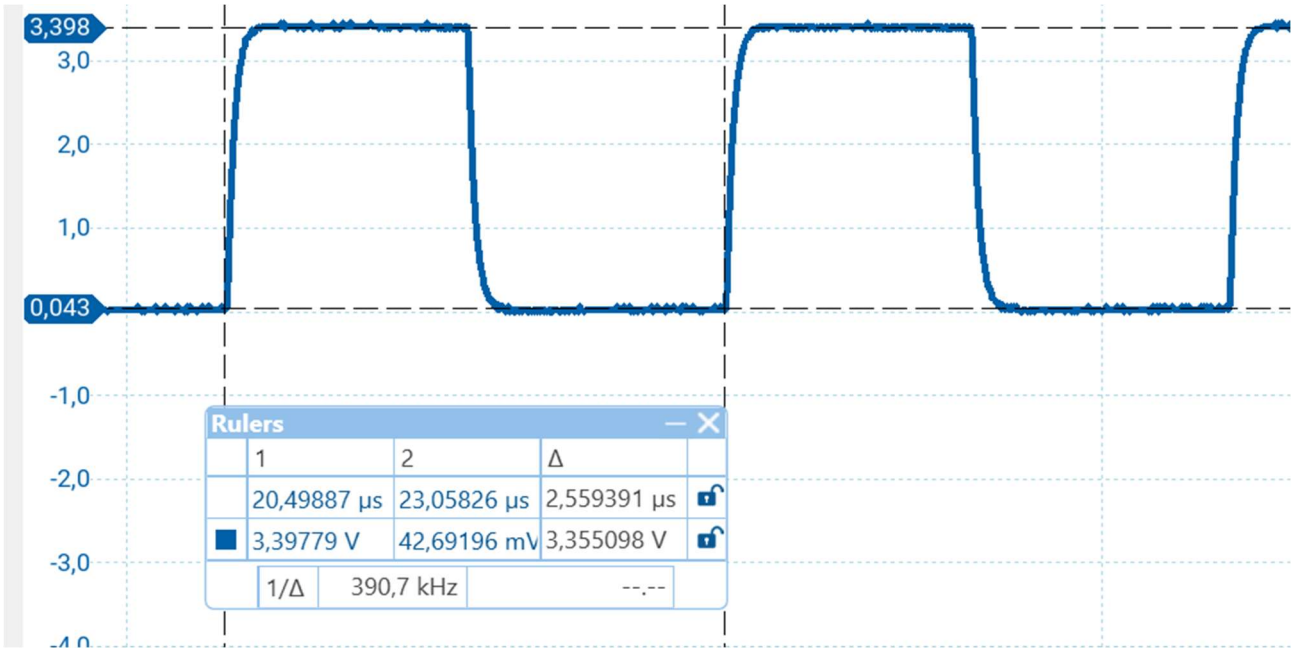
PWMwrap nr.	SigCnt før	FreqCnt	SigCnt efter	SigSaw = SigCnt(11:4)	SigSaw · Ampl	SigAmpl	PWM Duty Cycle
1	0x000	0x002	0x002	0x00	0x00 · 0xFF	0	0.78%
24	0x02E	0x002	0x030	0x03	0x03 · 0xFF	1	1.56%
32	0x03E	0x002	0x040	0x04	0x04 · 0xFF	1	1.56%
64	0x07E	0x002	0x080	0x08	0x08 · 0xFF	3	3.13%
128	0x0FE	0x002	0x100	0x10	0x10 · 0xFF	7	6.25%
256	0x1FE	0x002	0x200	0x20	0x20 · 0xFF	15	12.50%
512	0x3FE	0x002	0x400	0x40	0x40 · 0xFF	31	25.00%
1024	0x7FE	0x002	0x800	0x80	0x80 · 0xFF	63	50.00%
2047	0xFFC	0x002	0xFFE	0xFF	0xFF · 0xFF	127	100.00%
2048	0xFFE	0x002	0x000	0x00	0x00 · 0xFF	0	0.78%

Ved opsætning til picoskop uden filter ser vi måling på en savtakssignalperiode, på ~23,78Hz, mod den teoretiske signalfrekvens på 24Hz, se figur 30.



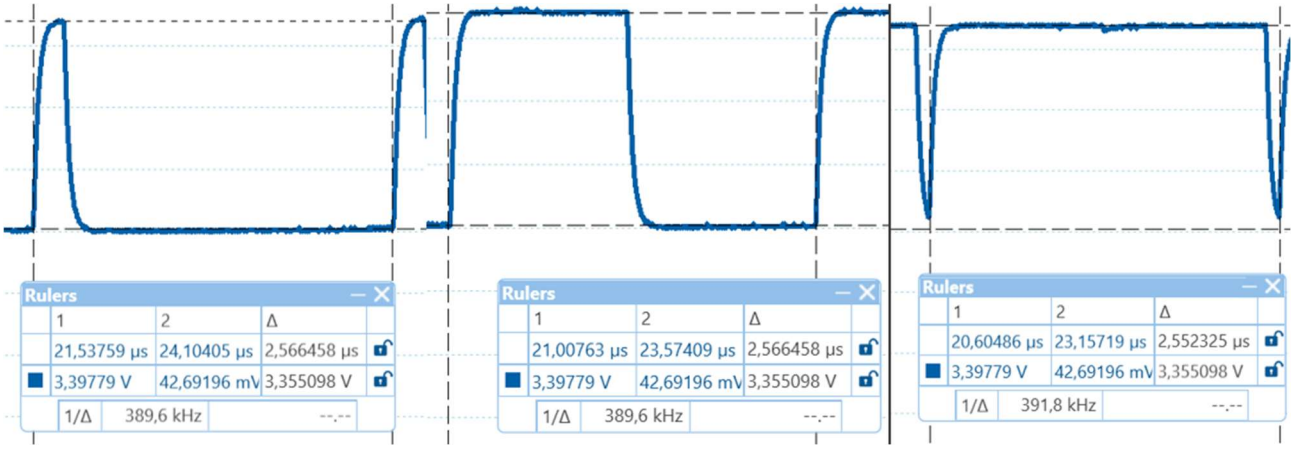
Figur 30 Picoskoptest af savtakssignal uden filter.

Vi zoomer ind til 2us/div og ser hver enkelt PWM duty cycle, her målt til ~390,7KHz mod den teoretiske PWM-frekvens på 390625Hz, se figur 31.



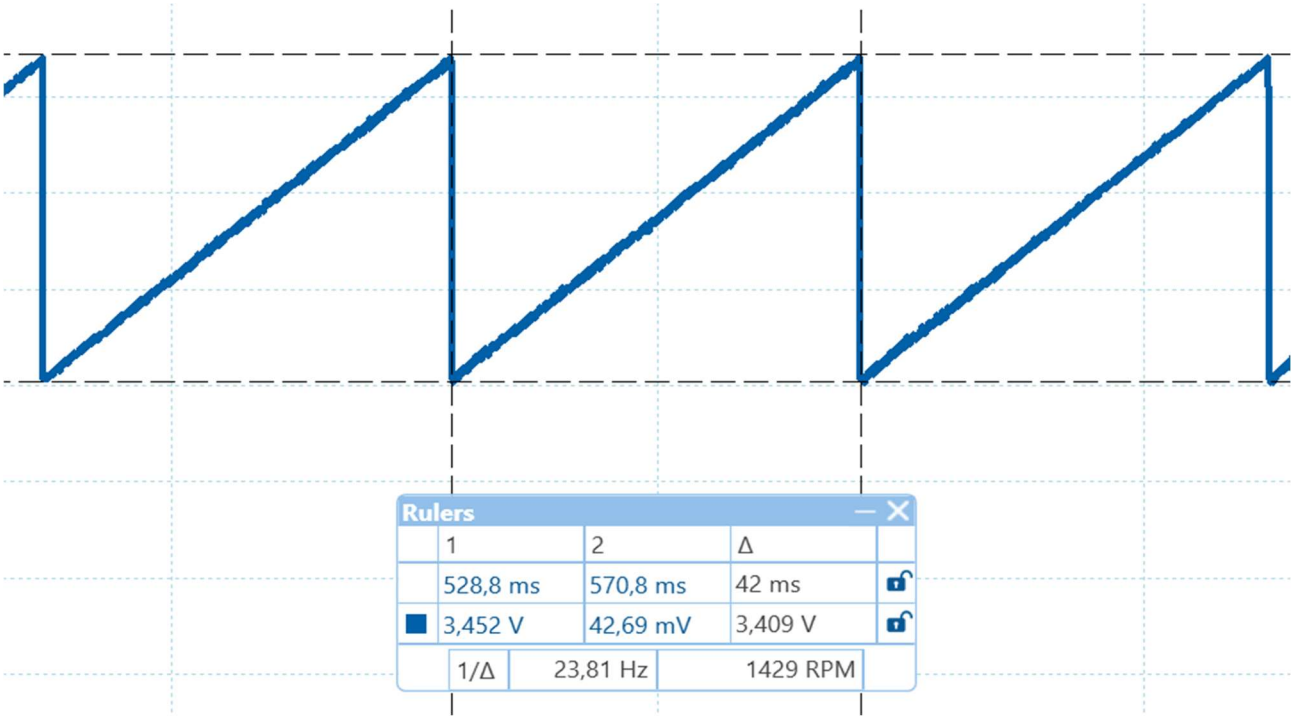
Figur 31. Zoomet ind på picoskop test fra figur 10.

Her ser vi hvordan PWM-duty cycle forøges gradvist ved savtak signal, se figur 32.



Figur 32. Gradvis forøgelse af duty-cycle.

Her ses samme savtak signal, med filteret, se figur 33.



Figur 33. Savtakssignal med filter.

Frekvens

Frekvensen er på 8 bit fra MCU'en, som er lavet semi-logaritmisk. Her er springene ved lavere frekvenser mindre, så vi får bedre opløsning. Mens ved større mest betydende bit, bliver springene mellem hver frekvens større. Modsat hvis man gjorde frekvensen lineær, så man fik samme størrelse spring mellem hver bit, betyder ændringer på lave frekvenser meget mere end ændringer ved høje frekvenser. På den måde fås mere præcision ved lavere frekvenser, og vi få lavet højere frekvenser. På tabel 8 ses frekvensen omregnet, og der kan se større spring fx mellem bit 15 til 16. Den måde det er gjort på er at tilføje 0'er ind imellem og foran bitene, da SigCnt er på 12 bit, og skal plusses sammen med den nye frekvens, FreqCnt, på 8 bit :

Freq: $f_7\ f_6\ f_5\ f_4\ f_3\ f_2\ f_1\ f_0$ → FreqDec: $0\ 0\ f_7\ f_6\ f_5\ f_4\ 0\ f_3\ f_2\ 0\ f_1\ f_0$

Formlen er : $\sim 24Hz \cdot FreqDec$

Tabel 7. Oversigt af frekvenser.

Frekvens i 8-bittal	Frekvens i hex	FreqDec	Frekvens i Hz
00000000 (0)	00	000000000000	0
00000001 (1)	01	000000000001	24
00001111 (15)	0F	000000011011	648
00010000 (16)	10	000001000000	1536
01111111 (127)	7F	000111011011	11400
10000000 (128)	80	001000000000	12288
11111111 (255)	FF	001111011011	23688

PWM-out

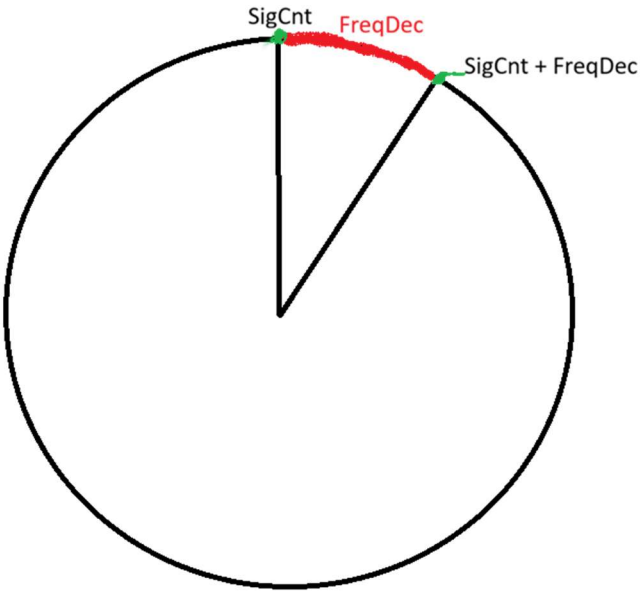
I den udleverede kode havde vi en PWM-frekvens på $\frac{12.5MHz}{128} = 97.7KHz$. Med den frekvens er det nært umuligt at skabe et PWM-filter som tilfredsstillende filtrerer PWM-frekvensen fra uden at betydeligt dæmpe de ønskede signaloutputs. Vi valgt derfor, at anvende den fuldstændige clock på 50MHz, og fik derved $4 \cdot \frac{12.5MHz}{128} = 4 \cdot 97.7KHz = 390.6KHz$ som vores nye PWM-frekvens, hvilket giver os det ønskede råderum. Da vores signalfrekvens har direkte sammenhæng med vores PWM-frekvens, og vi ikke ønsker at signalfrekvensen skal firdobles, er vi nødt til at skalere den ned. Det opnås i VHDL koden ved at tilføje et nyt signal, PhaseAcc,

som øger faseakumulatoren fra 12 bit til 14 bit. Derved bliver faseakkumulatoren 4 gange større, samtidig med at PWMcnt beholder sin 7-bittæller. SigCnt beholder sin 12-bit ved at være de 12 MSB af PhaseAcc.

SigReg

SigReg er vores faseakkumulator, se figur 34, den bruges til at skabe en bølgefrekvens. Den dannes når vores PWMcnt har talt til 512 og wrapper rundt, et flag PWMWrap = 1 sættes og SigCnt <= SigCnt + FreqCnt.

FreqCnt er et udtryk for, hvor langt et skridt vi træder i fasen, for hvert PWMWrap. Jo længere skridt, jo hurtigere har man gået gennem cirklen, og jo hurtigere er frekvensen. SigCnt viser hvorhenne i fasen vi befinder os aktuelt. Derved ved højere frekvens, er der mindre målepunkter på en PWMwrap, det ses i test 4.2.2



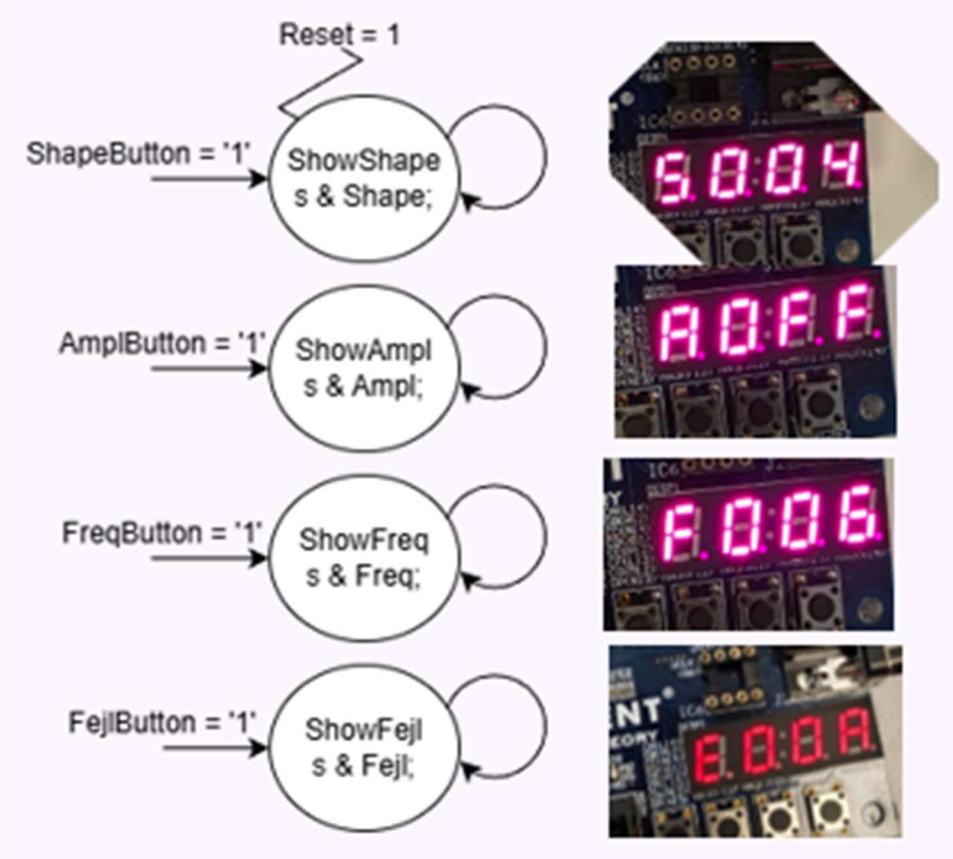
Figur 34. Tegning over faseakkumulator.

2.4 SigGenTop.ucf

Denne UCF-fil fortæller til FPGA'en hvilke fysiske pin på boardet der er forbundet til hvilke signaler i koden. På den måde kan vi få de indbygget 7-segment displayet, knapperne og master clk til at virke. Desuden kan vi forbinde SPI-forbindelse og PWM-out med I/O pins.

2.5 DispMux & SevenSeg5

Nedenstående tilstandsdiagram viser hvordan vi med knaptryk viser tilstande på 7seg, se figur 35. Heraf kigges der på enable signaler, som bliver høje, når en knap er trykket, og man derved skifter state.



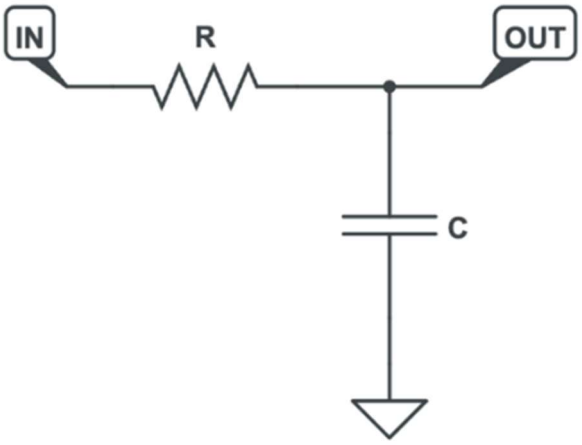
Figur 35 Tilstandsdiagram Dispmux

3. Filter

Generelt om filtre

Et filter er noget man bruger til at filtrere et udvalgt spænd af frekvenser med. Det fungerer ved at man dæmper amplituden for det filtrerede, se figur 36.

Det er især reaktansen som vi er interesserede i. Reaktansen er udtryk for modstanden gennem kondensatoren ifht frekvens og udregnes med følgende formel $X_C = \frac{1}{2 \cdot \pi \cdot f \cdot C}$. X er reaktansen, jo højere reaktans, jo højere modstand mod GND og jo mere vil Vout = Vin. Ved højere frekvens eller højere C (Kapacitans), bliver reaktansen lavere.



Figur 36. Filter diagram.

Derfor $f \to \infty \Rightarrow X_C \to 0\Omega$ svarende til en kortslutning, hvormed Vout er tæt på 0, derved er amplituden for den givne frekvens blevet dæmpet.

Formler:

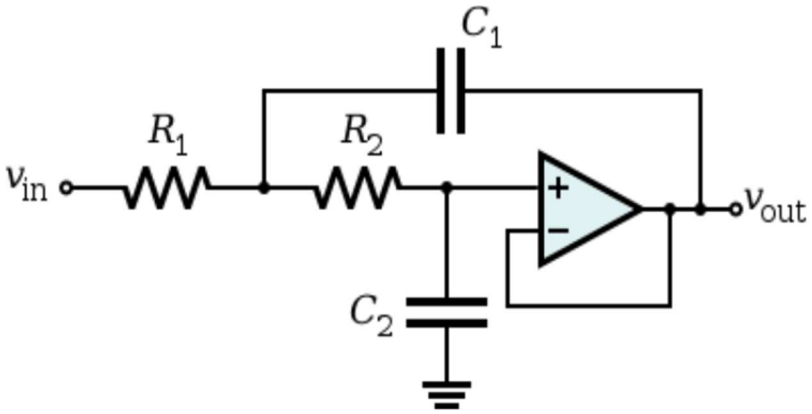
$$\text{Knækfrekvens: } f_c = \frac{1}{2 \cdot \pi \sqrt{R_1 C_1 R_2 C_2}} \quad Kvalitetsfaktor: Q = \frac{\sqrt{R_1 C_1 R_2 C_2}}{C_2 (R_1 + R_2)}$$

Vi vil i Q have isoleret vores kondensatorværdier. Vi sætter $R_1 = R_2 = R$

$$Q = \frac{\sqrt{R^2 C_1 C_2}}{C_2 (2R)} \to Q = \frac{R\sqrt{C_1 C_2}}{2 C_2 R} \to Q = \frac{1}{2} \frac{\sqrt{C_1}\sqrt{C_2}}{\sqrt{C_2}\sqrt{C_2}} \to Q = \frac{1}{2} \frac{\sqrt{C_1}}{\sqrt{C_2}}$$

Ønsket med filteret er at dæmpe PWM-frekvens væsentligt for at reducere ripple i signalet, med minimum reduktion af de signalfrekvenser vi ønsker at vise, altså fra 24 Hz til ~24KHz.

Da dæmpningen samtidigt skal ske ret kraftigt efter vores cut-off frekvens, vælger vi et aktivt 4. ordensfilter, opnået ved at sætte to Sallen-Key filtre i serie, se figur 37.



Figur 37. Sallen-Key filter.

På figur 38 ses, den grønne stiplede linie er den valgte cutoff-frekvens på ~100KHz, den blå er

Vi sætter C_1 til 470pF. Vi solver i maple og ser at vi skal have en $C_2 \approx 235pF$

Det tætteste vi har er 220pF

$$Q = \frac{1}{2}\frac{\sqrt{470}}{\sqrt{220}} = 0.73$$

altså har vi en lidt stærkere dæmpning end ønsket i ideelt filter, men det er godt nok til vores brug.

Nu da vi har vores kondensatorværdier skal vi finde vores modstande:

Vi sætter $f_c = 100000$, da vi ønsker knækfrekvens over vores maks frekvens på 23.7kHz, men tidligt nok til at kunne dæmpe vores PWMFrekvens så meget som muligt.

$$100000 = \frac{1}{2 \cdot \pi\sqrt{R^2 \cdot 470pF \cdot 220pF}}$$

Vi solver i maple for R og får: $R = 4949 \wedge R = -4949$

Da vi ikke kan have negativ værdi som modstand, vælges den modstand tættest på værdien vi har i komponentshoppen; $R = 4k99\Omega$. Vi indsætter værdierne i formlen og får:

$$f_c = \frac{1}{2 \cdot \pi\sqrt{4990^2 \cdot (470pF \cdot 220pF) \cdot 10^{-1}}} = 99188$$

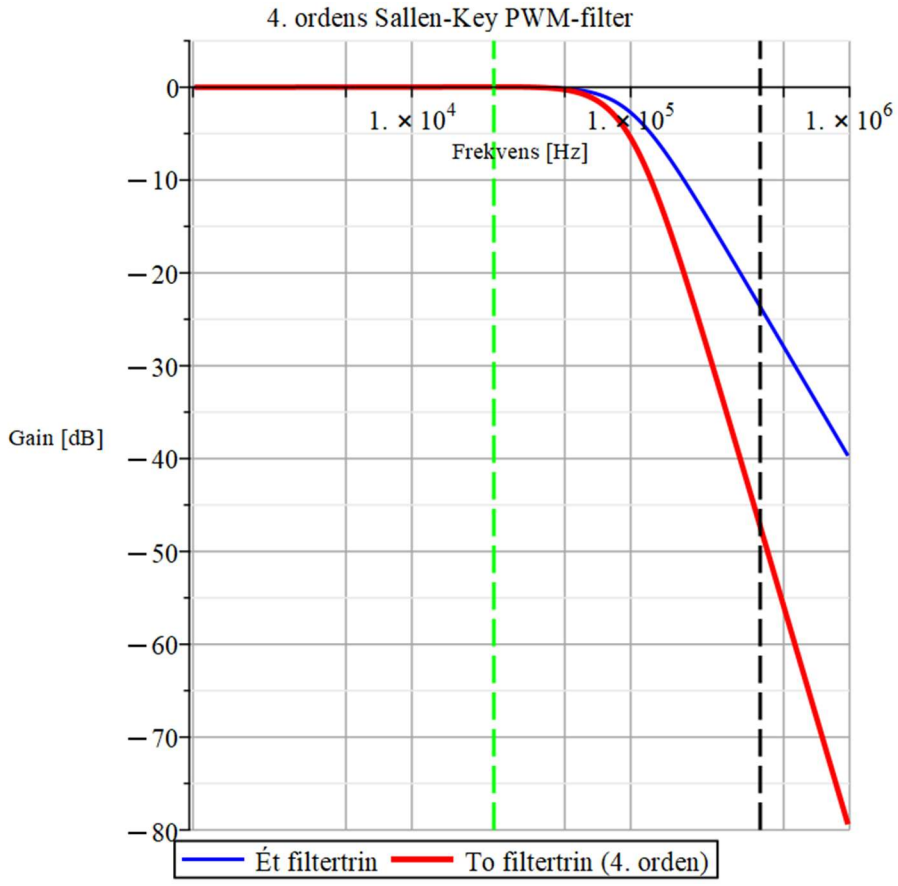
Vi har derfor kun en 0.8% afvigelse fra den ønskede knækfrekvens på 100kHz.

Dæmpning ved 2. ordens filter:

$$20 \cdot log_{10}\left(\frac{1}{\sqrt{\left(1-\left(\frac{390625}{99188}\right)^2\right)^2+\left(\frac{390625}{0.73 \cdot 99188}\right)^2}}\right) = \underline{-23.3dB}$$

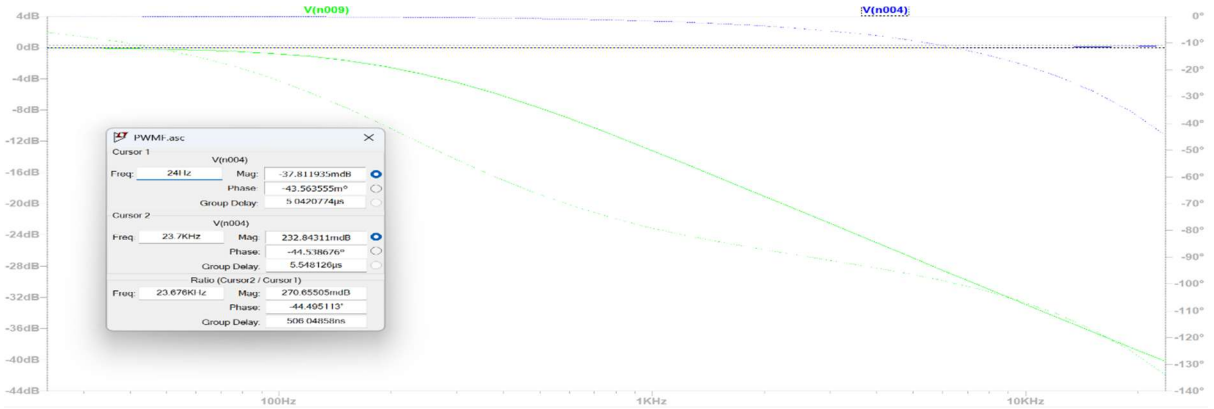
Dæmpning ved 4. ordensfilter.

$$20 \cdot log_{10}\left(\left(\frac{1}{\sqrt{\left(1-\left(\frac{390625}{99188}\right)^2\right)^2+\left(\frac{390625}{0.73 \cdot 99188}\right)^2}}\right)^2\right) = \underline{-46.6dB}$$

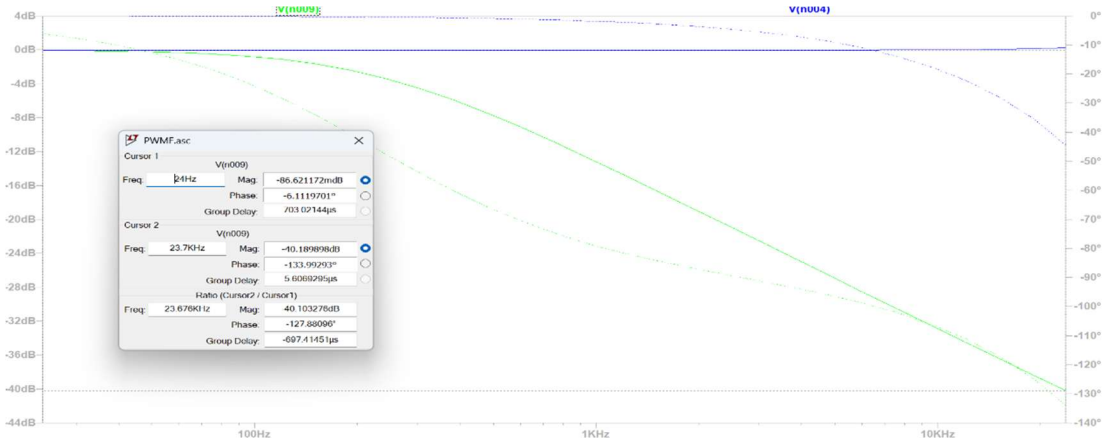


Figur 38 PWM-filter, plottet i maple

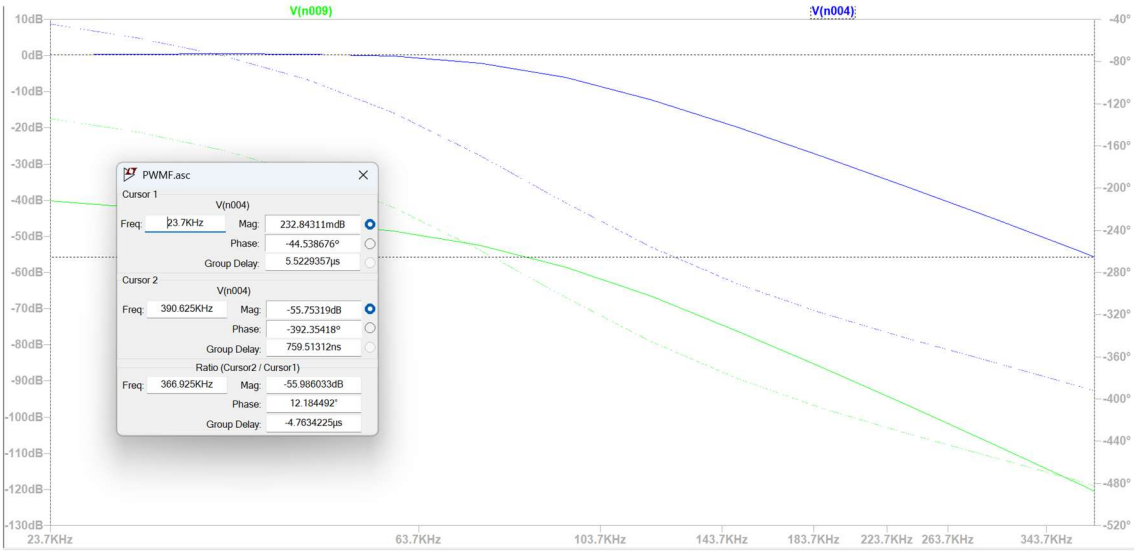
Nedenstående LT-spice simuleringen viser -55.75dB dæmpning ifht vores -46.6dB udregning, stemmer nogenlunde overens, når man medregner simulering af ikke-ideelle kompontenter.



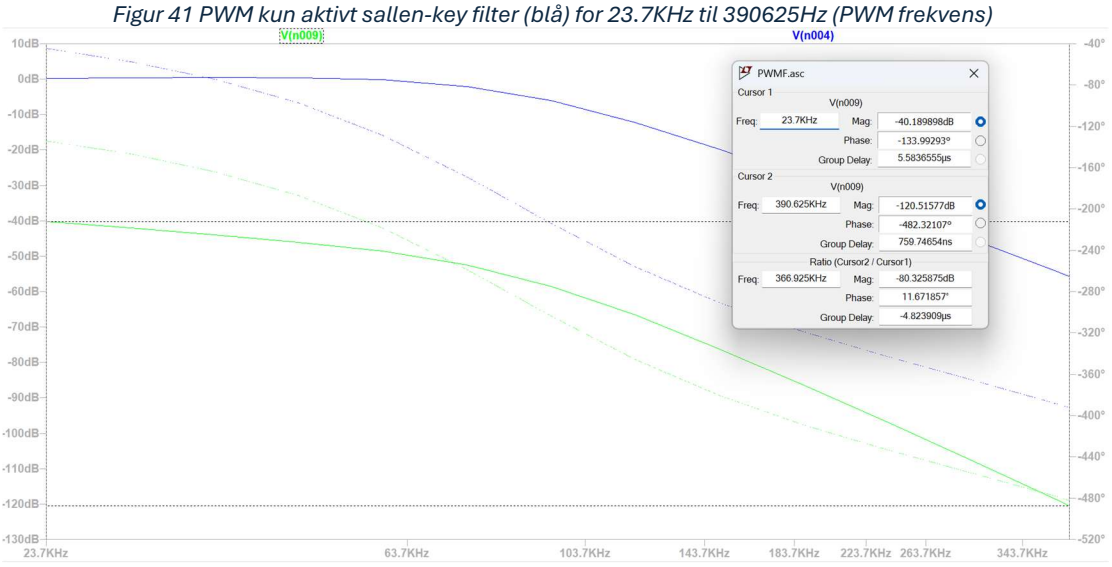
Figur 39 PWM-filteret (blå) 24-24KHz



Figur 40 PWM med sallen-key + passivt lavpasfilter (Grøn) for 24Hz til 23.7KHz



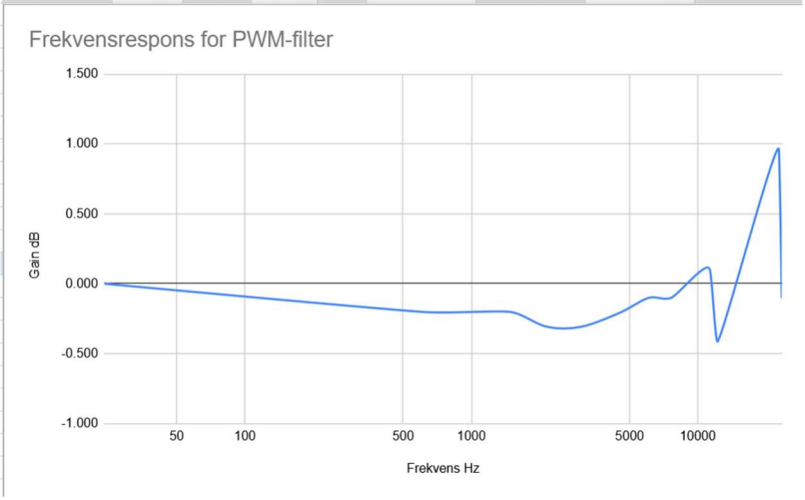
Figur 41 PWM kun aktivt sallen-key filter (blå) for 23.7KHz til 390625Hz (PWM frekvens)



Figur 42 PWM med sallen-key + passivt lavpasfilter (Grøn) for 23.7KHz til 390625Hz (PWM-frekvens)

Herunder manuelt bodeplot, hvor vi ser at der ingen væsentlig dæmpning er på signalet fra 24-24KHz. Det kraftige udsving vurderes til at målefejl pga. signalforvrængningen ved højere frekvenser, formentlig grundet op-amp havde 0-5V og ikke -5V til 5V, se figur 43. Dette bodeplot blev tegnet da vi anvendte en anden kode til signalgeneratoren, med den nye kode har vi ikke samme forvrængning.

Bit hex	Signal Fre	Min mV	Min V	Max V	Vpp	Vref 24Hz	Vpp - reference	dB
0x01	24	39.37	0.03937	3.425	3.38563	3.38563	1	0.000
0x0F	643	78.74	0.07874	3.386	3.30726	3.38563	0.98	-0.203
0x10	1524	118.1	0.1181	3.425	3.3069	3.38563	0.98	-0.204
0x1F	2167	157.5	0.1575	3.425	3.2675	3.38563	0.97	-0.308
0x20	3049	157.5	0.1575	3.425	3.2675	3.38563	0.97	-0.308
0x30	4573	118.100	0.1181	3.425	3.3069	3.38563	0.98	-0.204
0x40	6098	118.1	0.1181	3.465	3.3469	3.38563	0.99	-0.100
0x50	7622	157.5	0.1575	3.504	3.3465	3.38563	0.99	-0.101
0x7F	11314	157.5	0.1575	3.583	3.4255	3.38563	1.01	0.102
0x80	12195	275.6	0.2756	3.504	3.2284	3.38563	0.95	-0.413
0xF0	22866	275.6	0.2756	4.055	3.7794	3.38563	1.12	0.956
0xFF	23509	787.4	0.7874	4.134	3.3466	3.38563	0.99	-0.101

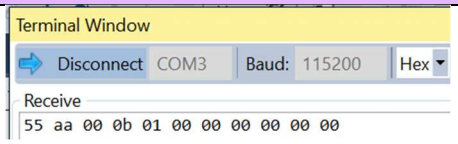
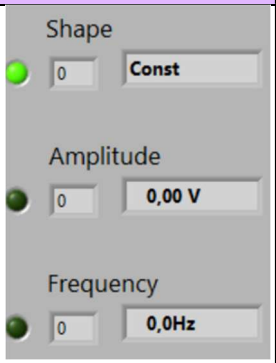
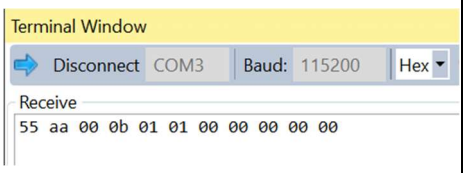
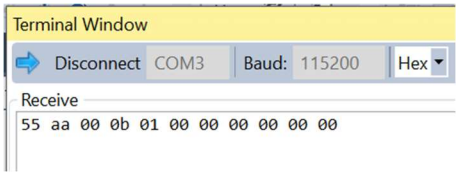
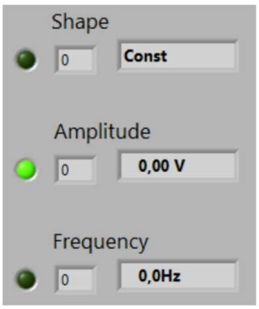
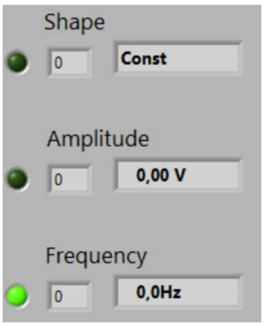


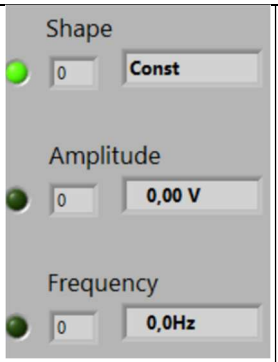
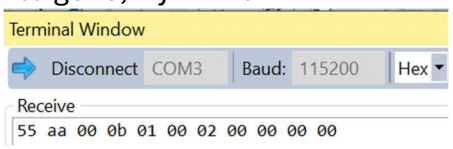
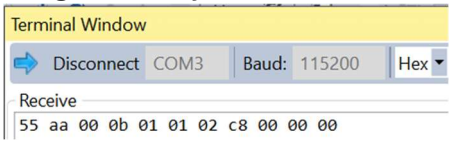
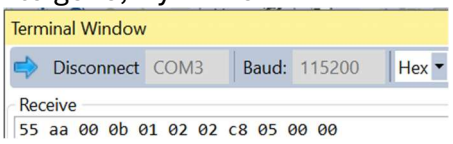
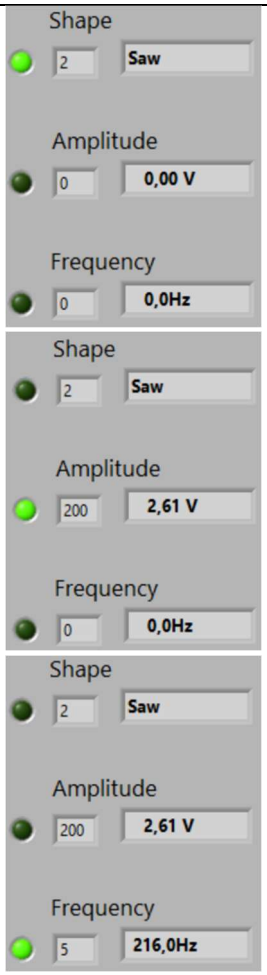
Figur 43 Manuelt bodeplot

4. Test

4.1 Oscilloscope test

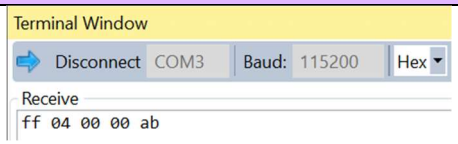
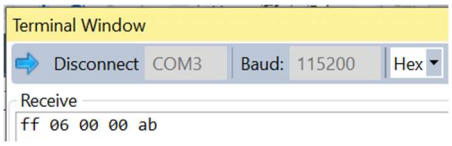
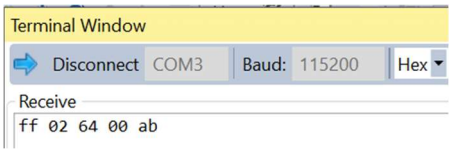
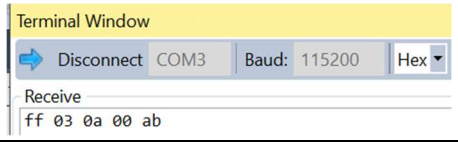
4.1.1 Labview Generator Panel

Knaptryk	MCU Datapakke til Labview (check med UART0)	Labview Generator	Kommentar
BTN3 (Reset)			1. BTN3-knappen bruges til at resette de tre parametre.
BTN2 (Run/Stop)	Ikke gældende: BTN2 bruges til at stoppe/genstarte analogt signal fra FPGA Generator panel ændrer sig ikke		
BTN1 (Select)	1.tryk  2.tryk  3.tryk 	1.tryk  2.tryk  3.tryk	1. BTN1-knappen bruges til at skifte mellem tre forskellige parametre.

			
BTN0 (Enter)	1. BTN1 vælger "Shape", SW vælger 3, tryk Enter  2. BTN1 vælger "Amplitude", SW vælger 200, tryk Enter  3. BTN1 vælger "Frequency", SW vælger 5, tryk Enter 		1. BTN0 (Enter) bruges til at opdatere SWs digitale værdi til den tilsvarende parameter.

LabVIEW-generatorpanelet kan opdateres korrekt ved hjælp af BTNx-knapperne i samspil med MCU'en.

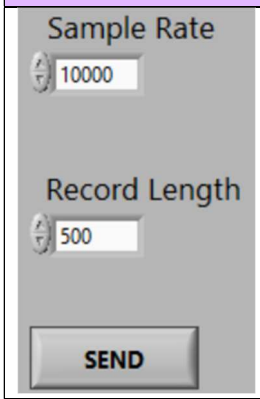
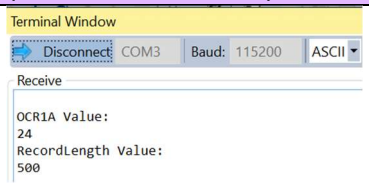
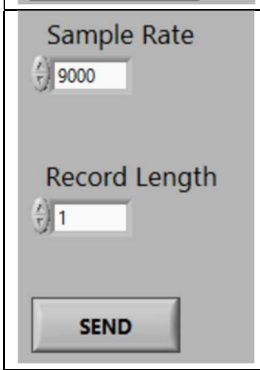
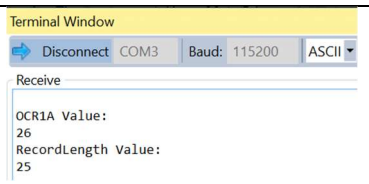
4.1.2 Labview Generator Kommandofunktionen

Knaptryk	MCU Datapakke til FPGA (check med UART0)	Kommentar
BTN3 (Reset)		Type-byte 0x04 følger protokollen (Protokollen findes i afsnit 1.3.7)
BTN2 (Run/Stop)	1.tryk  2.tryk 	Type-byte 0x05 og 0x06 følger protokollen (Protokollen findes i afsnit 1.3.7)
BTN1 (Select)	Ikke gældende: BTN1-knappen bruges til at skifte mellem tre forskellige parametre. Knappen styrer ikke direkte FPGA	
BTN0 (Enter)	1. BTN1 vælger "Shape", SW vælger 2, tryk Enter  2. BTN1 vælger "Amplitude", SW vælger 100, tryk Enter  3. BTN1 vælger "Frequency", SW vælger 10, tryk Enter 	Type-byte og Data/byte følger protokollen (Protokollen findes i afsnit 1.3.7)

Bekræftet: LabVIEW sender korrekte, præcise kommandoer til FPGAen via MCUen.

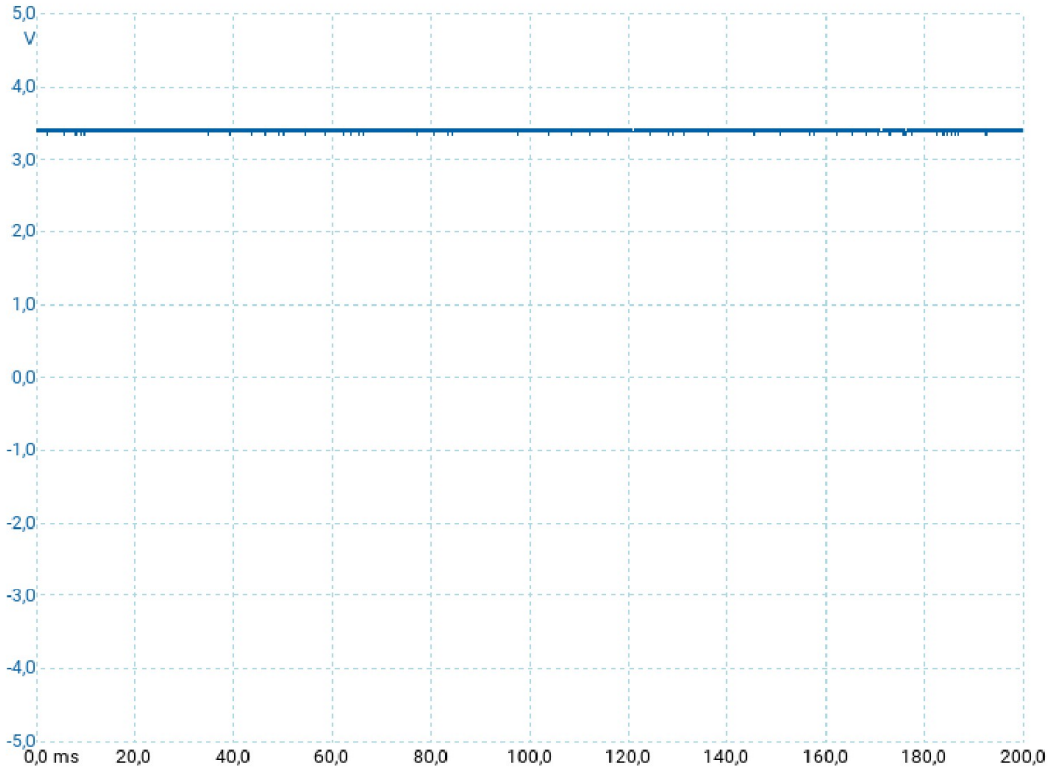
4.1.3 Sample Rate and Record Length Control

$$\text{MinimumRecordLegnth} = \frac{70 * SampleRate + (115.2\text{K} - 10 * SampleRate) - 1}{115.2\text{K} - 10 * SampleRate}$$

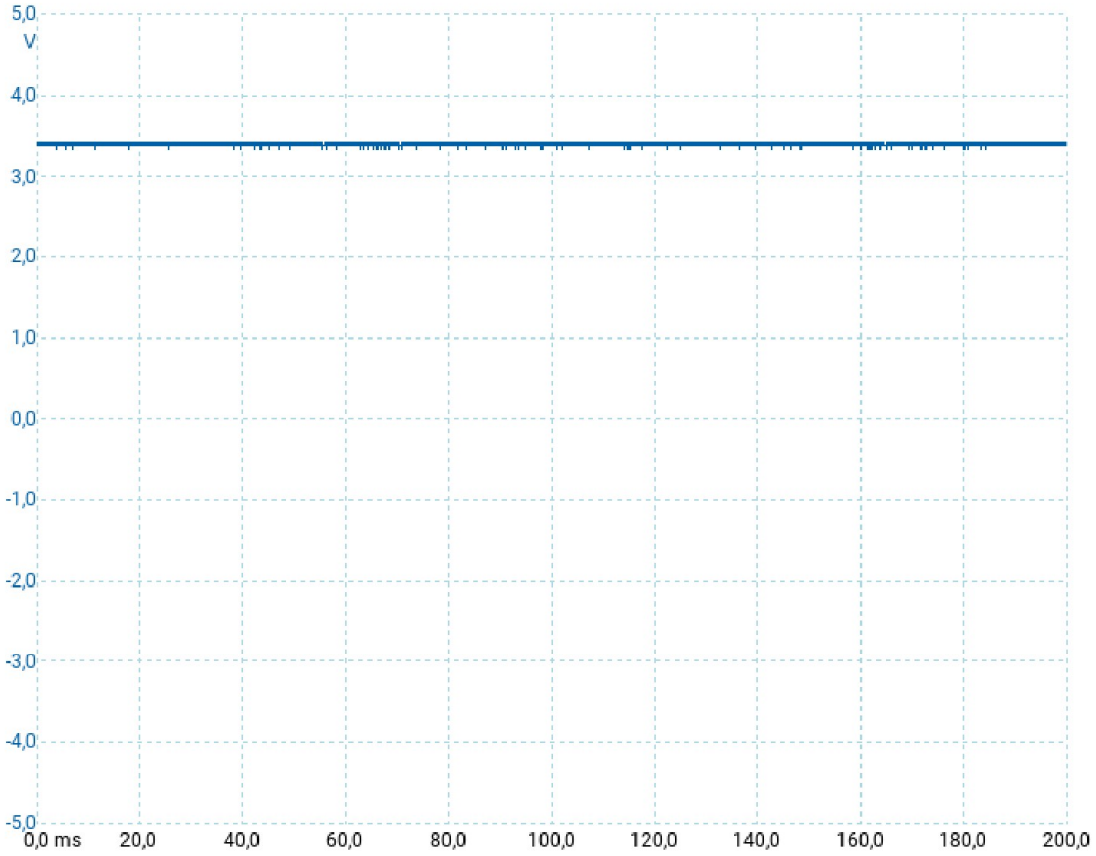
Labview	OCR1A og RecordLength (check med UART0)	Kommentar
		$OCR1A = 24$ (formlen findes i afsnit 1.3.1) $MinRecordLegnth \approx 47$ (formlen findes i afsnit 1.3.2) 500 > MinRecordLength
		$OCR1A = \frac{250\text{K}}{9\text{K}} - 1 \approx 26$ (formlen findes i afsnit 1.3.1) $MinRecordLegnth \approx 25$ (formlen findes i afsnit 1.3.2) 1 < MinRecordLength Record Length tager højde for sample rate

4.2 FPGA og filter test

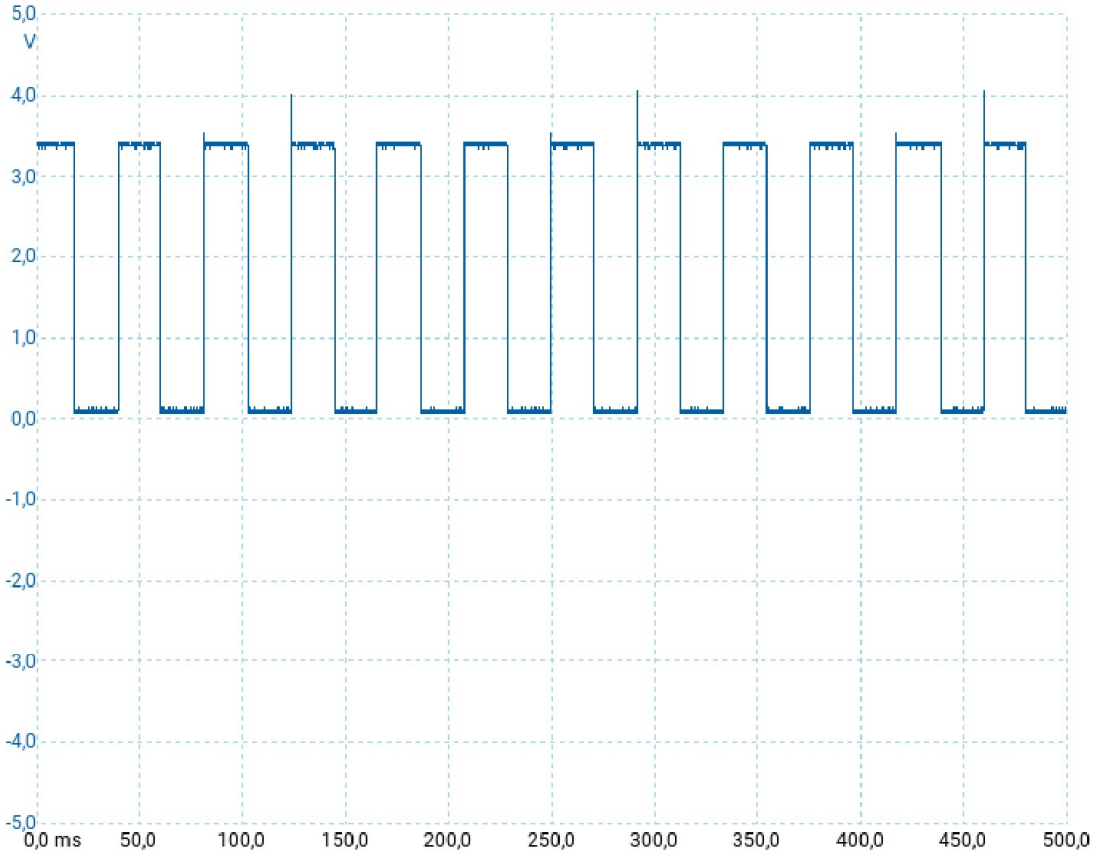
4.2.1 Signaler ved hhv. 24Hz og 24 kHz



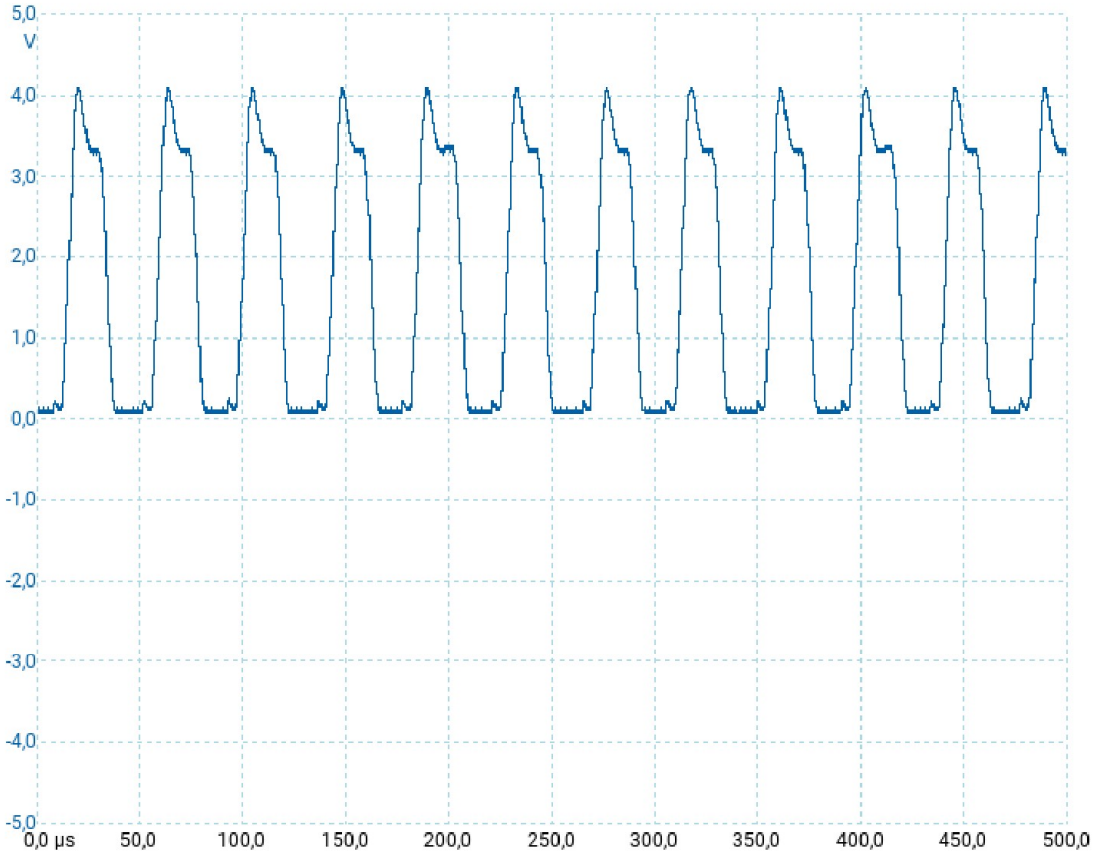
Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	278,3 Hz	23,79 Hz	314,1 Hz	50,22 Hz	74,94 Hz
A	Amplitude	Whole trace	78,61 mV	78,59 mV	3,277 V	2,873 V	994,3 mV
A	Minimum	Whole trace	3,346 V	39,37 mV	3,346 V	443,7 mV	1,033 V
A	Maximum	Whole trace	3,425 V	3,386 V	3,701 V	3,397 V	35,57 mV



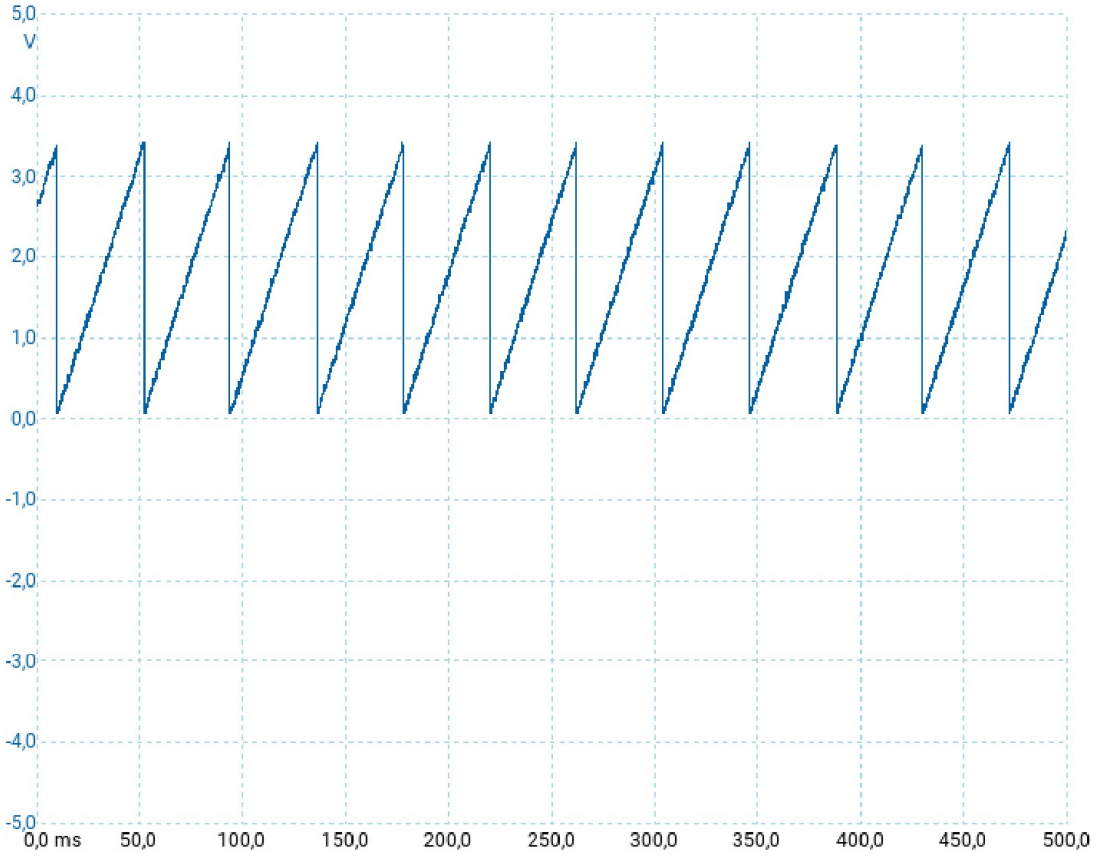
Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	462,4 Hz	341 Hz	462,4 Hz	401,7 Hz	60,67 Hz
A	Amplitude	Whole trace	78,67 mV	78,63 mV	78,67 mV	78,65 mV	21,89 µV
A	Minimum	Whole trace	3,346 V	3,346 V	3,346 V	3,346 V	0 V
A	Maximum	Whole trace	3,425 V	3,425 V	3,425 V	3,425 V	0 V



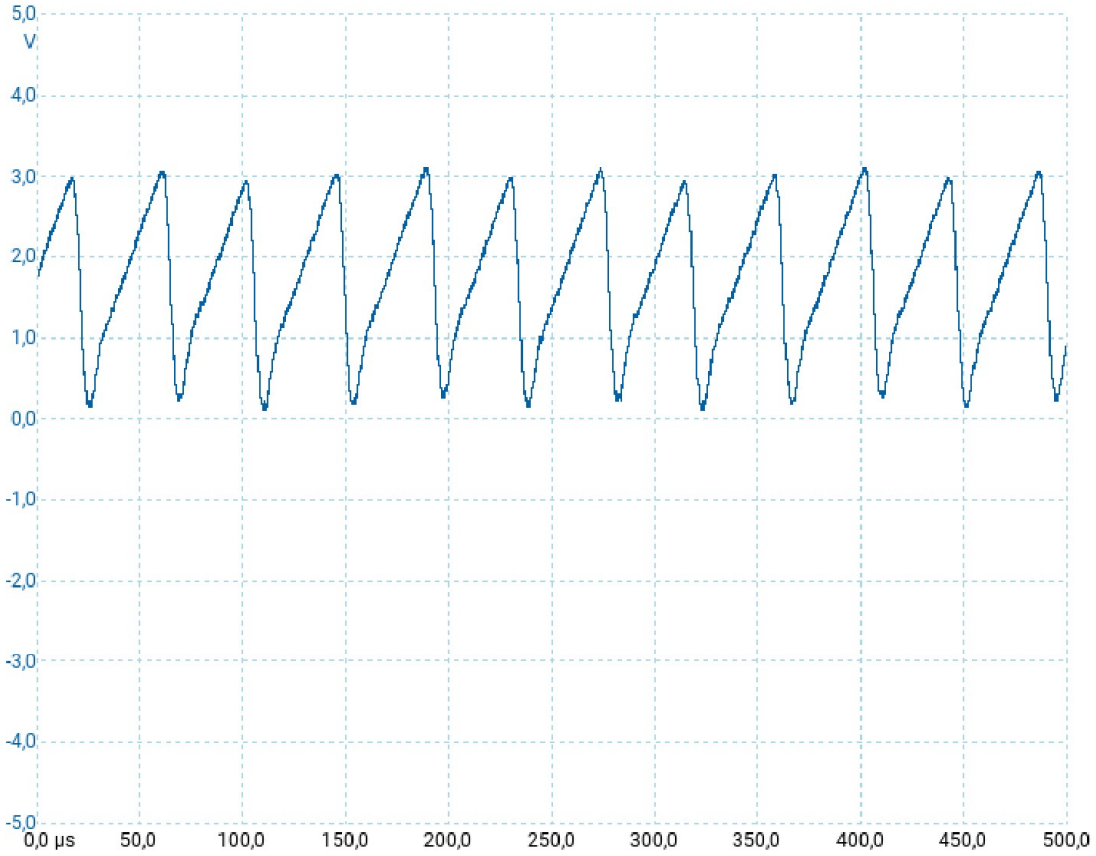
Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,81 Hz	23,81 Hz	23,81 Hz	23,81 Hz	0 Hz
A	Amplitude	Whole trace	3,294 V	3,291 V	3,294 V	3,293 V	0 V
A	Minimum	Whole trace	78,74 mV	39,37 mV	78,74 mV	62,99 mV	19,29 mV
A	Maximum	Whole trace	4,055 V	4,016 V	4,055 V	4,047 V	15,79 mV



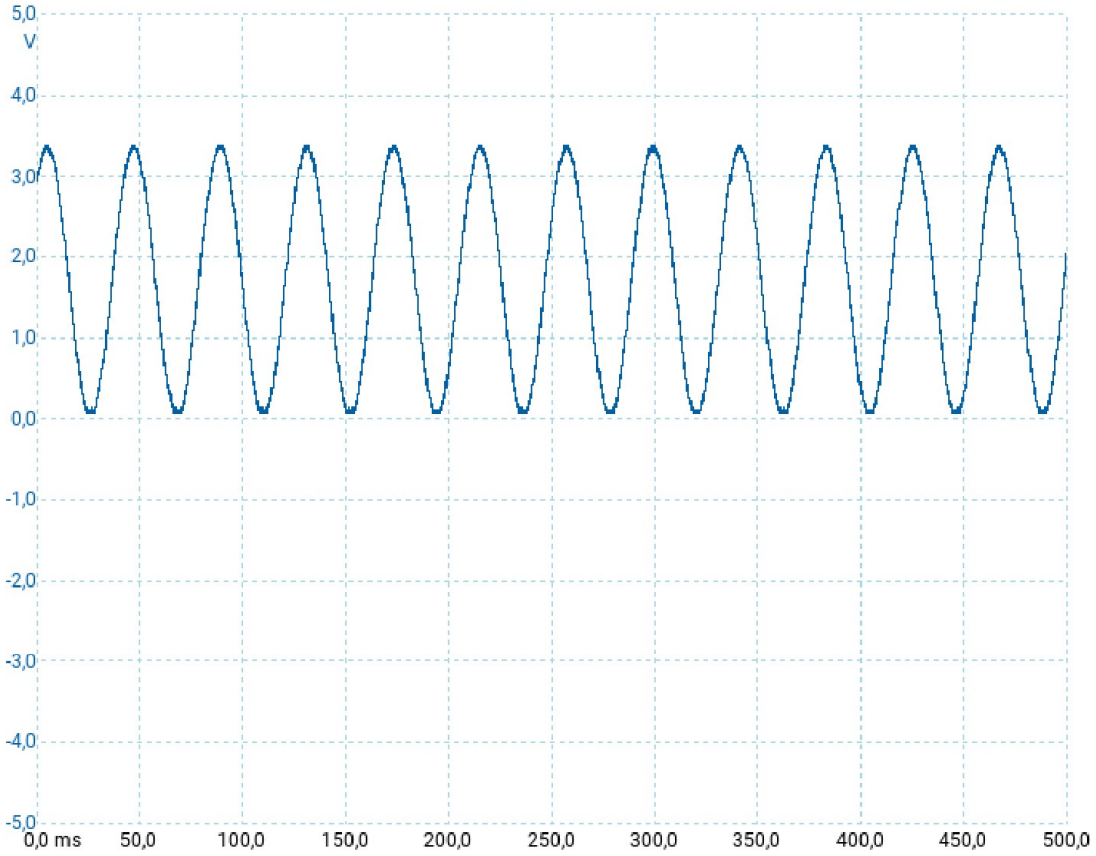
Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,45 kHz	23,44 kHz	23,65 kHz	23,49 kHz	66,19 Hz
A	Amplitude	Whole trace	3,263 V	3,192 V	3,264 V	3,241 V	29,78 mV
A	Minimum	Whole trace	78,74 mV	39,37 mV	78,74 mV	54,61 mV	19,18 mV
A	Maximum	Whole trace	4,094 V	4,094 V	4,134 V	4,098 V	11,13 mV



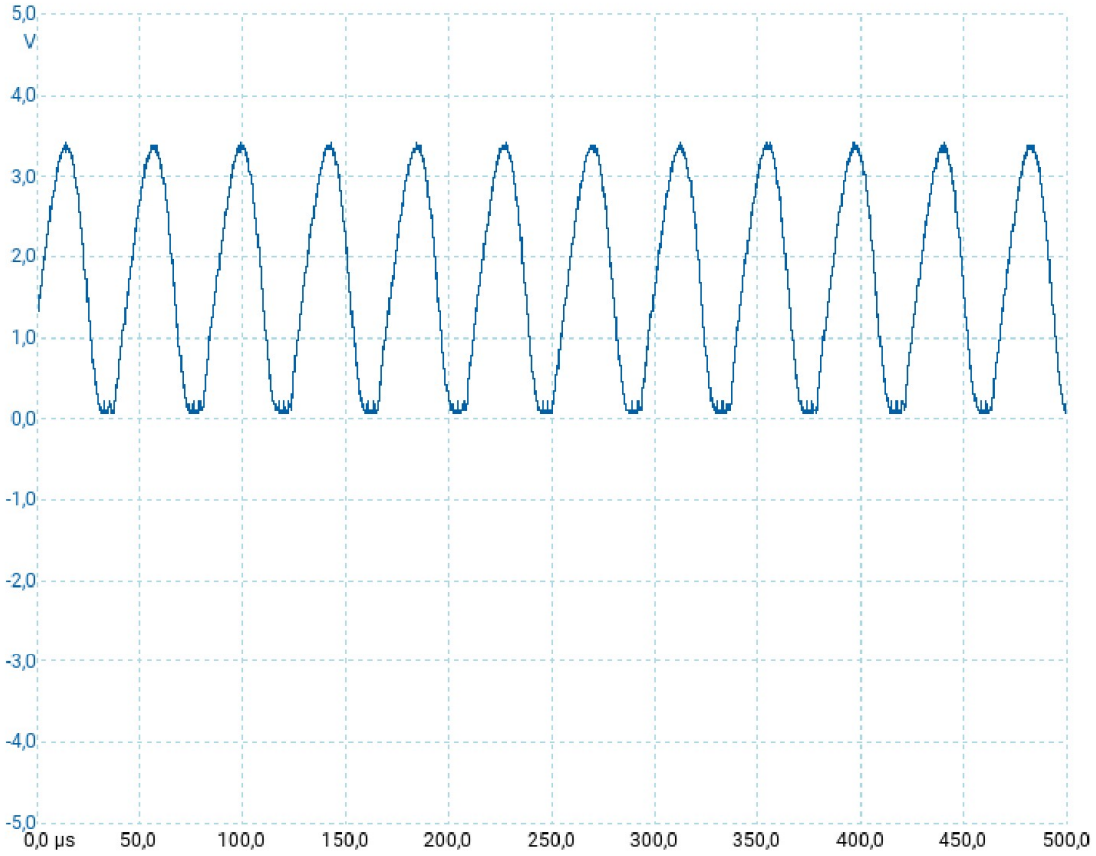
Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,82 Hz	23,78 Hz	23,82 Hz	23,8 Hz	20,25 mHz
A	Amplitude	Whole trace	2,518 V	2,085 V	2,519 V	2,374 V	204,3 mV
A	Minimum	Whole trace	78,74 mV	78,74 mV	78,74 mV	78,74 mV	0 V
A	Maximum	Whole trace	3,425 V	3,425 V	3,425 V	3,425 V	945,2 µV



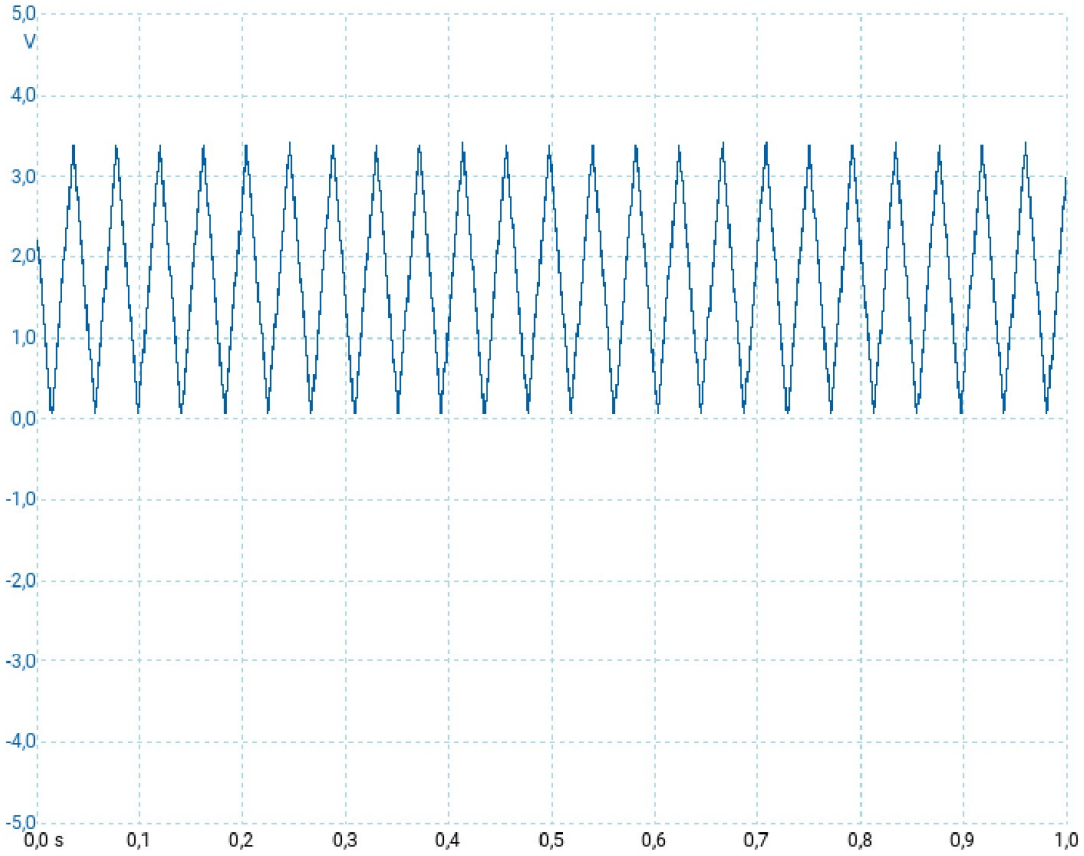
Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,43 kHz	23,23 kHz	23,59 kHz	23,48 kHz	67,31 Hz
A	Amplitude	Whole trace	2,528 V	1,654 V	2,622 V	2,026 V	384,5 mV
A	Minimum	Whole trace	118,1 mV	78,74 mV	118,1 mV	116,8 mV	7,068 mV
A	Maximum	Whole trace	3,11 V	3,071 V	3,15 V	3,112 V	18,93 mV



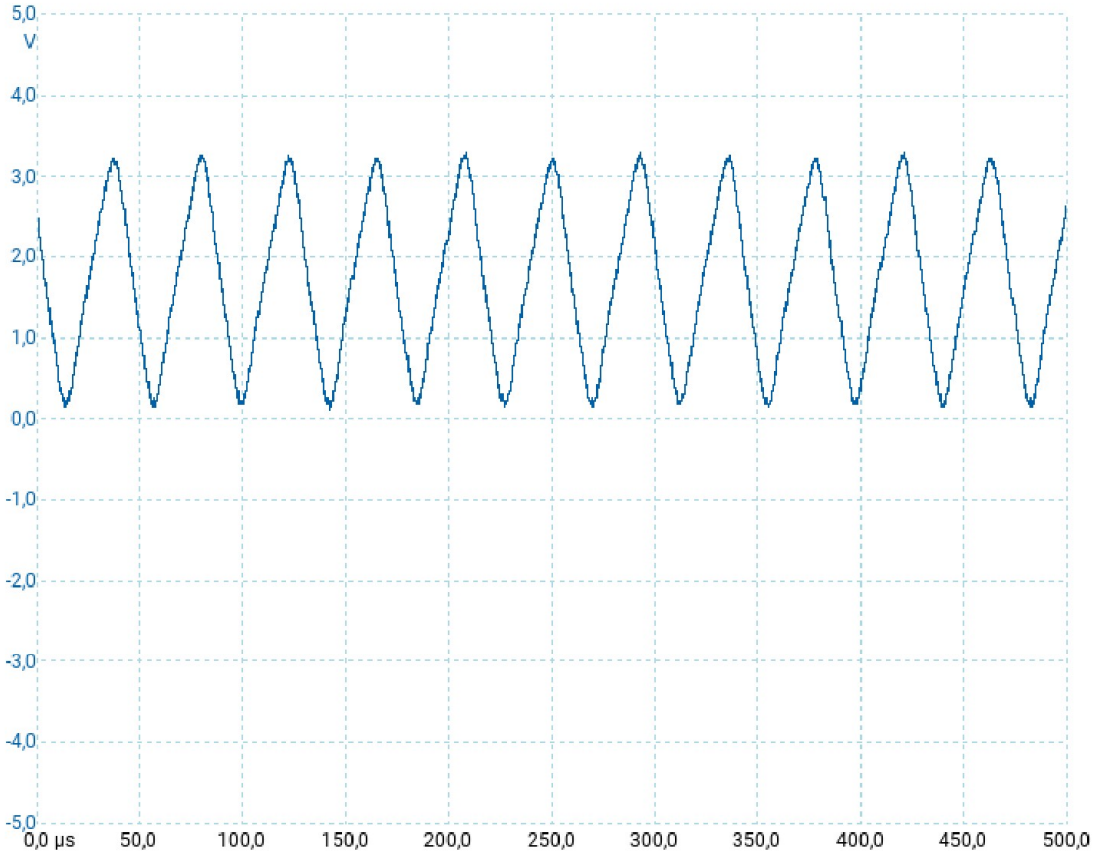
Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,81 Hz	23,81 Hz	23,81 Hz	23,81 Hz	0 Hz
A	Amplitude	Whole trace	3,232 V	3,232 V	3,232 V	3,232 V	0 V
A	Minimum	Whole trace	78,74 mV	78,74 mV	78,74 mV	78,74 mV	0 V
A	Maximum	Whole trace	3,386 V	3,386 V	3,386 V	3,386 V	987,8 µV



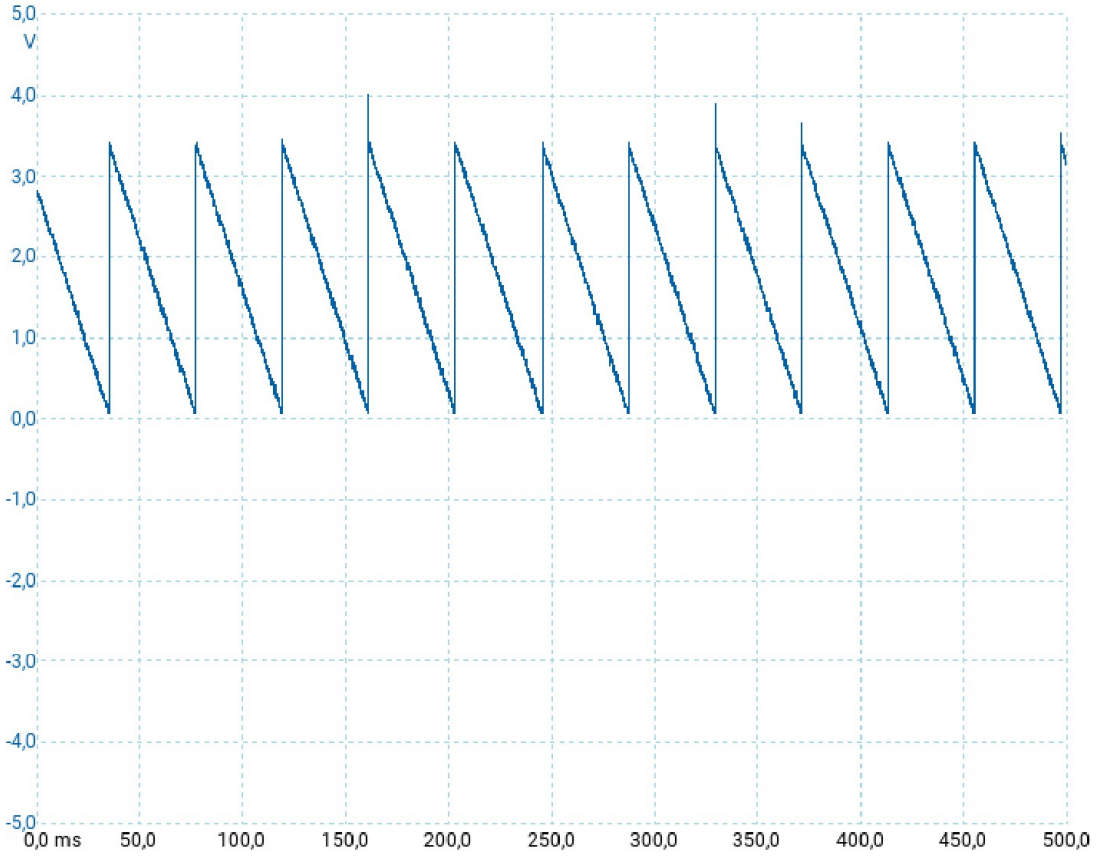
Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,49 kHz	23,49 kHz	23,51 kHz	23,5 kHz	0 Hz
A	Amplitude	Whole trace	3,249 V	3,247 V	3,26 V	3,251 V	3,368 mV
A	Minimum	Whole trace	78,74 mV	39,37 mV	78,74 mV	72,18 mV	14,67 mV
A	Maximum	Whole trace	3,425 V	3,425 V	3,425 V	3,425 V	2,623 mV



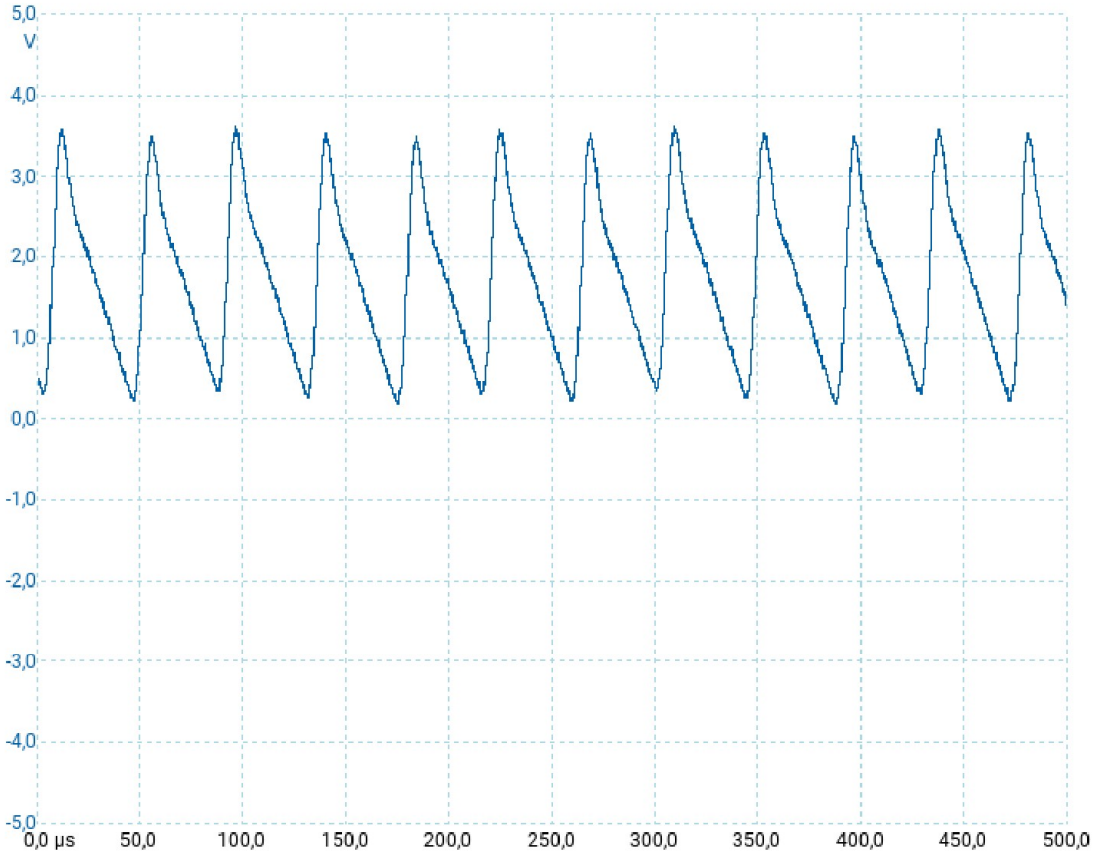
Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,81 Hz	23,81 Hz	23,82 Hz	23,81 Hz	3,254 mHz
A	Amplitude	Whole trace	2,203 V	2,203 V	2,322 V	2,262 V	59,64 mV
A	Minimum	Whole trace	78,74 mV	78,74 mV	78,74 mV	78,74 mV	0 V
A	Maximum	Whole trace	3,425 V	3,425 V	3,425 V	3,425 V	0 V



Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,49 kHz	23,48 kHz	23,52 kHz	23,5 kHz	21,36 Hz
A	Amplitude	Whole trace	2,891 V	2,856 V	2,959 V	2,897 V	20,06 mV
A	Minimum	Whole trace	118,1 mV	118,1 mV	157,5 mV	122,8 mV	12,79 mV
A	Maximum	Whole trace	3,307 V	3,307 V	3,346 V	3,308 V	6,906 mV

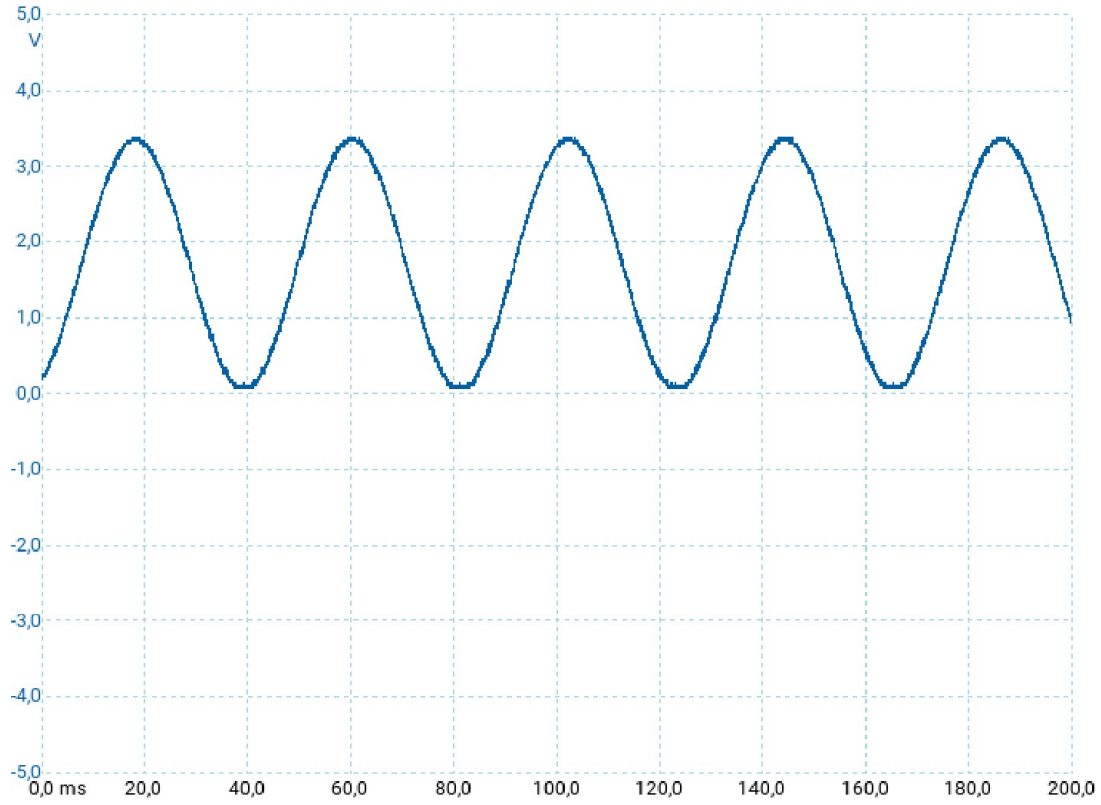


Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,81 Hz	23,81 Hz	23,81 Hz	23,81 Hz	4,591 mHz
A	Amplitude	Whole trace	1,969 V	1,969 V	2,363 V	2,231 V	185,3 mV
A	Minimum	Whole trace	78,74 mV	78,74 mV	78,74 mV	78,74 mV	0 V
A	Maximum	Whole trace	4,016 V	4,016 V	4,055 V	4,042 V	18,6 mV

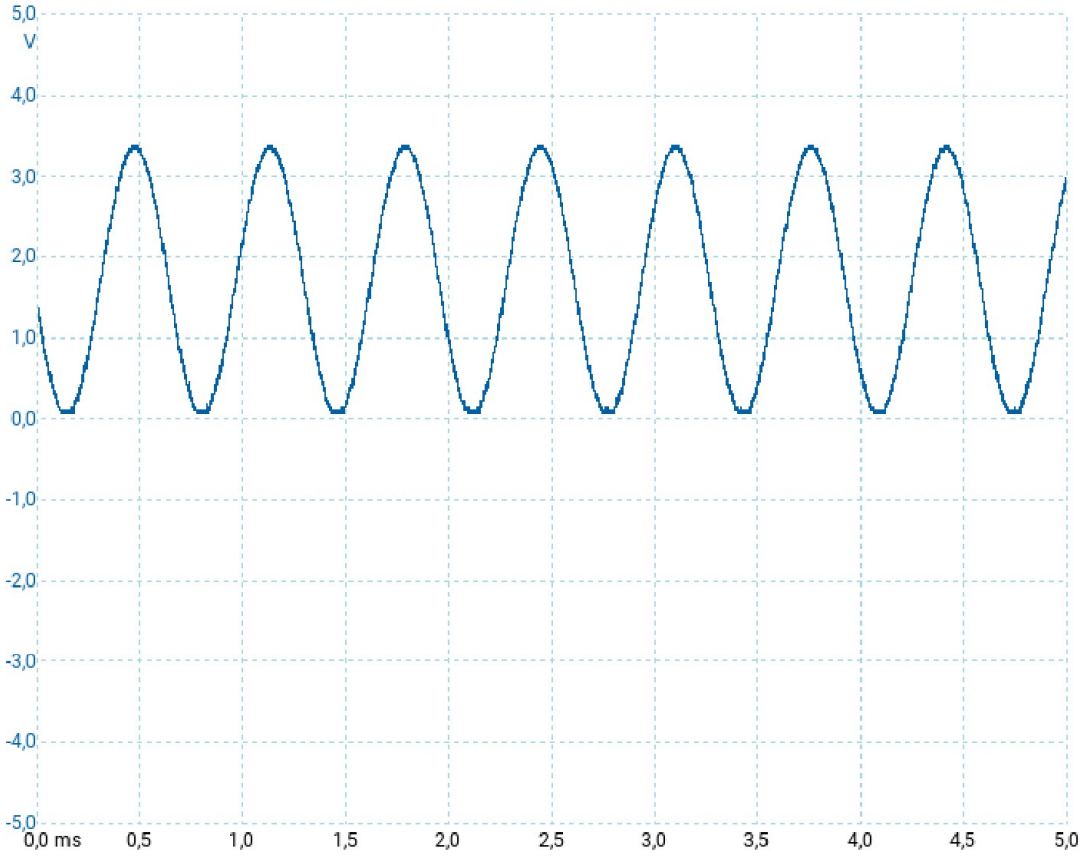


Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,44 kHz	23,43 kHz	23,59 kHz	23,46 kHz	46,28 Hz
A	Amplitude	Whole trace	3,107 V	3,031 V	3,143 V	3,105 V	24,22 mV
A	Minimum	Whole trace	196,9 mV	157,5 mV	196,9 mV	183,7 mV	18,56 mV
A	Maximum	Whole trace	3,622 V	3,583 V	3,622 V	3,615 V	14,73 mV

4.2.2 Sinussignaler ved forskellige frekvenser

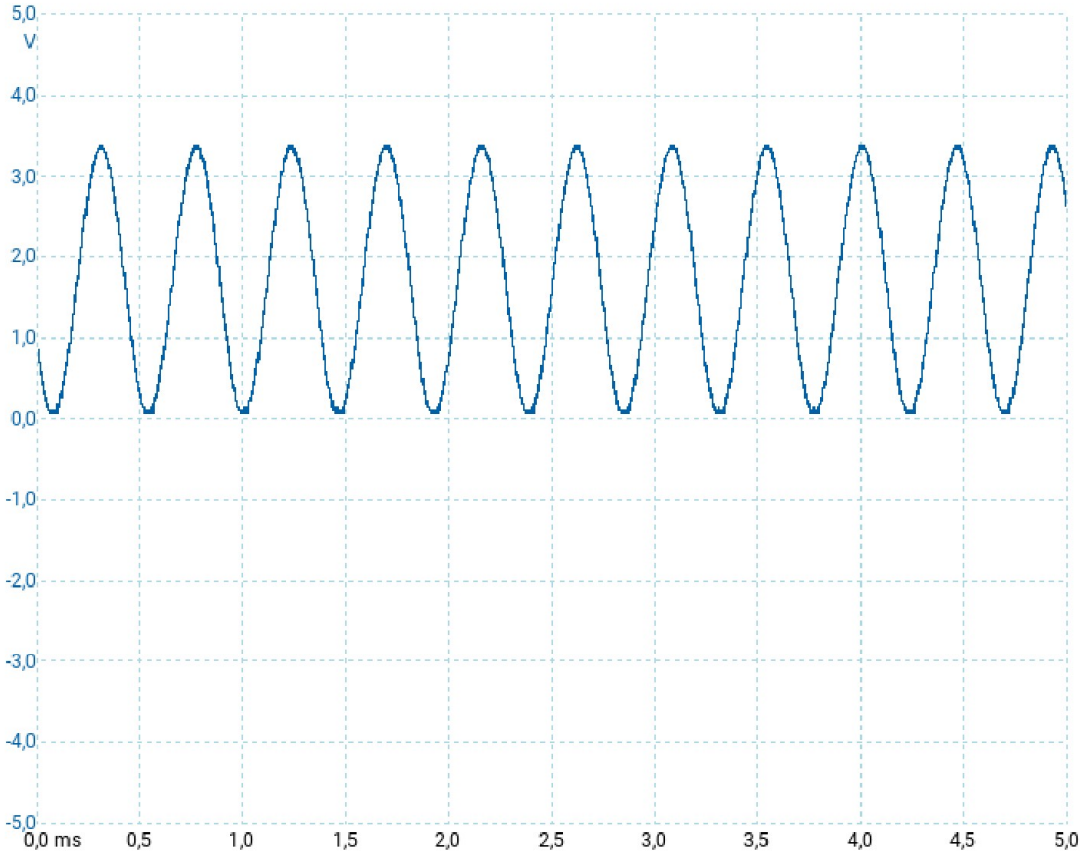


Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,8 Hz	23,79 Hz	4,124 MHz	589,1 kHz	1,443 MHz
A	Amplitude	Whole trace	3,233 V	118,1 mV	3,243 V	2,789 V	1,091 V
A	Minimum	Whole trace	78,74 mV	39,37 mV	354,3 mV	112,5 mV	99,66 mV
A	Maximum	Whole trace	3,386 V	551,2 mV	3,386 V	2,981 V	991,9 mV



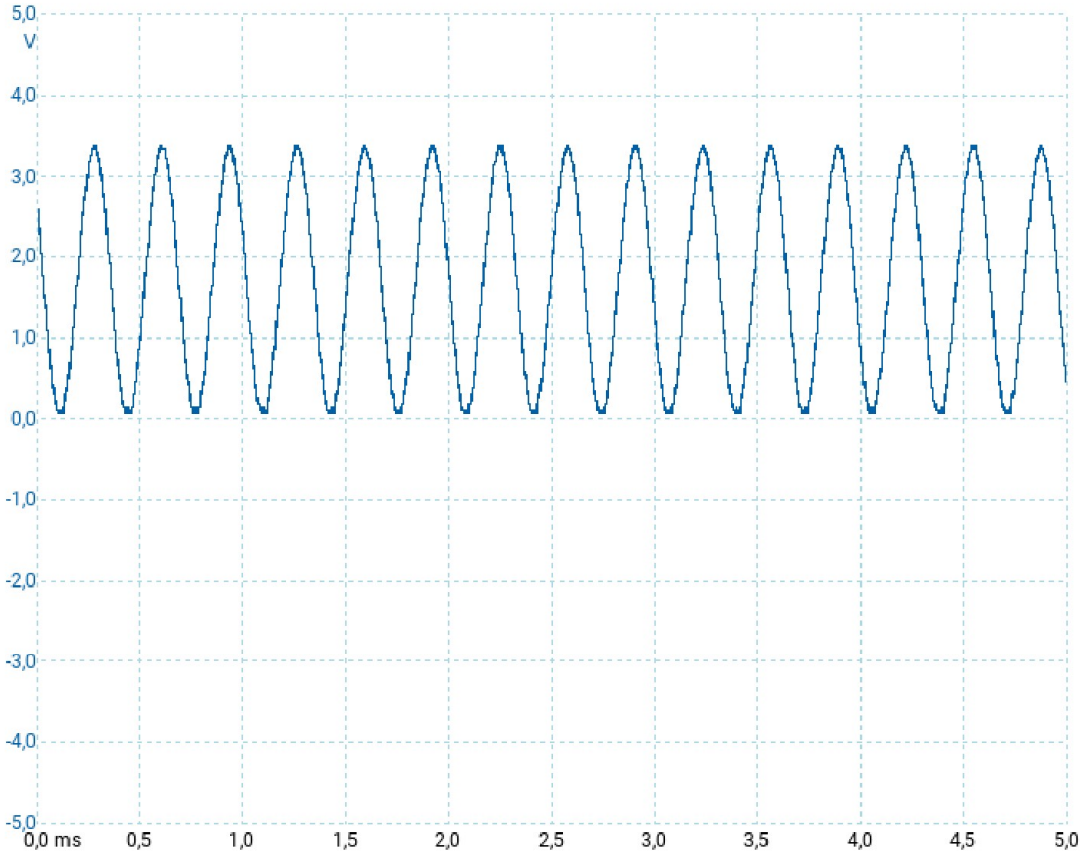
23/06/2026 22.56.38

Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	1,524 kHz	1,523 kHz	1,524 kHz	1,524 kHz	562,9 mHz
A	Amplitude	Whole trace	3,237 V	3,236 V	3,245 V	3,238 V	2,459 mV
A	Minimum	Whole trace	78,74 mV	39,37 mV	78,74 mV	72,52 mV	14,36 mV
A	Maximum	Whole trace	3,386 V	3,386 V	3,386 V	3,386 V	826,4 µV



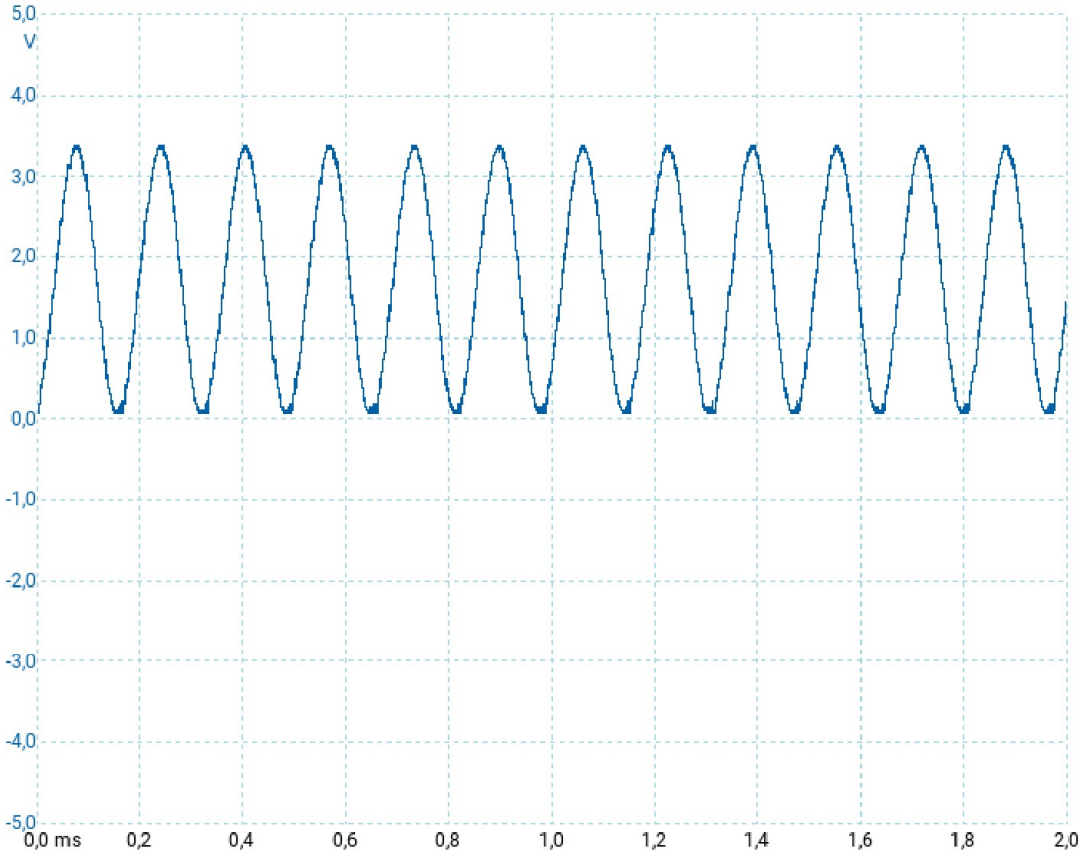
23/06/2026 22.57.23

Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	2,167 kHz	2,166 kHz	2,167 kHz	2,166 kHz	344,5 mHz
A	Amplitude	Whole trace	3,236 V	3,167 V	3,237 V	3,213 V	32,96 mV
A	Minimum	Whole trace	78,74 mV	78,74 mV	78,74 mV	78,74 mV	0 V
A	Maximum	Whole trace	3,386 V	3,386 V	3,425 V	3,399 V	18,55 mV

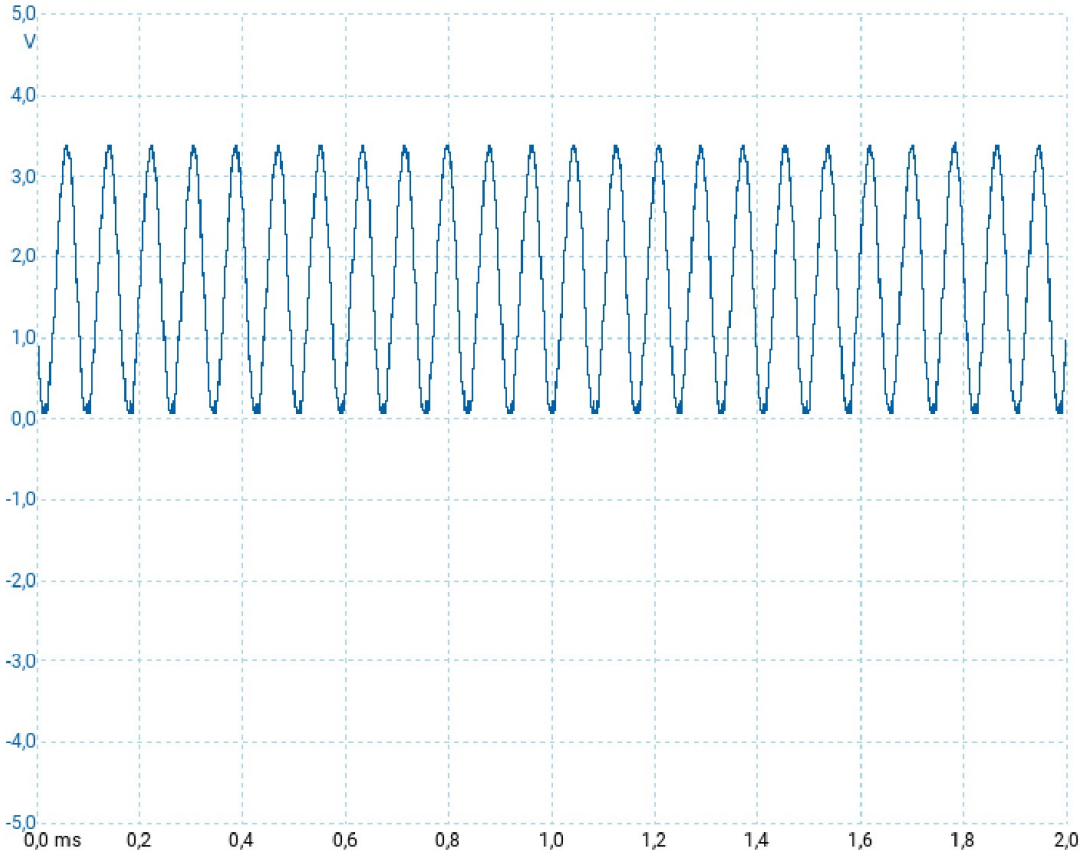


23/06/2026 22.58.04

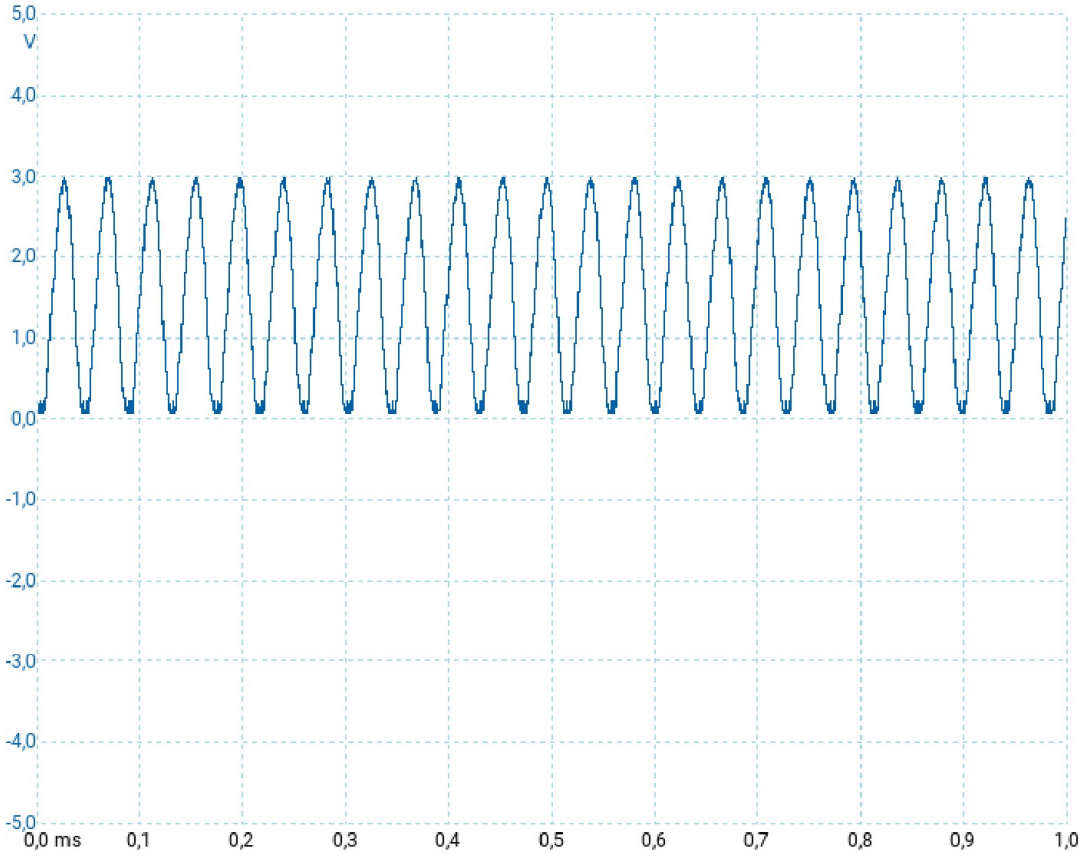
Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	3,047 kHz	3,047 kHz	3,048 kHz	3,047 kHz	1,831 Hz
A	Amplitude	Whole trace	3,234 V	3,158 V	3,242 V	3,223 V	26,15 mV
A	Minimum	Whole trace	78,74 mV	39,37 mV	78,74 mV	77,31 mV	7,37 mV
A	Maximum	Whole trace	3,386 V	3,386 V	3,425 V	3,392 V	13,46 mV



Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	6,094 kHz	6,094 kHz	6,096 kHz	6,095 kHz	2,971 Hz
A	Amplitude	Whole trace	3,233 V	3,233 V	3,252 V	3,241 V	7,134 mV
A	Minimum	Whole trace	78,74 mV	39,37 mV	78,74 mV	77,55 mV	6,749 mV
A	Maximum	Whole trace	3,386 V	3,386 V	3,425 V	3,402 V	18,99 mV



Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	12,19 kHz	12,19 kHz	12,19 kHz	12,19 kHz	0 Hz
A	Amplitude	Whole trace	3,251 V	3,233 V	3,252 V	3,25 V	2,884 mV
A	Minimum	Whole trace	78,74 mV	78,74 mV	78,74 mV	78,74 mV	67,87 µV
A	Maximum	Whole trace	3,425 V	3,386 V	3,425 V	3,423 V	8,354 mV

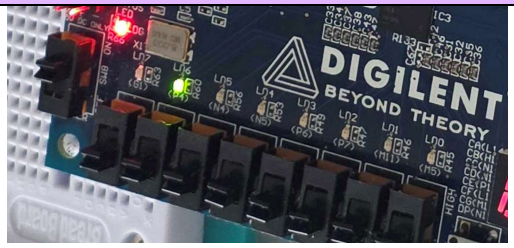
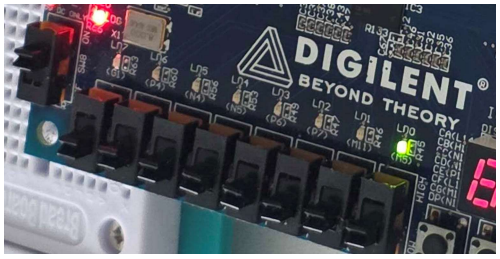


Channel	Name	Span	Value	Min.	Max.	x̄	σ
A	Frequency	Whole trace	23,5 kHz	23,49 kHz	23,5 kHz	23,5 kHz	0 Hz
A	Amplitude	Whole trace	2,825 V	2,78 V	2,834 V	2,795 V	17,64 mV
A	Minimum	Whole trace	78,74 mV	39,37 mV	78,74 mV	70,74 mV	15,84 mV
A	Maximum	Whole trace	2,992 V	2,992 V	3,031 V	3,022 V	17,27 mV

4.3 Integration mellem MCU, FPGA og Filtre

4.3.1 SPI Test

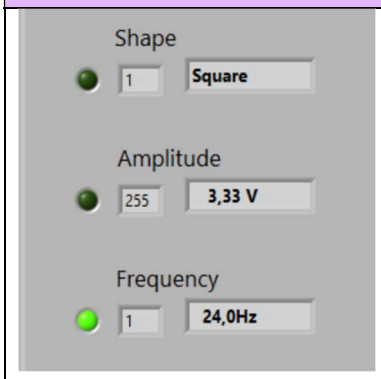
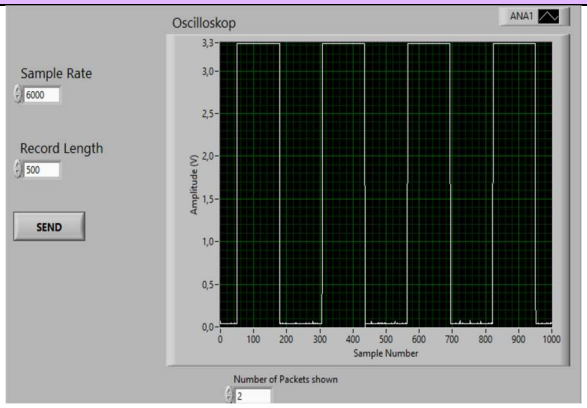
SPI-test med en byte

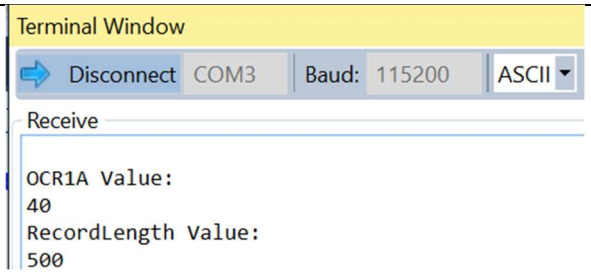
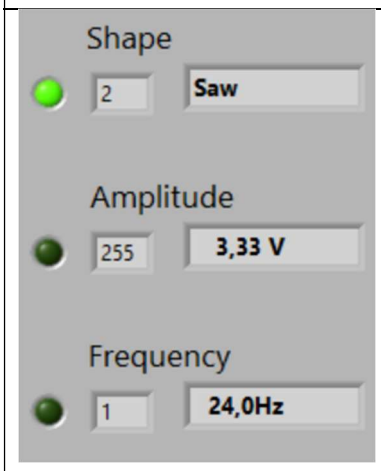
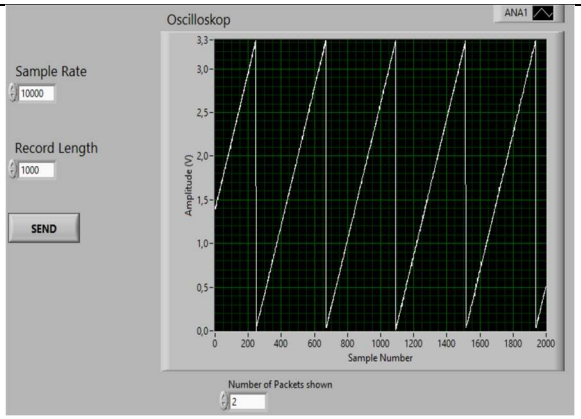
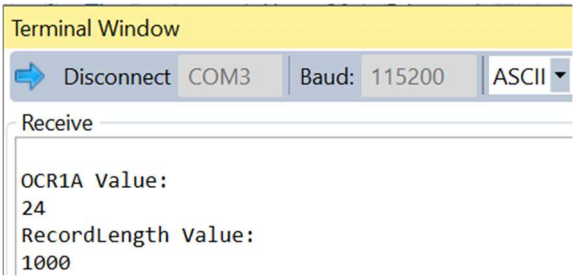
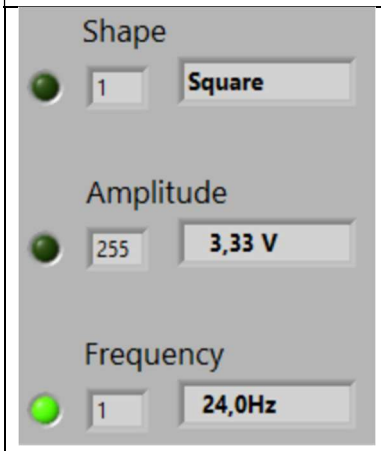
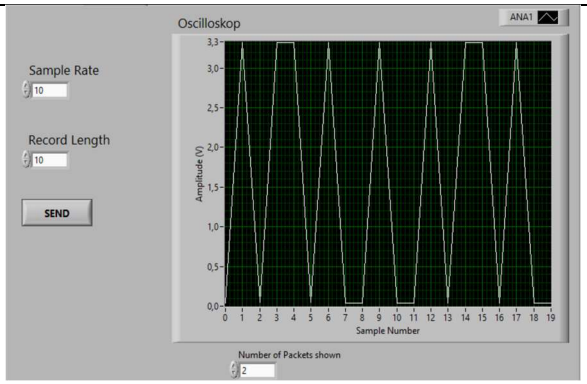
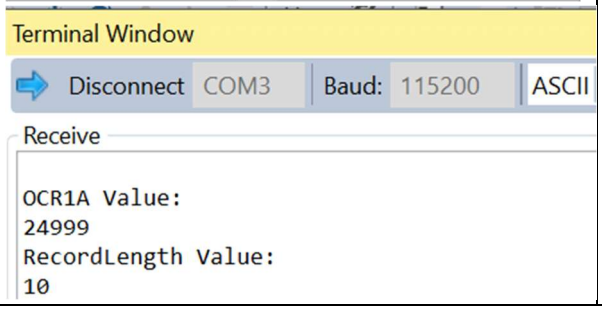
MCU	FPGA	Kommentar
MCU sender 0x40 og 0x01 i while-loop til FPGAen.	 	Via SPI er der shiftet bits ind, og grupperet i 8. Derefter vist på LED.

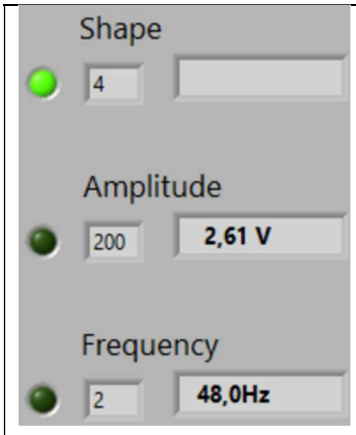
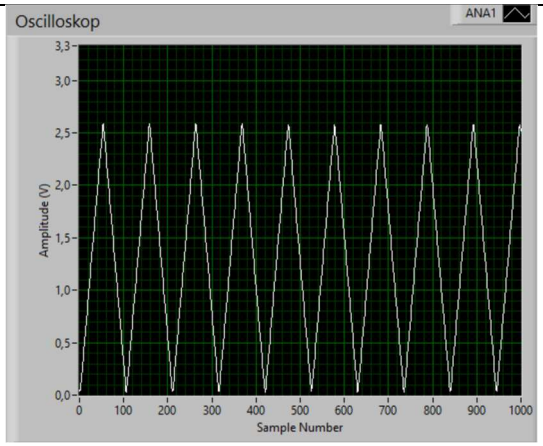
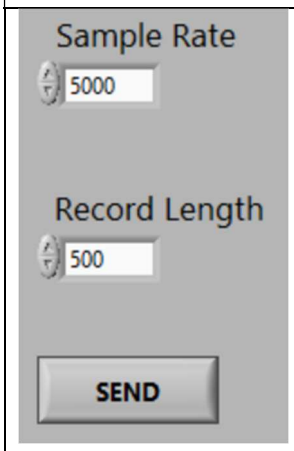
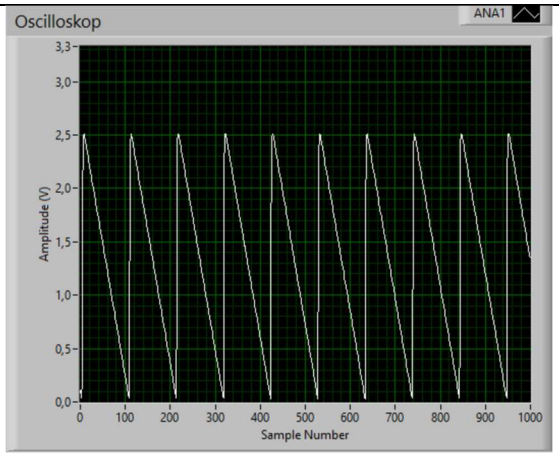
4.3.2 Dataintegritet

Formlen til beregning signalfrekvens i oscilloskop.

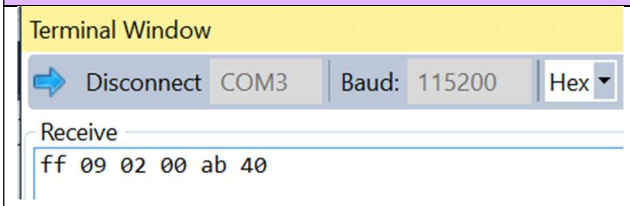
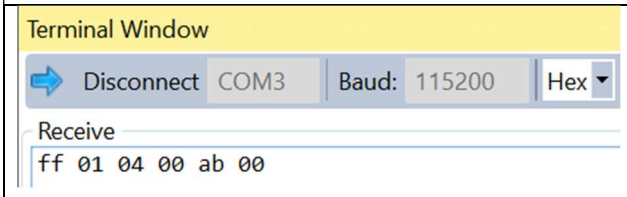
$$SignalFrekvens = \frac{Sample\ Rate}{Længden\ af\ en\ period}$$

Labview Generator	Labview Oscilloskop & PC UART0 Terminal	Kommentar
		• Amplitude: 3.3V • Frekvens: $\frac{6K}{260} \approx 23Hz$ • OCR1A (formlen findes i afsnit 1.3.1) og Record Length (formlen findes i afsnit 1.3.2) passer til formlerne.

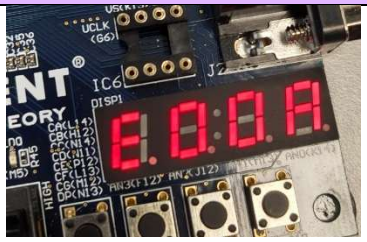
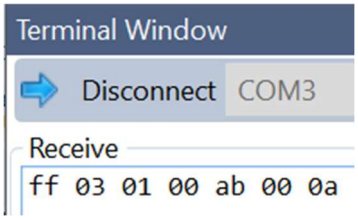
		
	 	• Amplitude: 3.3V • Frekvens: • $\frac{10}{400} \approx 25Hz$ • Sample Rate kan køre op til 10K. • Oscilloskopet kan køre med 1000 ADC-målinger i hver pakke for alle samplerate. • OCR1A (formlen findes i afsnit 1.3.1) og Record Length (formlen findes i afsnit 1.3.2) passer til formlerne.
	 	• Amplitude: 3.3V • Sample Rate kan køre ned til 10. • 10, som svar til kun 1 sample per 2,4 perioder. Det fører til • OCR1A (formlen findes i afsnit 1.3.1) og Record Length (formlen findes i afsnit 1.3.2) passer til formlerne.

		• Shape 4 (ekstra shape): Trekant
		• Shape 5 (ekstra shape): Inveteret Sawtak

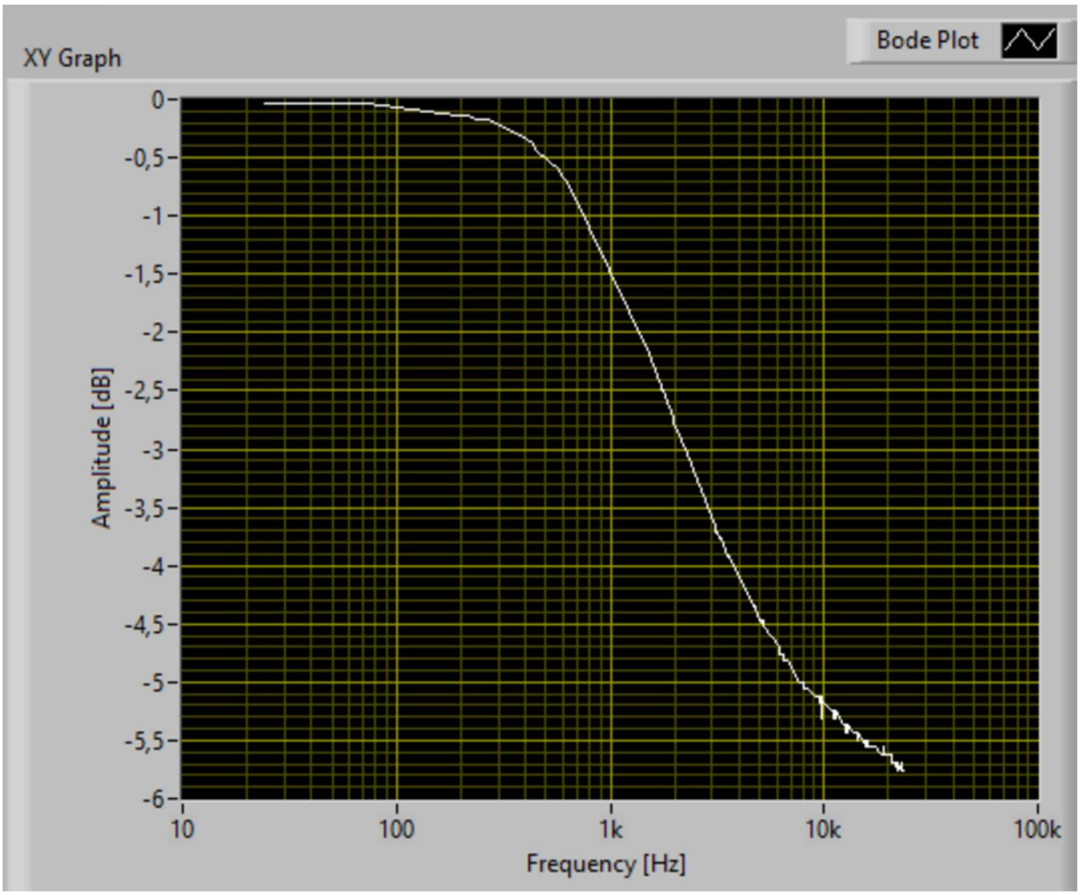
4.3.3 Acknowledge Byte

Package fra MCU til FPGA	Acknowledge Byte fra FPGA	Kommentar
 De første 5 byte er den package der sendes fra MCU til FPGA	FPGA sender 0x40 til MCU efter check package	• hvoraf vi bevidst havde skrevet en forkert adresseværdi. Her ses, at MISO sender 0x40 tilbage, altså fejl i adressen (se tabel 2 i afsnit 2.2)
 De første 5 byte er den package der sendes fra MCU til FPGA	FPGA sender 0x00 til MCU efter check package	• 0x00 svar til ingen fejl (se tabel 2 i afsnit 2.2)

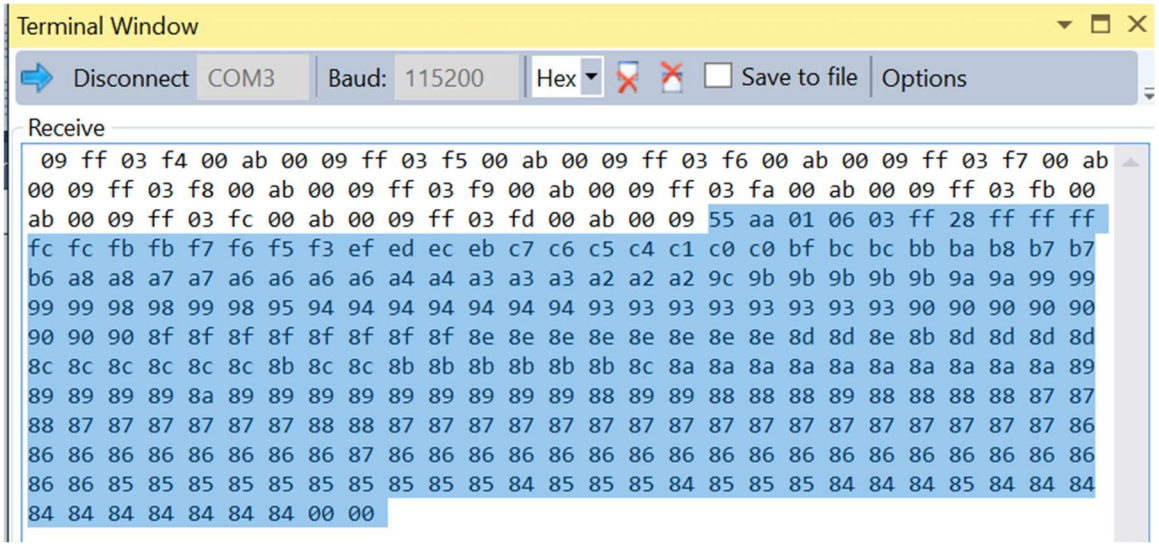
SPI-test med wrong package

MCU	Visning af antal fejl	Kommentar
MCU sendes 10K data package, og 10 af dem har fejl med forkert checksum. Endnu tælles antal fejl, og sammenligner med FPGA'ens fejltæller.	 	• Der er talt korrekt op til A (10) både med FPGA og UART0. • FPGA: I terminalens 6'te byte ses MISO byten, som FPGA'en sender tilbage. Der ses, at sendte pakke har ingen fejl, se tabel x. • UART0: De første fem bytes udgør data-pakken. Den sjette byte viser, om FPGA'en har fundet en fejl i pakken. Den sidste byte viser det totale antal fejl, som MCU har registreret indtil nu.

4.3.4 Bode Plot



Figur 44. Bodeplot i LabView.



Figur 45. Bodeplot målinger fra terminal.

Testresultaterne fra figur 50 og 51 viser, at signalet næsten ikke dæmpes ved lave frekvenser, mens vi omkring den teoretiske knækfrekvens (≈1061Hz) observerer en dæmpning på ca. -3dB, hvilket stemmer godt overens med den teoretiske beregning og dermed bekræfter, at RC-lavpasfilteret og Bodeplot-funktionen fungerer korrekt. Ved højere frekvenser (>10kHz) afviger kurven fra den teoretiske dæmpning på -20dB/dekade og flader ud - dette skyldes sandsynligvis primært ADC'ens 8-bit opløsning samt måleusikkerhed pga. signalforvrængning ved høje frekvenser, og ikke en fejl i selve filterdesignet.

5. Konklusion

I dette projekt har vi udviklet et oscilloskop baseret på Atmega2560-boardet (MCUen) og en signalgenerator baseret på FPGA Basys 2-boardet (FPGAen), samt et filter til at glatte PWM-pulserne ud. Det hele styres fra Labview, og vi har opfyldt de kravspecifikationer, der er beskrevet i afsnit 0.3.

For oscilloskopet har vi lavet ADC-konvertering med 8-bit opløsning for spændinger mellem 0-3.3V. Sampleraten styres med Timer1 i CTC-mode og er begrænset til mellem 10-10000 sps, mens record length er begrænset til mellem 10-1000 ADC-målinger. Vi har lavet en formel der sikrer, at den minimale record length altid passer til den valgte samplerate, så bufferen ikke risikerer at blive overskrevet. Ved at bruge to buffere og interrupts til ADC og UART-modtagelse (mens selve transmissionen kører med polling), har vi fået kontinuerte, uafbrudte målinger uden timing-fejl. UART0 bruges som en ekstra sikkerhed til at se datapakkerne i terminalen, og SPI-kommunikationen (500 kbaud) mellem MCU og FPGA er sikret med en acknowledge byte, der fanger fejl i adresse-, sync- og checksumbyten. Derudover har vi lavet en bodeplot-funktion, som kan teste lavpasfilterets frekvensrespons.

For signalgeneratoren har vi lavet en tilstandsmaskine og skifteregister på FPGA'en, der læser protokollen fra MCU'en korrekt og finder shape, amplitude og frekvens. Vi har også fået en 2-vejs handshake til at virke, og testet at fejl i sync-, adresse- og checksumbyte bliver fanget og sendt tilbage som fejlkoder via MISO. Vi kan generere 6 forskellige signalformer (konstant,

firkant, savtak, sinus, trekant og inverteret savtak), altså to flere end krævet, og sinus er lavet med en look-up table. Alle valgte parametre vises på 7-segment displayet.

Filteret er lavet som en kombination af et aktivt Sallen-Key filter og et passivt RC-filter (Butterworth), med en knækfrekvens tæt på de ønskede 100kHz - vi måler ca. 99.188kHz, så vi er kun 0.8% fra målet. Testene viser, at filteret dæmper meget lidt i vores signalfrekvensområde (24Hz-24kHz), mens det dæmper PWM-frekvensen på 390.625kHz markant, hvilket var hele formålet med filteret.

I testafsnittet (kapitel 4) har vi testet Labview-generatorpanelet, styring af samplerate og record length, SPI-kommunikationen og dataintegriteten, og det virker som forventet. Ved høje frekvenser ser vi dog noget signalforvrængning og overshoot, hvilket vi tror skyldes op-amp'ens spændingsområde og lidt måleusikkerhed - men det påvirker ikke om vi opfylder kravene.

Samlet set har vi fået oscilloskopet, signalgeneratoren og filteret til at opfylde de krav, der er stillet i rapporten, og vi har endda klaret flere af de ekstra krav (ekstra signalformer, 7-segment display, 2-vejs handshake). Testene viser, at systemet kører stabilt med forskellige samplerates, record lengths og signaltyper.

6. Koden Bilag

6.1 Koden Bilag for Oscilloskop

Oscillator Mega.c

```
1  /*
2   * Oscillator Mega.c
3   *
4   * Created: 08-06-2026 09:33:53
5   * Author : Xingjia
6   */
7  #define F_CPU 16000000UL
8  #define F_timer 250000UL
9
10 #include <avr/io.h>
11 #include <avr/interrupt.h>
12 #include <stdio.h>
13 #include <string.h>
14 #include <util/delay.h>
15
16 #include "UART.h"
17 #include "SPI.h"
18 #include "timer.h"
19 #include "ADC.h"
20
21 #define BAUD 115200
22 #define MYUBRRF (F_CPU/8/BAUD -1) //double speed
23
24 ISR (TIMER1_COMPA_vect){
25
26     ADCSRA |= (1<<ADSC);  //start conversion for next time data reading
27
28 }
29
```

```
volatile unsigned int RecordLength = 300; //default record length
volatile unsigned int ADCcounter = 0; //record how many values
volatile unsigned int ADCInd = 5; // index of buffer
volatile unsigned char Data_Ready = 0; //buffer ready

volatile unsigned char ADCValue1[1007] = {0}; //buffer 1 for Oscil
volatile unsigned char ADCValue2[1007] = {0}; //buffer 2 for Oscil

volatile unsigned char BPMode = 0; //BodePlotMode
volatile unsigned char ComStart = 0; //start Comparison
volatile unsigned char AmpValues[262] = {0}; //buffer for Amps
volatile unsigned char AmpMax = 0; //max Amp
volatile unsigned char AmpMin = 0; //min Amp
volatile unsigned char MinNow = 0; //current min Amp
volatile unsigned char MaxNow = 0; // current max Amp

ISR (ADC_vect){
      switch (BPMode)
      {
                case 0: // oscil mode
                if (ADCcounter<RecordLength){
                    ADCValue1[ADCInd] = ADCH;
                    ADCInd++;
                    ADCcounter++;
                    } else if (ADCcounter == RecordLength){
                    ADCValue1[ADCInd] = ADCH;
                    Data_Ready = 1;
                    ADCcounter++;
                    ADCInd = 5;
                    } else if (ADCcounter<(2*RecordLength) && ADCcounter>RecordLength){
                    ADCValue2[ADCInd] = ADCH;
                    ADCInd++;
                    ADCcounter++;
                    } else if (ADCcounter == (2*RecordLength)){
                    ADCValue2[ADCInd] = ADCH;
```

```
65                      Data_Ready = 2;
66                      ADCcounter = 1;
67                      ADCInd = 5;
68                  }
69                       break;
70                  case 1: //BP mode
71                       if (ComStart){
72                           if (ADCcounter<1000)
73                           {
74                               MaxNow = ADCH;
75                               MinNow = MaxNow;
76                               if  (AmpMax < MaxNow) AmpMax = MaxNow;
77                               if (AmpMin > MinNow) AmpMin = MinNow;
78                               ADCcounter++;
79                           }else if (ADCcounter == 1000)
80                           {
81                               MaxNow = ADCH;
82                               MinNow = MaxNow;
83                               if  (AmpMax < MaxNow) AmpMax = MaxNow;
84                               if (AmpMin > MinNow) AmpMin = MinNow;
85                           }
86                       }
87                       break;
88           }//end switch
89  }
90
91  volatile char ReceiveFlag = 0;
92  volatile int ind = 0;
93  volatile int Lab_length = 0;
94  volatile unsigned char GetLab[11] = {0};
95  ISR(USART1_RX_vect){
96      GetLab[ind] = UDR1;
97      ind++;
98
99      if (ind == 5){
```

```
100         switch(GetLab[4]){
101             case 0x01: Lab_length = 9;  break;
102             case 0x02: Lab_length = 11; break;
103             case 0x03: Lab_length = 7;  break;
104         }
105     }
106
107     if (Lab_length > 0 && ind >= Lab_length){
108         ReceiveFlag = 1;
109     }
110 }
111
112 //send data to labview
113 void PutStrlab(volatile unsigned char *ptr, unsigned int length){
114     for(int i = 0; i < length; i++){
115         putchUART1(*ptr);
116         ptr++;
117     }
118 }
119
120 //send data to PC terminal for testing
121 void PutStrPC(volatile unsigned char *ptr, unsigned int length){
122     for (int i=0; i<length; i++){
123         putchUART0(*ptr);
124         ptr++;
125     }
126 }
127
128 //error counter
129 unsigned int ErrorCounter = 0;
130
131 //send data to FPGA
132 unsigned char PutStrFPGA(unsigned char *fpga) {
133     PORTB &=~(1<<PB0); //wake the slave!
134     _delay_us(1);
```

```
135
136     for(int i = 0; i < 5; i++){
137         SPDR = fpga[i];
138         while(!(SPSR &(1<<SPIF))) ;
139     }
140     PORTB |=(1<<PB0);
141     //let the slave sleep
142     unsigned char ABReturn = SPDR;
143     if (!(ABReturn == 0x00)) ErrorCounter++;
144     PutStrPC(fpga,5); //control what I sent to FPGA
145     putchUART0(ABReturn);
146     putchUART0(ErrorCounter);
147     return;
148 }
149
150 //from claude prompt:transfer digital number to frequency according to the formula in the pic
151 uint32_t DigitalToFrequency(uint8_t FDigital){
152
153     uint8_t f7 = (FDigital >> 7) & 1;
154     uint8_t f6 = (FDigital >> 6) & 1;
155     uint8_t f5 = (FDigital >> 5) & 1;
156     uint8_t f4 = (FDigital >> 4) & 1;
157     uint8_t f3 = (FDigital >> 3) & 1;
158     uint8_t f2 = (FDigital >> 2) & 1;
159     uint8_t f1 = (FDigital >> 1) & 1;
160     uint8_t f0 = (FDigital >> 0) & 1;
161
162     uint16_t FNew = (f7 << 9) | (f6 << 8) | (f5 << 7) | (f4 << 6) |
163     (f3 << 4) | (f2 << 3) |
164     (f1 << 1) | (f0 << 0);
165
166     return (uint32_t)FNew * 24;
167 }
168
169 int main(void)
```

```
170 {
171     uart0_Init(MYUBRRF); //communication with PC for test
172     uart1_Init(MYUBRRF); //communication with Labview
173     spiinit(); //send message to FPGA
174     initTimer1CTC(400); //default start from 5000 sample rate
175     initADC(); //receive FPGA data
176 
177     sei();
178 
179             //data protocol for oscil
180             ADCValue1[0] = 0x55;
181             ADCValue1[1] = 0xaa;
182             ADCValue1[4] = 0x02;
183             ADCValue2[0] = 0x55;
184             ADCValue2[1] = 0xaa;
185             ADCValue2[4] = 0x02;
186             ADCValue1[3] = 0x33;
187             ADCValue1[2] = 0x01;
188             ADCValue2[3] = 0x33;
189             ADCValue2[2] = 0x01;
190             ADCValue1[305] = 0x00;
191             ADCValue1[306] = 0x00;
192             ADCValue2[305] = 0x00;
193             ADCValue2[306] = 0x00;
194     unsigned char UpdateGenPan[11] = {0}; //data protocol for labgenpan
195             UpdateGenPan[0] = 0x55;
196             UpdateGenPan[1] = 0xaa;
197             UpdateGenPan[2] = 0x00;
198             UpdateGenPan[3] = 0x0b;
199             UpdateGenPan[4] = 0x01;
200             UpdateGenPan[9] = 0x00;
201             UpdateGenPan[10] = 0x00;
202     unsigned char UpdateGenSig[5] = {0}; //data protocol for FPGA
203             UpdateGenSig[0] = 0xFF;
204             UpdateGenSig[3] = 0x00;
```

```
205             UpdateGenSig[4] = 0xAB;//Ack Byte
206
207             //data protocol for bodeplot
208             AmpValues[0] = 0x55;
209             AmpValues[1] = 0xaa;
210             AmpValues[2] = 0x01;
211             AmpValues[3] = 0x06;
212             AmpValues[4] = 0x03;
213             AmpValues[5] = 0xFF;
214             AmpValues[6] = 0xFF;
215             AmpValues[260] = 0x00;
216             AmpValues[261] = 0x00;
217
218
219     //ACK return
220
221
222     //test information
223     char *Info1="\nOCR1A Value: \n";
224     char buf1[10];
225     char *Info2="\nRecordLength Value: \n";
226     char buf2[10];
227
228     //run or stop of signal generating
229     char RunorStop = 0;
230
231     //calculate minimum record length
232     unsigned int NewSampleRate = 0;
233     unsigned int MinimumRecordLen = 0;
234
235
236
237     //test SPI med 10000 packages with 10 wrong packages
238        unsigned char testpackage[5] = {0};
239         testpackage[0] = 0xff;
```

```
240         testpackage[1] = 0x04;
241         testpackage[3] = 0x00;
242         testpackage[4] = 0xab;
243         PutStrFPGA(testpackage); //reset
244         testpackage[0] = 0xff;
245         testpackage[1] = 0x01;
246         testpackage[2] = 0xcc;
247         testpackage[3] = 0x00;
248         testpackage[4] = 0xab;
249
250     for (unsigned int i = 0; i < 5000; i++){
251         PutStrFPGA(testpackage);
252     }//send 5000 right packages
253     testpackage[3] = 0xcc;
254     for (unsigned int i = 0; i < 10; i++){
255         PutStrFPGA(testpackage);
256     }//send 10 incorrect packages
257     testpackage[3] = 0x00;
258     for (unsigned int i = 0; i < 4990; i++){
259         PutStrFPGA(testpackage);
260     }
261
262
263      while (1)
264     {
265
266         if (ReceiveFlag){
267         //clear flag
268         ReceiveFlag = 0;
269         ind = 0;
270         Lab_length = 0;
271
272               switch (GetLab[4])
273             {
274                       case 0x01: //update generator panel accordingly
```

```
275                             switch (GetLab[5]){
276                                     case 0x00: // enter data
277                                               switch (UpdateGenPan[5])
278                                                   {
279                                                   case 0x00: //shape
280                                                   UpdateGenPan[6] = GetLab[6];
281                                                   UpdateGenSig[1] = 0x01; //address byte
282                                                   UpdateGenSig[2] = GetLab[6]+1;
283                                                     break;
284                                                   case 0x01: //amp
285                                                   UpdateGenPan[7] = GetLab[6];
286                                                   UpdateGenSig[1] = 0x02; //address byte
287                                                   UpdateGenSig[2] = GetLab[6];
288                                                     break;
289                                                  case 0x02: //freq
290                                                   UpdateGenPan[8] = GetLab[6];
291                                                   UpdateGenSig[1] = 0x03; //address byte
292                                                   UpdateGenSig[2] = GetLab[6];
293                                                     break;
294                                                   }
295                                             PutStrlab(UpdateGenPan, 11); // send to panel
296                                             PutStrFPGA(UpdateGenSig); // send to FPGA with AB
byte
297                                             break; //BTN0 end
298                                     case 0x01: //select: update indicator light
299                                             if (UpdateGenPan[5] < 0x02) UpdateGenPan[5]++;
300                                             else UpdateGenPan[5] = 0x00;
301                                             PutStrlab(UpdateGenPan, 11); // send to panel
302                                             break;
303                                     case 0x02: //run or stop
304                                             RunorStop = !RunorStop;
305                                             if (RunorStop) UpdateGenSig[1] = 0x05; //address
byte
306                                             else UpdateGenSig[1] = 0x06; //address byte
307                                             PutStrFPGA(UpdateGenSig); // send to FPGA with AB
byte
```

```
308                                         break;
309                                     case 0x03: //reset
310                                         UpdateGenPan[5]=0x00;
311                                         UpdateGenPan[6]=0x00;
312                                         UpdateGenPan[7]=0x00;
313                                         UpdateGenPan[8]=0x00;
314                                         UpdateGenSig[1] = 0x09; //address byte
315                                         PutStrFPGA(UpdateGenSig);// send to FPGA with AB
byte
316                                         PutStrlab(UpdateGenPan,11);// send to panel
317                                         break;
318                                           }
319                             break; //Panel End
320                         case 0x02: //update ADC parameters accordingly
321                                 //update Sample Rate
322                                 NewSampleRate = ((uint16_t)GetLab[5] << 8) | GetLab[6];
323
324                                 OCR1A = (F_timer / NewSampleRate)-1;//update sample rate
325                                 //check OCR1A value with PC terminal
326                                 sprintf(buf1, "%d", OCR1A);
327                                 putstrUART0(Info1);
328                                 putstrUART0(buf1);
329
330                                 //update and control Record Length
331                                 RecordLength = (GetLab[7] << 8) | GetLab[8];
332                                 //get from Claude. Prompt: how to round up the result instead of
truncation
333                                 MinimumRecordLen = (70UL * NewSampleRate + (115200UL - 10UL *
NewSampleRate) - 1)
334                                 / (115200UL - 10UL * NewSampleRate);
335                                 if (RecordLength < MinimumRecordLen) RecordLength =
MinimumRecordLen;
336                                 //check RecordLength value with PC terminal
337                                 sprintf(buf2, "%d", RecordLength);
338                                 putstrUART0(Info2);
339                                 putstrUART0(buf2);
```

```
340
341                             ADCValue1[3] = (RecordLength+7);
342                             ADCValue1[2] = (RecordLength+7)>>8;
343                             ADCValue2[3] = (RecordLength+7);
344                             ADCValue2[2] = (RecordLength+7)>>8;
345                             ADCValue1[RecordLength+5] = 0x00;
346                             ADCValue1[RecordLength+6] = 0x00;
347                             ADCValue2[RecordLength+5] = 0x00;
348                             ADCValue2[RecordLength+6] = 0x00;
349                             ADCcounter = 0;
350                             ADCInd = 5;
351                       break;
352                case 0x03: //Bode Plot start
353                     // start BPmode
354                     BPMode = 1;
355                     ADCInd = 6; //op t0 259
356
357
358                     // sin + 255 AMp
359                     UpdateGenSig[1] = 0x01;
360                     UpdateGenSig[2] = 0x04;
361                     PutStrFPGA(UpdateGenSig);
362                     UpdateGenSig[1] = 0x02;
363                     UpdateGenSig[2] = 0xFF;
364                     PutStrFPGA(UpdateGenSig);
365                     UpdateGenSig[1] = 0x03;
366                        break;
367             }
368          }//receive_data end
369
370         // SigFre increment under BPmode
371         if (BPMode){
372             if (ADCcounter == 1000)
373             {
374                 if (ADCInd < 260) {
```

```
375
376                     AmpValues[ADCInd] = AmpMax-AmpMin;
377                     ADCInd++;
378
379                 }
380                 ComStart = 0;
381                 ADCcounter = 0;
382             }
383
384             if (ADCInd < 260) {
385                 if (!ComStart)
386                 {
387                     if (DigitalToFrequency(ADCInd-6)>2000)
388                     {
389                         OCR1A = 25;
390                     }else (OCR1A = F_timer / (10 * DigitalToFrequency(ADCInd-3)));
391
392                     UpdateGenSig[2] = (ADCInd - 6);
393
394                     PutStrFPGA(UpdateGenSig);
395
396                     AmpMax = 0;
397                     AmpMin = 0;
398                     ComStart = 1;
399                 }
400                 } else {
401                 BPMode = 0;
402                 ComStart = 0;
403                 ADCcounter = 0;
404                 ADCInd = 5;
405                 PutStrlab(AmpValues,262);
406                 PutStrPC(AmpValues,262);
407                 OCR1A = 200;
408             }
409         }   // end bodeplot
```

```
410
411           //when ADCValue collection is ready
412           switch (Data_Ready)
413           {
414           case 1:
415                    Data_Ready = 0;
416                    ADCValue1[RecordLength+5] = 0x00;
417                    ADCValue1[RecordLength+6] = 0x00;
418                    PutStrlab(ADCValue1, (RecordLength+7));
419              break;
420           case 2:
421                    Data_Ready = 0;
422                    ADCValue2[RecordLength+5] = 0x00;
423                    ADCValue2[RecordLength+6] = 0x00;
424                    PutStrlab(ADCValue2, (RecordLength+7));
425              break;
426           }//end data ready switch
427         }//while loop end
428  }
```

6.2 Koden Bilag for Signalgenerator

SigGenKontrolUpdate

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity SigGenKontrolUpdate is
  Port (
         Clk_SPI : in std_logic;
         Clk     : in std_logic;
         SS      : in std_logic;
         MOSI : in std_logic;
         Shape   : out std_logic_vector(7 downto 0) := x"00";
         Ampl    : out std_logic_vector(7 downto 0) := x"00";
         Freq    : out std_logic_vector(7 downto 0) := x"00";
         SigEn   : out std_logic := '0';
         reset : out std_logic;
         MisoOut : out std_logic := '0';
         fejlOutput : out std_logic_vector(11 downto 0)
         );
end SigGenKontrolUpdate;

architecture Behavioral of SigGenKontrolUpdate is

signal reset_int, packageReady, amplEN, freqEn, shapeEn: std_logic := '0';
signal SigEnStart, StopEn, fejlEN, handShake : std_logic := '0';
signal syncfejl, AdressFejl, CheckFejl, HandShake_last, loadMiso: std_logic := '0';
signal MisoReg, MisoShift, adressReg, dataReg, checkSumReg, syncReg : STD_LOGIC_VECTOR(7 downto 0) := x"00";
signal Counter_Fejl : std_logic_vector(11 downto 0) := x"000";
signal Qreg : std_logic_vector(39 downto 0) := x"0000000000";
signal Counter_32 : integer := 0;
type StateType is (Start, PackageCheck, SyncCheck, AdressS, CheckSum, RunS, HandShakeS);
signal State, nextState: StateType := Start;

begin
Reset <= Reset_int;
fejlOutput <= Counter_Fejl;
```

```
SaveReceivedPacket: process(Clk)
   begin
      if rising_edge(clk) then
         packageReady <= '0';
         handShake_last <= handShake;

         if reset_int = '1' then
            syncReg <= x"00";
            adressReg <= x"00";
            dataReg <= x"00";
            checkSumReg <= x"00";

         elsif handShake = '1' and HandShake_last = '0' then
            syncReg <= Qreg(31 downto 24);
            adressReg <= QReg(23 downto 16);
            dataReg <= QReg(15 downto 8);
            checkSumReg <= Qreg(7 downto 0);
            packageReady <= '1';
         end if;
      end if;
   end process SaveReceivedPacket;

SigENReg: process(Clk)
   begin
      if Clk'event and Clk = '1' then
         if Reset_int = '1' then
            SigEn <= '0';
         elsif SigEnStart = '1' then
            SigEn <= '1';
         elsif stopEn = '1' then
            sigEn <= '0';
         end if;
      end if;
   end process SigENReg;
```

```
ShapeReg: process(Clk)
   begin
      if Clk'event and Clk = '1' then
         if Reset_int = '1' then
            Shape <= x"00";
         elsif ShapeEn = '1' then
            Shape <= dataReg;
         end if;
      end if;
   end process ShapeReg;

AmplReg: process(Clk)
   begin
      if Clk'event and Clk = '1' then
         if Reset_int = '1' then
            Ampl <= x"00";
         elsif AmplEn = '1' then
            Ampl <= dataReg;
         end if;
      end if;
   end process AmplReg;

freqReg: process(Clk)
   begin
      if Clk'event and Clk = '1' then
         if Reset_int = '1' then
            freq <= x"00";
         elsif freqEn = '1' then
            freq <= dataReg;
         end if;
      end if;
   end process FreqReg;
```

```
SPIShiftReg: process(Clk_SPI)
   begin
      if rising_edge(Clk_SPI) then
         if reset_int = '1' then
            Qreg <= x"0000000000";
         elsif SS = '0' then
            Qreg <= Qreg(38 downto 0) & mosi;
         end if;
      end if;
   end process SPIShiftReg;

ControlMISOreg: process(Clk,ss)
   begin
      if rising_edge(Clk) then
         if reset_int = '1' then
            MisoReg <= x"00";
         elsif ss = '1' then
            MisoReg <= X"00";
         elsif syncFejl = '1' then
            MisoReg(7) <= '1';
         elsif adressFejl = '1' then
            MisoReg(6) <= '1';
         elsif checkFejl = '1' then
            MisoReg(5) <= '1';
         end if;

      end if;
   end process ControlMISOReg;
```

```
CounterReg: process(clk_SPI, SS)
   begin
      if SS = '1' then
         Counter_32 <= 0;
         handShake <= '0';
      elsif rising_edge(CLK_SPI) then
         if reset_int = '1' then
            counter_32 <= 0;
            handShake <= '0';
         else
            counter_32 <= counter_32 + 1;
            if counter_32 > 30 then
               handShake <= '1';
            end if;
         end if;
      end if;
   end process CounterReg;

MISOShiftReg: process(Clk_SPI, loadMiso)
   begin
      if LoadMiso = '1' then
         MisoShift <= MisoReg;

      elsif falling_edge(Clk_SPI) then
         if handShake = '1' then
            MisoShift <= MisoShift(6 downto 0) & '0';
            MISOout <= Misoshift(7);
         end if;
      end if;
   end process MISOShiftReg;
```

```
StateReg: process(Clk)
   begin
      if Clk'event and Clk = '1' then
         if reset_int = '1' then
            State <= start;

         else
            State <= nextState;
         end if;
      end if;
   end process StateReg;

fejlReg: process(clk)
   begin
      if rising_edge(clk) then
         if reset_int = '1' then
            Counter_Fejl <= x"000";
         elsif fejlEN = '1' then
            Counter_Fejl <= Counter_Fejl + 1;
            if counter_Fejl > x"FFF" then
               counter_Fejl <= x"000";
            end if;
         end if;
      end if;
   end process fejlReg;
```

```
StateDec: process (state, ss, adressReg, dataReg, checkSumReg, packageReady, syncReg)
begin
   nextState <= start;
   shapeEn <= '0';
   amplEn <= '0';
   freqEN <= '0';
   reset_int <= '0';
   SigEnStart <= '0';
   stopEN <= '0';
   fejlEN <= '0';
   syncFejl <= '0';
   AdressFejl <= '0';
   CheckFejl <= '0';
   LoadMiso <= '0';

   case state is
      when start =>
         if ss = '0' then
            nextState <= packageCheck;
         else
            nextState <= start;
         end if;

      when packageCheck =>
         if packageReady = '1' then
            nextState <= SyncCheck;
         else
            nextState <= packageCheck;
         end if;

      when handshakeS =>
         loadMISO <= '1';
         nextState <= start;
```

```
when syncCheck =>
   if syncReg = x"FF" then
      nextState <= AdressS;
   else
      SyncFejl <= '1';
      fejlEN <= '1';
      nextState <= handShakeS;
   end if;

when AdressS =>
   if AdressReg = x"01" then
      shapeEN <= '1';
      nextState <= checkSum;

   elsif AdressReg = x"02" then
      amplEN <= '1';
      nextState <= checkSum;

   elsif AdressReg = x"03" then
      freqEN <= '1';
      nextState <= checkSum;

   elsif adressReg = x"04" then
      reset_int <= '1';
      nextState <= start;

   elsif adressReg = x"05" then
      stopEn <= '1';
      NextState <= start;

   elsif adressReg = x"06" then
      sigEnStart <= '1';
      nextState <= start;

   else
      adressFejl <= '1';
      fejlEN <= '1';
      nextState <= HandSHakeS;
   end if;


    when checkSum =>
       if checkSumReg = X"00" then
          nextState <= RunS;
       else
          checkFejl <= '1';
          fejlEN <= '1';
          nextState <= HandShakeS;
       end if;

    when RunS =>
       sigEnStart <= '1';
       nextState <= start;
 end case;

end process StateDec;
end Behavioral;
```

SigGenDataPath

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity SigGenDatapath is
  generic( PWMinc : std_logic_vector(6 downto 0) := "0000001" );
  Port ( Reset  : in std_logic;
         Clk    : in std_logic;
         SigEN  : in std_logic;
         Shape  : in std_logic_vector(7 downto 0);
         Ampl   : in std_logic_vector(7 downto 0);
         Freq   : in std_logic_vector(7 downto 0);
         PWMOut : out std_logic);
end SigGenDatapath;

architecture Behavioral of SigGenDatapath is

signal PhaseAcc: STD_LOGIC_VECTOR(13 downto 0);
signal SigCnt, FreqCnt: std_logic_vector(11 downto 0);
signal Sig, SigSquare, SigSaw, SigSinus, SigTriangle, SigInvSaw : std_logic_vector(7 downto 0);
signal SigAmpl: std_logic_vector(6 downto 0);
signal PWMcnt: std_logic_vector(6 downto 0) := "0000000";
signal PWM, PWMwrap : std_logic;


begin

FreqDec: FreqCnt <= "00" & Freq(7 downto 6) & Freq(5 downto 4) & '0' & Freq(3 downto 2) & '0' & Freq(1 downto 0);
--laver 8 bit til 12 bit 0 0 f7 f6 f5 f4 0 f3 f2 0 f1 f0
SigCnt <= PhaseAcc(13 downto 2);
--faseforskydning
SigReg: process (Reset, Clk)
begin
  if Reset = '1' then
   PhaseAcc <= "00000000000000";

 elsif Clk'event and Clk = '1' then
    if PWMwrap = '1' then
       PhaseAcc <= PhaseAcc + ("00" & FreqCnt);
   end if;
  end if;
end process SigReg;
```

```
SinusDec : entity WORK.SinusLUT PORT MAP (clka => Clk, addra => SigCnt, douta => SigSinus);

PWMcount: process(Reset, Clk)
begin
  if Reset = '1' then PWMcnt <= "0000000";
  elsif Clk'event and Clk = '1' then
    PWMcnt <= PWMcnt + PWMinc;
  end if;
end process;

PWMdec: PWMwrap <= '1' when PWMcnt = "0000000" else '0';

SquareDec: SigSquare <= "00000000" when SigCnt < X"800" else "11111111";

SawDec: SigSaw <= SigCnt(11 downto 4);

TriangleDec:  SigTriangle <= SigCnt(10 downto 3) when SigCnt(11) = '0' else not SigCnt(10 downto 3);

InvSawDec: SigInvSaw <= not SigCnt(11 downto 4);

SigMux: Sig <= X"FF" when Shape = x"01" else
               SigSquare when Shape = x"02" else
               SigSaw when Shape = x"03" else
               SigSinus when Shape = x"04" else
               SigTriangle when shape = x"05" else
               SigInvSaw when Shape = x"06" else
               SigSinus;
```

```
AmplDec: process(Ampl, Sig)
variable MulA, MulB, MulC: std_logic_vector(15 downto 0);
begin
  MulB := X"00" & Sig;
  MulA := X"00" & Ampl;
  MulC := X"0000";
  for j in 0 to 15 loop
    if MulA(j) = '1' then
      MulC := MulC + MulB;
    end if;
    MulB := MulB(14 downto 0) & '0';
  end loop;
  SigAmpl <= MulC(15 downto 9);
end process;


PWMcomp: PWM <= '1' when PWMcnt(6 downto 0) <= SigAmpl else '0';

PWMon: PWMout <= PWM when SigEn = '1' else '0';

end Behavioral;
```

DispMux

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity DispMux is
    Port (
           Clk: in std_logic;
           Reset: in std_logic;
           ShapeButton : in std_logic;
           freqButton : in std_logic;
           amplButton : in std_logic;
           fejlButton : in std_logic;
           Disp : out  STD_LOGIC_VECTOR (15 downto 0);
           Shape : in  STD_LOGIC_VECTOR(7 downto 0);
           Ampl : in  STD_LOGIC_VECTOR(7 downto 0);
           Freq : in  STD_LOGIC_VECTOR(7 downto 0);
           Fejl : in std_logic_vector(11 downto 0));
end DispMux;

architecture Behavioral of DispMux is
type DispStateType is (ShowShape, ShowAmpl, ShowFreq, showFejl);
signal DispState : DispStateType;
begin

DispStateReg : process(Clk,Reset)
begin
if Reset = '1' then
   DispState <= ShowShape;
elsif rising_edge(Clk) then
   if ShapeButton = '1' then
      DispState <= ShowShape;
   elsif AmplButton = '1' then
      DispState <= ShowAmpl;
   elsif FreqButton = '1' then
      DispState <= ShowFreq;
   elsif fejlButton = '1' then
      DispState <= showFejl;
   end if;
end if;
end process DispStateReg;
```

```
show7seg : process (DispState, Shape, Ampl, Freq, Fejl)
begin
   case DispState is
      when ShowShape =>
         Disp <=X"50" & Shape;
      when ShowAmpl =>
         Disp <= X"A0" & Ampl;
      when ShowFreq =>
         Disp <= X"F0" & Freq;
      when ShowFejl =>
         Disp <= X"E" & Fejl;
   end case;
end process show7seg;

end Behavioral;
```

DivClk

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity DivClk is
    port ( Reset: in STD_LOGIC;      -- Global Reset (BTN1)
           Clk: in STD_LOGIC;      -- Master Clock (50 MHz)
           TimeP: in integer;      -- Time periode of the divided clock (50e6)
           Clk1: out STD_LOGIC);    -- Divided clock1 (1 Hz)
end DivClk;

architecture DivClk_arch of DivClk is
--constant MaxCnt1: integer:= 14;
signal Cnt1: integer range 0 to 25000000;  -- 24 bit counter
signal Clear1: STD_LOGIC;
signal Clk1_D: STD_LOGIC;

begin

  -- T-register with enable and async.reset
  Div1Reg: process(clk,Reset)
  begin
    if Reset = '1' then Clk1_D <= '0';      -- async. reset
    elsif (clk'event and clk ='1') then
      if Clear1= '1' then                -- enable
        Clk1_D <= not Clk1_D;
      end if;
    end if;
  end process Div1Reg;

  Div1Dec: process(Cnt1, TimeP) -- Kombinatorisk
  begin
    Clear1 <= '0';
    if Cnt1 = TimeP/2 then
      Clear1 <= '1';
    end if;
  end process Div1Dec;


  -- 24 bit up-counter with clear and async. reset
  Div1Cnt:process(clk,Reset)
  begin
    if Reset = '1' then Cnt1 <= 1;         -- async. reset
    elsif (clk'event and clk ='1') then
      if Clear1 = '1' then Cnt1 <= 1;    -- clear
      else Cnt1 <= Cnt1 + 1; end if;
    end if;
  end process Div1Cnt;

  Clk1 <= Clk1_D;

end DivClk_arch;
```

BTNdb

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity BTNdb is
  port( Reset, Clk: in std_logic;
        BTNin: in std_logic;
        BTNout: out std_logic);
end BTNdb;

architecture Behavioral of BTNdb is

constant CountMax: std_logic_vector(11 downto 0) := X"F00";

type StateType is (BTNup, wBTNup, BTNdown, wBTNdown);
signal State, nState: StateType;
signal count: std_logic_vector(11 downto 0);
signal CountEN, CountClr: std_logic;

begin

StateReg: process (Reset, Clk)
begin
  if Reset = '1' then State <= BTNup;
  elsif Clk'event and Clk = '1' then
    State <= nState;
  end if;
end process;
```

```
StateDec: process (state, BTNin, Count)
begin
  CountEN <= '0';
  CountClr <= '0';
  BTNout <= '0';
  nState <= BTNup;
  case state is
    when BTNup =>
      CountClr <= '1';
      if BTNin = '1' then
        nState <= wBTNup;
      else
        nState <= BTNup;
      end if;

    when wBTNup =>
      CountEN <= '1';
      if Count = CountMax then
        BTNout <= '1';
        nState <= BTNdown;
      else
        nState <= wBTNup;
      end if;

    when BTNdown =>
      CountClr <= '1';
      if BTNin = '0' then
        nState <= wBTNdown;
      else
        nState <= BTNdown;
      end if;

    when wBTNdown =>
      CountEN <= '1';
      if Count = CountMax then
        nState <= BTNup;
      else
        nState <= wBTNdown;
      end if;

  end case;
```

```
end process;

CountReg: process(Reset, Clk)
begin
  if Reset = '1' then Count <= X"000";
  elsif Clk'event and Clk = '1' then
    if CountClr = '1' then
      Count <= X"000";
    elsif CountEN = '1' then
      Count <= Count + '1';
    end if;
  end if;
end process;

end Behavioral;
```

SevenSeg5

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity SevenSeg5 is
    Port ( Reset, Clk: in  std_logic;
           Data :   in  std_logic_vector (15 downto 0); -- Binary data
           cat :    out std_logic_vector(6 downto 0);  -- Common cathodes
           an :     out std_logic_vector(3 downto 0)); -- Common Anodes
end SevenSeg5;

architecture SevenSeg_arch of SevenSeg5 is
signal DispCount: integer range 0 to 3;
signal DataN: std_logic_vector (3 downto 0);

begin


   DispCountReg: process(Reset, Clk)
   begin
    if Reset = '1' then
    DispCount <= 0;
    elsif Clk'event and Clk = '1' then
      if DispCount = 3
        then DispCount <= 0;
        else DispCount <= DispCount + 1; end if;
      end if;
   end process DispCountReg;
```

```
   DispCountDec: process(DispCount, Data)
   begin
     case DispCount is
       when 0 =>
         an <= "1110";         --  Display 1 activated
         DataN <= Data(3 downto 0);
       when 1 =>
         an <= "1101";         --  Display 1 activated
         DataN <= Data(7 downto 4);
       when 2 =>
         an <= "1011";         --  Display 1 activated
         DataN <= Data(11 downto 8);
       when others =>
         an <= "0111";         --  Display 1 activated
         DataN <= Data(15 downto 12);
     end case;
    end process DispCountDec;

   with DataN SELect       --  Activate segments acc. to Data
     cat <= "1000000" when "0000",    --0
            "1111001" when "0001",    --1
            "0100100" when "0010",    --2
            "0110000" when "0011",    --3
            "0011001" when "0100",    --4
            "0010010" when "0101",    --5
            "0000010" when "0110",    --6
            "1111000" when "0111",    --7
            "0000000" when "1000",    --8
            "0011000" when "1001",    --9
            "0001000" when "1010",    --A
            "0000011" when "1011",    --b
            "1000110" when "1100",    --C
            "0100001" when "1101",    --d
            "0000110" when "1110",    --E
            "0001110" when "1111",    --F
            "1111111" when others;  --blank

end SevenSeg_arch;
```

SigGenTop

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity SigGenTop is
  Port ( Clk_SPI : in std_logic;
         Clk     : in std_logic;
         SS : in std_logic;
         MOSI : in std_logic;
         BTN0 : in std_logic;
         BTN1 : in std_logic;
         BTN2 : in std_logic;
         BTN3: in std_logic;
         An      : out std_logic_vector(3 downto 0);
         Cat     : out std_logic_vector(6 downto 0);
         PWMOut : out std_logic;
         MISO : out std_logic);
end SigGenTop;

architecture Behavioral of SigGenTop is

signal DispClk, SigEn, reset, shapeButton, amplButton, FreqButton, FejlButton: std_logic;
signal Disp: std_logic_vector(15 downto 0);
signal fejl : std_logic_vector(11 downto 0);
signal Ampl, Freq, shape: std_logic_vector(7 downto 0);

begin

U1: entity WORK.DivClk
    port map(Reset => Reset, Clk => Clk, TimeP => 50e3, Clk1 => DispClk);

U2: entity WORK.SigGenKontrolUpdate
    port map(Clk => Clk, Clk_SPI => Clk_SPI, SS => SS, MOSI => MOSI, shape => shape,
    Ampl => ampl, Freq => freq, sigEn => SigEN, reset => reset, MisoOut => MISO,
    FejlOutput => Fejl);

U3: entity WORK.DispMux
    port map(Shape => shape, Ampl => Ampl, Freq => Freq, Fejl => Fejl, Disp => Disp,
    shapeButton => shapeButton, AmplButton => AmplButton, FreqButton => freqButton,
    FejlButton => FejlButton, Clk => Clk, Reset=> Reset);
```

```
U4: entity WORK.SigGenDataPath generic map (PWMinc => "0000001")
    port map(Reset => Reset, Clk => Clk, Shape => Shape, Ampl => Ampl, Freq => Freq,
    SigEN=> SigEN, PWMOut => PWMOut);

U5: entity WORK.SevenSeg5
    port map(Reset => Reset, Clk => DispClk, Data => Disp, An => An, Cat => Cat);

U6: entity WORK.BTNdb
    port map(Reset => Reset, Clk => Clk, BTNin => BTN0, BTNout => shapeButton);

U7: entity WORK.BTNdb
    port map(Reset => Reset, Clk => Clk, BTNin => BTN1, BTNout => AmplButton);

U8: entity WORK.BTNdb
    port map(Reset => Reset, Clk => Clk, BTNin => BTN2, BTNout => FreqButton);

U9: entity WORK.BTNdb
    port map(Reset => Reset, Clk => Clk, BTNin => BTN3, BTNout => FejlButton);

end Behavioral;
```